

**ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**BÁO CÁO
BẢO MẬT HỆ THỐNG THÔNG TIN**

**HỆ THỐNG QUẢN LÝ
ĐIỂM ĐẠI HỌC
NGÀNH: KHOA HỌC MÁY TÍNH**

GVHD: TS. Lê Thị Bảo Thu
—o0o—

SVTH 1: Nguyễn Huỳnh Thái Bảo 2210238
SVTH 2: Nguyễn Gia Bảo 2210230

TP. HỒ CHÍ MINH, 12/2025



Mục lục

I	GIỚI THIỆU	12
1	Giới thiệu đề tài	12
2	Mục tiêu và Phạm vi	12
2.1	Mục tiêu chính	12
2.2	Phạm vi hệ thống	13
3	Kiến trúc tổng thể	13
3.1	Lớp Presentation (Frontend)	14
3.2	Lớp Business Logic (Backend)	14
3.3	Lớp Data (Database)	14
4	Công nghệ sử dụng	15
5	Các tính năng bảo mật chính	15
II	THIẾT KẾ CƠ SỞ DỮ LIỆU	16
6	Tổng quan Schema	16
7	Chi tiết các bảng	16
7.1	Bảng FACULTIES (Khoa)	16
7.2	Bảng DEPARTMENTS (Bộ môn)	17
7.3	Bảng CLASSES (Lớp học)	18
7.4	Bảng LECTURERS (Giảng viên)	19
7.5	Bảng STUDENTS (Sinh viên)	19
7.6	Bảng RELATIVES (Người thân)	20
7.7	Bảng STUDENT_RELATIVES (Liên kết SV - Người thân)	21
7.8	Bảng COURSES (Môn học)	21
7.9	Bảng COURSE_SECTIONS (Lớp học phần)	22
7.10	Bảng ENROLLMENTS (Đăng ký học)	23

7.11	Bảng GRADES (Điểm)	23
7.12	Bảng GRADE_SUBMISSION_DEADLINES (Deadline nhập điểm)	24
7.13	Bảng SYSTEM_USERS (Tài khoản hệ thống)	25
7.14	Bảng AUDIT_LOG (Nhật ký kiểm toán)	26
8	Sơ đồ quan hệ (ERD)	27
9	Indexes	27
10	Views	28
10.1	View V_STUDENT_GRADES	28
10.2	View V_STUDENT_GPA	28
11	Tổng quan về Oracle Users	29
12	Danh sách Oracle Users	30
13	Chi tiết từng User	30
13.1	GMS_ADMIN - Schema Owner	30
13.2	GMS_APP - Application Connection User	31
13.3	GMS_STUDENT - Student Role User	31
13.4	GMS_LECTURER - Lecturer Role User	32
13.5	GMS_ACADEMIC - Academic Affairs User	32
13.6	GMS_DEAN - Dean Role User	33
13.7	GMS_DEPT_HEAD - Department Head User	34
13.8	GMS_RELATIVE - Relative Role User	34
14	Ma trận phân quyền	35
15	Flow xác thực và phân quyền	37
III	BẢO MẬT CƠ SỞ DỮ LIỆU	37
16	Giới thiệu VPD	38
16.1	Khái niệm	38
16.2	Lợi ích của VPD	38
16.3	Thành phần của VPD	38



17 Application Context	38
17.1 Tạo Context	38
17.2 Các thuộc tính Context	39
18 Security Package	39
18.1 Package Specification	39
18.2 Procedure set_user_context	40
19 Policy Functions	42
19.1 Student Policy Function	42
19.2 Grade Policy Function (SELECT)	44
19.3 Lecturer Grade Policy (INSERT/UPDATE/DELETE)	46
19.4 Relative Policy Function	47
20 Đăng ký VPD Policies	48
20.1 Xóa Policies cũ	48
20.2 Đăng ký Policy cho STUDENTS	49
20.3 Đăng ký Policies cho GRADES	49
20.4 Đăng ký Policies cho RELATIVES	51
21 Danh sách VPD Policies	52
22 VPD Bypass cho GMS_ADMIN và GMS_APP	52
23 Flow hoạt động VPD	53
24 Ví dụ minh họa VPD	53
24.1 Sinh viên STU001 xem điểm	53
24.2 Giảng viên LEC001 nhập điểm	54
24.3 Trưởng khoa xem sinh viên	54
25 Giới thiệu Password Profiles	55
26 Danh sách 5 Password Profiles	55
27 Chi tiết từng Profile	55
27.1 GMS_STUDENT_PROFILE	55
27.2 GMS_LECTURER_PROFILE	56



27.3	GMS_ADMIN_PROFILE	56
27.4	GMS_RELATIVE_PROFILE	57
27.5	GMS_SYSADMIN_PROFILE	57
28	Custom Password Verification Function	58
29	Yêu cầu mật khẩu tổng hợp	60
30	Áp dụng Profile cho Users	60
31	Giới thiệu FGA	61
32	Bảng AUDIT_LOG	61
33	Audit Handler Procedure	62
34	Danh sách 8 FGA Policies	63
35	Chi tiết FGA Policies	63
35.1	FGA cho xem điểm (SELECT)	63
35.2	FGA cho sửa điểm (UPDATE)	64
35.3	FGA cho thông tin cá nhân sinh viên	64
36	Audit Triggers cho INSERT	65
37	View Audit Trail	66
38	Giới thiệu OLS	67
39	Các thành phần OLS	67
40	Policy EXAM_SEC_POLICY	68
41	Cấu hình Levels	68
42	Cấu hình Compartments	69
43	Tạo Labels	69
44	Áp dụng Policy vào bảng EXAM_QUESTIONS	69



45 Phân quyền User theo Label	70
46 Ma trận quyền OLS	71
47 Giới thiệu	72
48 Quy tắc Column-Level Security	72
49 GRANT Statements	72
50 Kết hợp với VPD	73
51 Test Column-Level Security	73
52 Tổng kết bảo mật đa lớp	74
IV BACKEND APPLICATION	74
53 Kiến trúc Backend	75
54 Cấu trúc Project	75
55 Controllers	76
55.1 AuthController	76
55.2 StudentController	77
55.3 LecturerController	78
56 Services	79
56.1 AuthService	79
56.2 VpdContextService	80
57 Repositories	82
57.1 StudentRepository	82
57.2 GradeRepository	82
58 Entities	83
58.1 Student Entity	83
58.2 Grade Entity	84



58.3 SystemUser Entity	85
59 JWT Authentication Flow	86
60 SecurityConfig	86
61 JwtTokenProvider	88
62 JwtAuthenticationFilter	90
63 VpdContextInterceptor	92
64 CustomUserDetailsService	93
65 WebMvcConfig - Đăng ký Interceptor	94
66 Request Processing Flow	95
67 API Endpoints Summary	96
68 Môi trường Test	96
68.1 Chuẩn bị môi trường	96
68.2 Danh sách user test	101
69 Test Cases cho VPD - Student	101
69.1 TC-VPD-STU-01: Student chỉ xem được thông tin của mình	101
69.2 TC-VPD-STU-02: Student xem điểm của mình	102
69.3 TC-VPD-STU-03: Student không thể xem điểm của người khác	103
69.4 TC-VPD-STU-04: Student cập nhật thông tin liên hệ	103
70 Test Cases cho VPD - Lecturer	104
70.1 TC-VPD-LEC-01: Lecturer xem sinh viên trong lớp dạy	104
70.2 TC-VPD-LEC-02: Lecturer nhập điểm cho sinh viên	105
70.3 TC-VPD-LEC-03: Lecturer không thể nhập điểm cho section khác	106
70.4 TC-VPD-LEC-04: Lecturer cập nhật điểm đã nhập	107
71 Test Cases cho VPD - Dean	108
71.1 TC-VPD-DEAN-01: Dean xem tất cả sinh viên trong khoa	108
71.2 TC-VPD-DEAN-02: Dean xem tất cả điểm trong khoa	108

71.3	TC-VPD-DEAN-03: Dean phê duyệt điểm	109
71.4	TC-VPD-DEAN-04: Dean không thể thao tác khoa khác	110
72	Test Cases cho VPD - Relative (Phụ huynh)	111
72.1	TC-VPD-REL-01: Relative xem thông tin con em	111
72.2	TC-VPD-REL-02: Relative xem điểm con em	112
72.3	TC-VPD-REL-03: Relative không thể xem sinh viên không liên quan . . .	112
73	Test Cases cho VPD - Academic Affairs	113
73.1	TC-VPD-AA-01: Academic Affairs xem tất cả sinh viên	113
73.2	TC-VPD-AA-02: Academic Affairs quản lý đăng ký học	114
73.3	TC-VPD-AA-03: Academic Affairs không thể sửa điểm	115
74	Test Cases cho Password Policies	115
74.1	TC-PWD-01: Test password complexity	115
74.2	TC-PWD-02: Test account lockout	116
74.3	TC-PWD-03: Test password expiry	117
75	Test Cases cho Fine-Grained Auditing	118
75.1	TC-FGA-01: Audit grade viewing	118
75.2	TC-FGA-02: Audit grade modifications	119
75.3	TC-FGA-03: Audit student information access	120
76	Test Cases cho Column-Level Security	121
76.1	TC-COL-01: Student chỉ update được cột cho phép	121
76.2	TC-COL-02: Verify column grants	122
77	Test Cases tổng hợp - Security Scenarios	123
77.1	TC-SEC-01: Horizontal privilege escalation prevention	123
77.2	TC-SEC-02: Vertical privilege escalation prevention	124
77.3	TC-SEC-03: SQL injection prevention	124
77.4	TC-SEC-04: Context manipulation prevention	125
77.5	TC-SEC-05: Data exfiltration prevention	126
78	Ma trận kết quả Test	129



79 Tổng kết	130
79.1 Các mục tiêu đã đạt được	130
79.2 Kiến trúc bảo mật đa tầng	131
80 Hạn chế và Thách thức	132
80.1 Oracle Label Security (OLS)	132
80.2 Performance Considerations	132
80.3 Scalability	132
81 Kiến nghị phát triển	132
81.1 Short-term Improvements	132
81.2 Long-term Enhancements	133
82 Bài học kinh nghiệm	133
A Oracle XE Installation	133
B Connection Strings	134
C application.yml	135
D pom.xml Dependencies	136
E Thứ tự chạy script	137
F Cleanup Script	138
G Authentication Endpoints	139
H Student Endpoints	140
I Lecturer Endpoints	140
J Dean Endpoints	140
K Academic Affairs Endpoints	141
L Relative Endpoints	141



M Common Errors và Solutions	141
M.1 ORA-28000: The account is locked	141
M.2 ORA-01031: insufficient privileges	142
M.3 ORA-12458: Oracle Label Security not enabled	142
M.4 VPD policy not working	142
N Performance Tuning	143

Danh sách bảng

1	Danh sách công nghệ sử dụng trong hệ thống	15
2	Tóm tắt các tính năng bảo mật	15
3	Danh sách 14 bảng trong hệ thống	16
4	Mô tả các cột bảng FACULTIES	17
5	Danh sách 8 Oracle Users trong hệ thống	30
6	Ma trận quyền SELECT theo vai trò	35
7	Ma trận quyền INSERT/UPDATE/DELETE trên GRADES	36
8	Các thuộc tính trong gms_context	39
9	Tổng hợp 7 VPD Policies trong hệ thống	52
10	5 Password Profiles trong hệ thống GMS	55
11	Yêu cầu mật khẩu trong hệ thống GMS	60
12	8 FGA Policies trong hệ thống GMS	63
13	3 Security Levels trong OLS	68
14	2 Compartments trong OLS	69
15	Ma trận quyền truy cập OLS	71
16	Quyền UPDATE theo cột	72
17	Danh sách API Endpoints	96
18	Danh sách user dùng cho test	101
19	Ma trận kết quả test cases	129
20	Authentication API Endpoints	139
21	Student API Endpoints	140
22	Lecturer API Endpoints	140
23	Dean API Endpoints	140
24	Academic Affairs API Endpoints	141
25	Relative API Endpoints	141



Danh sách hình vẽ

1	Kiến trúc tổng thể hệ thống GMS	13
2	Sơ đồ quan hệ giữa các bảng (đơn giản hóa)	27
3	Flow xác thực và thiết lập VPD context	37
4	Flow thiết lập VPD context từ HTTP request	53
5	Mô hình bảo mật đa lớp trong hệ thống GMS	74
6	Kiến trúc Backend Spring Boot	75
7	JWT Authentication Flow	86
8	Request Processing Flow với VPD Integration	95
9	Kiến trúc bảo mật đa tầng	131

Phần I

GIỚI THIỆU

Tổng quan Hệ thống

1 Giới thiệu đề tài

Hệ thống Quản lý Điểm Đại học (University Grade Management System - GMS) là một ứng dụng web được phát triển nhằm quản lý toàn bộ quy trình quản lý điểm trong môi trường đại học. Hệ thống được thiết kế với trọng tâm đặc biệt vào **bảo mật thông tin** và **kiểm soát truy cập** đa cấp.

Trong môi trường giáo dục đại học, việc quản lý điểm đòi hỏi:

- **Tính bảo mật:** Điểm số là thông tin nhạy cảm, cần được bảo vệ khỏi truy cập trái phép
- **Tính toàn vẹn:** Đảm bảo điểm không bị sửa đổi trái phép
- **Tính sẵn sàng:** Người dùng được phép phải truy cập được thông tin khi cần
- **Kiểm soát truy cập:** Mỗi vai trò chỉ được xem/sửa dữ liệu trong phạm vi quyền hạn
- **Ghi nhật ký:** Mọi thao tác quan trọng phải được ghi lại để kiểm tra sau này

2 Mục tiêu và Phạm vi

2.1 Mục tiêu chính

1. **Triển khai Row-Level Security (RLS):** Sử dụng Oracle Virtual Private Database (VPD) để kiểm soát truy cập ở cấp độ dòng dữ liệu
2. **Triển khai Column-Level Security:** Giới hạn quyền UPDATE trên các cột cụ thể
3. **Triển khai Data-Level Security:** Sử dụng Oracle Label Security (OLS) cho dữ liệu mật (đề thi)

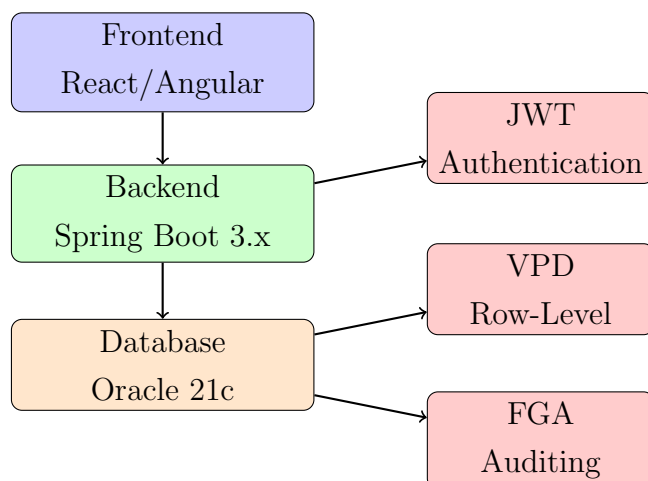
4. **Triển khai Fine-Grained Auditing (FGA):** Theo dõi mọi thao tác trên dữ liệu nhạy cảm
5. **Triển khai Password Policies:** Đảm bảo mật khẩu người dùng đạt chuẩn bảo mật
6. **Xây dựng Backend API:** Tích hợp các cơ chế bảo mật vào ứng dụng Spring Boot

2.2 Phạm vi hệ thống

- Quản lý thông tin sinh viên, giảng viên, khoa, bộ môn, lớp học
- Quản lý môn học, lớp học phần, đăng ký học
- Quản lý điểm (giữa kỳ, cuối kỳ, tổng kết)
- Quản lý deadline nhập điểm
- Quản lý quan hệ sinh viên - phụ huynh
- Ghi nhật ký kiểm toán (audit log)

3 Kiến trúc tổng thể

Hệ thống được xây dựng theo kiến trúc 3 lớp (3-tier architecture):



Hình 1: Kiến trúc tổng thể hệ thống GMS

3.1 Lớp Presentation (Frontend)

- Giao diện người dùng web
- Gửi request đến Backend API
- Hiển thị kết quả truy vấn

3.2 Lớp Business Logic (Backend)

- Spring Boot 3.x với Java 17+
- Spring Security cho JWT Authentication
- JPA/Hibernate cho ORM
- VPD Context Interceptor để thiết lập ngữ cảnh bảo mật

3.3 Lớp Data (Database)

- Oracle Database 21c
- 14 bảng dữ liệu
- 7 VPD policies cho row-level security
- 8 FGA policies cho audit
- 5 password profiles
- 1 OLS policy (Enterprise Edition only)

4 Công nghệ sử dụng

Bảng 1: *Danh sách công nghệ sử dụng trong hệ thống*

Thành phần	Công nghệ	Phiên bản
Database	Oracle Database	21c Express Edition
Backend Framework	Spring Boot	3.2.x
Security Framework	Spring Security	6.x
ORM	Hibernate/JPA	6.x
API Documentation	SpringDoc OpenAPI	2.2.0
Build Tool	Maven	3.9.x
Java Version	OpenJDK	17+
Authentication	JWT (JSON Web Token)	-
Row-Level Security	Oracle VPD	Built-in
Auditing	Oracle FGA	Built-in
Label Security	Oracle OLS	Enterprise Edition

5 Các tính năng bảo mật chính

Bảng 2: *Tóm tắt các tính năng bảo mật*

Tính năng	Công nghệ	Mô tả
Row-Level Security	Oracle VPD	Tự động lọc dữ liệu theo vai trò người dùng
Column-Level Security	GRANT	Giới hạn UPDATE trên cột cụ thể
Data Classification	Oracle OLS	Phân loại dữ liệu theo cấp độ mật
Authentication	JWT	Xác thực người dùng qua token
Password Policy	Oracle Profiles	Đảm bảo mật khẩu mạnh
Audit Trail	Oracle FGA	Ghi log mọi thao tác quan trọng

Phần II

THIẾT KẾ CƠ SỞ DỮ LIỆU

Cấu trúc Database

6 Tổng quan Schema

Hệ thống GMS sử dụng 14 bảng dữ liệu được thiết kế theo chuẩn hóa 3NF (Third Normal Form). Tất cả các bảng được tạo trong schema `GMS_ADMIN`.

Bảng 3: Danh sách 14 bảng trong hệ thống

STT	Tên bảng	Mô tả	Khóa chính
1	FACULTIES	Khoa	faculty_id
2	DEPARTMENTS	Bộ môn	department_id
3	CLASSES	Lớp học	class_id
4	LECTURERS	Giảng viên	lecturer_id
5	STUDENTS	Sinh viên	student_id
6	RELATIVES	Người thân	relative_id
7	STUDENT_RELATIVES	Liên kết SV-Người thân	(student_id, relative_id)
8	COURSES	Môn học	course_id
9	COURSE_SECTIONS	Lớp học phần	section_id
10	ENROLLMENTS	Đăng ký học	enrollment_id
11	GRADES	Điểm	grade_id
12	GRADE_SUBMISSION_DEADLINES	Deadline nộp điểm	deadline_id
13	SYSTEM_USERS	Tài khoản hệ thống	user_id
14	AUDIT_LOG	Nhật ký kiểm toán	log_id

7 Chi tiết các bảng

7.1 Bảng FACULTIES (Khoa)

Lưu trữ thông tin các khoa trong trường đại học.

```
1 CREATE TABLE FACULTIES (
```

```
2  faculty_id VARCHAR2(10) PRIMARY KEY,  
3  faculty_name VARCHAR2(100) NOT NULL,  
4  dean_id VARCHAR2(10),  
5  established_date DATE,  
6  description VARCHAR2(500),  
7  created_date DATE DEFAULT SYSDATE,  
8  updated_date DATE DEFAULT SYSDATE,  
9  CONSTRAINT fk_faculty_dean  
10     FOREIGN KEY (dean_id)  
11     REFERENCES LECTURERS(lecturer_id)  
12 );
```

Listing 1: DDL bảng FACULTIES

Bảng 4: Mô tả các cột bảng FACULTIES

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
faculty_id	VARCHAR2(10)	PK	Mã khoa
faculty_name	VARCHAR2(100)	NOT NULL	Tên khoa
dean_id	VARCHAR2(10)	FK	Mã trưởng khoa
established_date	DATE	-	Ngày thành lập
description	VARCHAR2(500)	-	Mô tả
created_date	DATE	DEFAULT SYSDATE	Ngày tạo
updated_date	DATE	DEFAULT SYSDATE	Ngày cập nhật

7.2 Bảng DEPARTMENTS (Bộ môn)

Lưu trữ thông tin các bộ môn thuộc khoa.

```
1 CREATE TABLE DEPARTMENTS (  
2     department_id VARCHAR2(10) PRIMARY KEY,  
3     department_name VARCHAR2(100) NOT NULL,  
4     faculty_id VARCHAR2(10) NOT NULL,  
5     department_head_id VARCHAR2(10),  
6     established_date DATE,  
7     description VARCHAR2(500),  
8     created_date DATE DEFAULT SYSDATE,  
9     updated_date DATE DEFAULT SYSDATE,  
10    CONSTRAINT fk_dept_faculty  
11        FOREIGN KEY (faculty_id)  
12        REFERENCES FACULTIES(faculty_id),
```

```
13 CONSTRAINT fk_dept_head
14     FOREIGN KEY (department_head_id)
15     REFERENCES LECTURERS(lecturer_id)
16 );
```

Listing 2: DDL bảng DEPARTMENTS

7.3 Bảng CLASSES (Lớp học)

Lưu trữ thông tin các lớp học (lớp sinh viên).

```
1 CREATE TABLE CLASSES (
2     class_id VARCHAR2(10) PRIMARY KEY,
3     class_name VARCHAR2(50) NOT NULL,
4     faculty_id VARCHAR2(10) NOT NULL,
5     academic_year NUMBER(4) NOT NULL,
6     homeroom_teacher_id VARCHAR2(10),
7     total_students NUMBER DEFAULT 0,
8     created_date DATE DEFAULT SYSDATE,
9     updated_date DATE DEFAULT SYSDATE,
10    CONSTRAINT fk_class_faculty
11        FOREIGN KEY (faculty_id)
12        REFERENCES FACULTIES(faculty_id),
13    CONSTRAINT fk_class_homeroom
14        FOREIGN KEY (homeroom_teacher_id)
15        REFERENCES LECTURERS(lecturer_id)
16 );
```

Listing 3: DDL bảng CLASSES

Lưu ý quan trọng: Cột `homeroom_teacher_id` tham chiếu đến giáo viên chủ nhiệm lớp. Một giảng viên có thể đồng thời là:

- Giảng viên giảng dạy (qua bảng `COURSE_SECTIONS`)
- Giáo viên chủ nhiệm (qua cột `homeroom_teacher_id`)
- Trưởng bộ môn (qua bảng `DEPARTMENTS`)
- Trưởng khoa (qua bảng `FACULTIES`)

7.4 Bảng LECTURERS (Giảng viên)

Lưu trữ thông tin giảng viên.

```
1 CREATE TABLE LECTURERS (  
2     lecturer_id VARCHAR2(10) PRIMARY KEY,  
3     first_name VARCHAR2(50) NOT NULL,  
4     last_name VARCHAR2(50) NOT NULL,  
5     date_of_birth DATE NOT NULL,  
6     gender VARCHAR2(10) CHECK (gender IN ('Male', 'Female', 'Other')),  
7     hometown VARCHAR2(100),  
8     email VARCHAR2(100) UNIQUE NOT NULL,  
9     phone_number VARCHAR2(20),  
10    contact_address VARCHAR2(200),  
11    department_id VARCHAR2(10) NOT NULL,  
12    start_date DATE NOT NULL,  
13    academic_degree VARCHAR2(50),  
14    specialization VARCHAR2(100),  
15    lecturer_status VARCHAR2(20) DEFAULT 'Active'  
16        CHECK (lecturer_status IN ('Active', 'Inactive',  
17                                   'On Leave', 'Retired')),  
18    created_date DATE DEFAULT SYSDATE,  
19    updated_date DATE DEFAULT SYSDATE,  
20    CONSTRAINT fk_lecturer_dept  
21        FOREIGN KEY (department_id)  
22        REFERENCES DEPARTMENTS(department_id)  
23 );
```

Listing 4: DDL bảng LECTURERS

7.5 Bảng STUDENTS (Sinh viên)

Lưu trữ thông tin sinh viên - đây là một trong những bảng quan trọng nhất được bảo vệ bởi VPD.

```
1 CREATE TABLE STUDENTS (  
2     student_id VARCHAR2(10) PRIMARY KEY,  
3     first_name VARCHAR2(50) NOT NULL,  
4     last_name VARCHAR2(50) NOT NULL,  
5     date_of_birth DATE NOT NULL,  
6     gender VARCHAR2(10) CHECK (gender IN ('Male', 'Female', 'Other')),  
7     hometown VARCHAR2(100),
```

```
8  ethnicity VARCHAR2(50),
9  religion VARCHAR2(50),
10 email VARCHAR2(100) UNIQUE,
11 phone_number VARCHAR2(20),
12 contact_address VARCHAR2(200),
13 class_id VARCHAR2(10) NOT NULL,
14 enrollment_date DATE DEFAULT SYSDATE,
15 student_status VARCHAR2(20) DEFAULT 'Active'
16     CHECK (student_status IN ('Active', 'Inactive',
17                               'Graduated', 'Suspended')),
18 created_date DATE DEFAULT SYSDATE,
19 updated_date DATE DEFAULT SYSDATE,
20 CONSTRAINT fk_student_class
21     FOREIGN KEY (class_id)
22     REFERENCES CLASSES(class_id)
23 );
```

Listing 5: DDL bảng STUDENTS

Column-Level Security: Sinh viên chỉ được UPDATE các cột: email, phone_number, contact_address.

7.6 Bảng RELATIVES (Người thân)

Lưu trữ thông tin phụ huynh/người thân của sinh viên.

```
1 CREATE TABLE RELATIVES (
2     relative_id VARCHAR2(10) PRIMARY KEY,
3     first_name VARCHAR2(50) NOT NULL,
4     last_name VARCHAR2(50) NOT NULL,
5     date_of_birth DATE,
6     gender VARCHAR2(10) CHECK (gender IN ('Male', 'Female', 'Other')),
7     contact_address VARCHAR2(200),
8     phone_number VARCHAR2(20) NOT NULL,
9     occupation VARCHAR2(100),
10    email VARCHAR2(100),
11    created_date DATE DEFAULT SYSDATE,
12    updated_date DATE DEFAULT SYSDATE
13 );
```

Listing 6: DDL bảng RELATIVES

7.7 Bảng STUDENT_RELATIVES (Liên kết SV - Người thân)

Bảng trung gian thể hiện quan hệ nhiều-nhiều giữa sinh viên và người thân.

```
1 CREATE TABLE STUDENT_RELATIVES (  
2     student_id VARCHAR2(10) NOT NULL,  
3     relative_id VARCHAR2(10) NOT NULL,  
4     relationship VARCHAR2(50) NOT NULL  
5     CHECK (relationship IN ('Father', 'Mother',  
6         'Guardian', 'Sibling', 'Other')),  
7     is_primary_contact VARCHAR2(1) DEFAULT 'N'  
8     CHECK (is_primary_contact IN ('Y', 'N')),  
9     created_date DATE DEFAULT SYSDATE,  
10    PRIMARY KEY (student_id, relative_id),  
11    CONSTRAINT fk_sr_student  
12        FOREIGN KEY (student_id)  
13        REFERENCES STUDENTS(student_id),  
14    CONSTRAINT fk_sr_relative  
15        FOREIGN KEY (relative_id)  
16        REFERENCES RELATIVES(relative_id)  
17 );
```

Listing 7: DDL bảng STUDENT_RELATIVES

Lưu ý: Một sinh viên có thể có nhiều người thân, và một người thân (ví dụ: phụ huynh) có thể là người thân của nhiều sinh viên (ví dụ: anh chị em).

7.8 Bảng COURSES (Môn học)

Lưu trữ thông tin các môn học.

```
1 CREATE TABLE COURSES (  
2     course_id VARCHAR2(10) PRIMARY KEY,  
3     course_name VARCHAR2(100) NOT NULL,  
4     credits NUMBER(2) NOT NULL  
5     CHECK (credits > 0 AND credits <= 10),  
6     department_id VARCHAR2(10) NOT NULL,  
7     course_type VARCHAR2(20)  
8     CHECK (course_type IN ('Mandatory', 'Elective',  
9         'Specialized')),  
10    prerequisite_course_id VARCHAR2(10),  
11    description VARCHAR2(500),  
12    created_date DATE DEFAULT SYSDATE,
```

```
13 updated_date DATE DEFAULT SYSDATE,  
14 CONSTRAINT fk_course_dept  
15     FOREIGN KEY (department_id)  
16     REFERENCES DEPARTMENTS(department_id),  
17 CONSTRAINT fk_course_prerequisite  
18     FOREIGN KEY (prerequisite_course_id)  
19     REFERENCES COURSES(course_id)  
20 );
```

Listing 8: DDL bảng COURSES

7.9 Bảng COURSE_SECTIONS (Lớp học phần)

Lưu trữ thông tin các lớp học phần (một môn học có thể có nhiều lớp học phần trong một học kỳ).

```
1 CREATE TABLE COURSE_SECTIONS (  
2     section_id VARCHAR2(15) PRIMARY KEY,  
3     course_id VARCHAR2(10) NOT NULL,  
4     lecturer_id VARCHAR2(10) NOT NULL,  
5     semester VARCHAR2(10) NOT NULL,  
6     academic_year NUMBER(4) NOT NULL,  
7     max_students NUMBER DEFAULT 50,  
8     enrolled_students NUMBER DEFAULT 0,  
9     classroom VARCHAR2(20),  
10    schedule VARCHAR2(100),  
11    section_status VARCHAR2(20) DEFAULT 'Open'  
12        CHECK (section_status IN ('Open', 'Closed',  
13                                'Completed', 'Cancelled')),  
14    created_date DATE DEFAULT SYSDATE,  
15    updated_date DATE DEFAULT SYSDATE,  
16    CONSTRAINT fk_section_course  
17        FOREIGN KEY (course_id)  
18        REFERENCES COURSES(course_id),  
19    CONSTRAINT fk_section_lecturer  
20        FOREIGN KEY (lecturer_id)  
21        REFERENCES LECTURERS(lecturer_id)  
22 );
```

Listing 9: DDL bảng COURSE_SECTIONS

7.10 Bảng ENROLLMENTS (Đăng ký học)

Lưu trữ thông tin đăng ký học của sinh viên vào các lớp học phần.

```
1 CREATE TABLE ENROLLMENTS (  
2     enrollment_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
3     student_id VARCHAR2(10) NOT NULL,  
4     section_id VARCHAR2(15) NOT NULL,  
5     enrollment_date DATE DEFAULT SYSDATE,  
6     enrollment_status VARCHAR2(20) DEFAULT 'Enrolled'  
7     CHECK (enrollment_status IN ('Enrolled', 'Dropped',  
8         'Completed', 'Failed')),  
9     created_date DATE DEFAULT SYSDATE,  
10    updated_date DATE DEFAULT SYSDATE,  
11    CONSTRAINT fk_enroll_student  
12        FOREIGN KEY (student_id)  
13        REFERENCES STUDENTS(student_id),  
14    CONSTRAINT fk_enroll_section  
15        FOREIGN KEY (section_id)  
16        REFERENCES COURSE_SECTIONS(section_id),  
17    CONSTRAINT uk_student_section  
18        UNIQUE (student_id, section_id)  
19 );
```

Listing 10: DDL bảng ENROLLMENTS

Lưu ý: Ràng buộc UNIQUE uk_student_section đảm bảo một sinh viên chỉ đăng ký một lớp học phần một lần.

7.11 Bảng GRADES (Điểm)

Bảng quan trọng nhất về mặt bảo mật - lưu trữ điểm số của sinh viên. Được bảo vệ bởi nhiều VPD policies.

```
1 CREATE TABLE GRADES (  
2     grade_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
3     enrollment_id NUMBER NOT NULL,  
4     midterm_score NUMBER(4,2)  
5     CHECK (midterm_score >= 0 AND midterm_score <= 10),  
6     final_score NUMBER(4,2)  
7     CHECK (final_score >= 0 AND final_score <= 10),  
8     total_score NUMBER(4,2)  
9     CHECK (total_score >= 0 AND total_score <= 10),
```



```
10 letter_grade VARCHAR2(2),
11 grade_status VARCHAR2(20) DEFAULT 'Pending'
12     CHECK (grade_status IN ('Pending', 'Submitted',
13                             'Approved', 'Modified')),
14 submitted_by VARCHAR2(10),
15 submitted_date DATE,
16 approved_by VARCHAR2(10),
17 approved_date DATE,
18 modified_by VARCHAR2(10),
19 modified_date DATE,
20 modification_reason VARCHAR2(500),
21 created_date DATE DEFAULT SYSDATE,
22 updated_date DATE DEFAULT SYSDATE,
23 CONSTRAINT fk_grade_enrollment
24     FOREIGN KEY (enrollment_id)
25     REFERENCES ENROLLMENTS(enrollment_id),
26 CONSTRAINT uk_enrollment_grade
27     UNIQUE (enrollment_id)
28 );
```

Listing 11: DDL bảng GRADES

Điểm quan trọng:

- Mỗi enrollment chỉ có một bản ghi điểm (UNIQUE constraint)
- Điểm từ 0-10, lưu với 2 chữ số thập phân
- Theo dõi ai nhập điểm, ai duyệt, ai sửa
- Phải ghi lý do khi sửa điểm

7.12 Bảng GRADE_SUBMISSION_DEADLINES (Deadline nhập điểm)

Quản lý deadline nhập điểm cho từng học kỳ.

```
1 CREATE TABLE GRADE_SUBMISSION_DEADLINES (
2     deadline_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
3     semester VARCHAR2(10) NOT NULL,
4     academic_year NUMBER(4) NOT NULL,
5     submission_deadline DATE NOT NULL,
6     is_active VARCHAR2(1) DEFAULT 'Y'
```

```
7      CHECK (is_active IN ('Y', 'N')),
8      created_by VARCHAR2(10),
9      created_date DATE DEFAULT SYSDATE,
10     updated_date DATE DEFAULT SYSDATE,
11     CONSTRAINT uk_semester_year
12         UNIQUE (semester, academic_year)
13 );
```

Listing 12: DDL bảng GRADE_SUBMISSION_DEADLINES

Logic nghiệp vụ:

- Giảng viên chỉ được nhập điểm TRƯỚC deadline
- Phòng đào tạo có thể sửa điểm SAU deadline

7.13 Bảng SYSTEM_USERS (Tài khoản hệ thống)

Quản lý tài khoản đăng nhập hệ thống.

```
1 CREATE TABLE SYSTEM_USERS (
2     user_id VARCHAR2(10) PRIMARY KEY,
3     username VARCHAR2(50) UNIQUE NOT NULL,
4     password_hash VARCHAR2(100) NOT NULL,
5     user_type VARCHAR2(20) NOT NULL
6         CHECK (user_type IN ('Student', 'Lecturer',
7                               'Academic_Affairs', 'Dean',
8                               'Department_Head', 'Relative', 'Admin')),
9     reference_id VARCHAR2(10) NOT NULL,
10    is_active VARCHAR2(1) DEFAULT 'Y'
11        CHECK (is_active IN ('Y', 'N')),
12    last_login DATE,
13    failed_login_attempts NUMBER DEFAULT 0,
14    account_locked VARCHAR2(1) DEFAULT 'N'
15        CHECK (account_locked IN ('Y', 'N')),
16    password_change_date DATE DEFAULT SYSDATE,
17    must_change_password VARCHAR2(1) DEFAULT 'N'
18        CHECK (must_change_password IN ('Y', 'N')),
19    created_date DATE DEFAULT SYSDATE,
20    updated_date DATE DEFAULT SYSDATE
21 );
```

Listing 13: DDL bảng SYSTEM_USERS

7 loại user _type:

1. **Student:** Sinh viên
2. **Lecturer:** Giảng viên
3. **Academic _Affairs:** Phòng đào tạo
4. **Dean:** Trưởng khoa
5. **Department _Head:** Trưởng bộ môn
6. **Relative:** Phụ huynh/Người thân
7. **Admin:** Quản trị viên

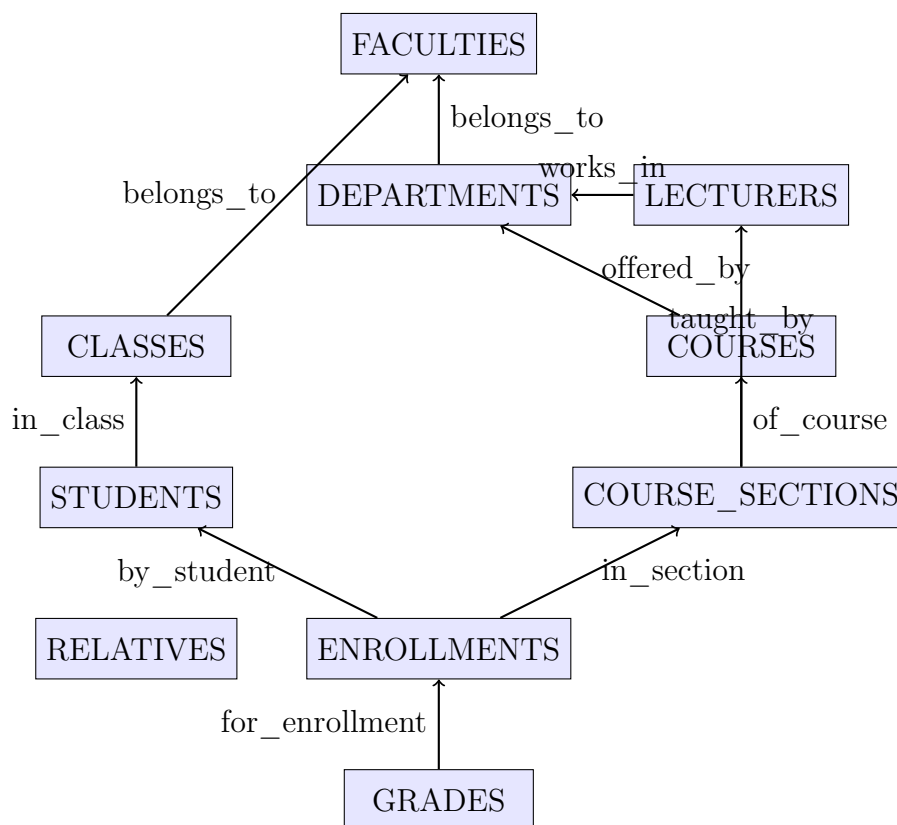
7.14 Bảng AUDIT_LOG (Nhật ký kiểm toán)

Ghi lại tất cả các thao tác quan trọng trên hệ thống.

```
1 CREATE TABLE AUDIT_LOG (  
2     log_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
3     table_name VARCHAR2(30) NOT NULL,  
4     operation VARCHAR2(10) NOT NULL  
5         CHECK (operation IN ('INSERT', 'UPDATE',  
6                             'DELETE', 'SELECT')),  
7     user_id VARCHAR2(10),  
8     username VARCHAR2(50),  
9     record_id VARCHAR2(50),  
10    old_values CLOB,  
11    new_values CLOB,  
12    operation_date DATE DEFAULT SYSDATE,  
13    ip_address VARCHAR2(50),  
14    session_id VARCHAR2(100)  
15 );
```

Listing 14: DDL bảng AUDIT_LOG

8 Sơ đồ quan hệ (ERD)



Hình 2: Sơ đồ quan hệ giữa các bảng (đơn giản hóa)

9 Indexes

Các index được tạo để tối ưu hiệu năng truy vấn:

```

1 -- Index cho foreign keys
2 CREATE INDEX idx_student_class ON STUDENTS(class_id);
3 CREATE INDEX idx_lecturer_dept ON LECTURERS(department_id);
4 CREATE INDEX idx_course_dept ON COURSES(department_id);
5 CREATE INDEX idx_section_course ON COURSE_SECTIONS(course_id);
6 CREATE INDEX idx_section_lecturer ON COURSE_SECTIONS(lecturer_id);
7 CREATE INDEX idx_enrollment_student ON ENROLLMENTS(student_id);
8 CREATE INDEX idx_enrollment_section ON ENROLLMENTS(section_id);
9
10 -- Index cho queries ph bin
11 CREATE INDEX idx_section_semester ON COURSE_SECTIONS(semester, academic_year);

```

```
12 CREATE INDEX idx_audit_date ON AUDIT_LOG(operation_date);
```

Listing 15: Các index trong hệ thống

Lưu ý: Không tạo index cho các cột đã có UNIQUE constraint (như email, enrollment_id trong GRADES) vì Oracle tự động tạo unique index.

10 Views

Hệ thống cung cấp các views để truy vấn dữ liệu thuận tiện:

10.1 View V_STUDENT_GRADES

```
1 CREATE OR REPLACE VIEW V_STUDENT_GRADES AS
2 SELECT
3     s.student_id,
4     s.first_name || ' ' || s.last_name AS student_name,
5     c.course_id,
6     c.course_name,
7     c.credits,
8     cs.semester,
9     cs.academic_year,
10    g.midterm_score,
11    g.final_score,
12    g.total_score,
13    g.letter_grade,
14    g.grade_status
15 FROM STUDENTS s
16 JOIN ENROLLMENTS e ON s.student_id = e.student_id
17 JOIN COURSE_SECTIONS cs ON e.section_id = cs.section_id
18 JOIN COURSES c ON cs.course_id = c.course_id
19 LEFT JOIN GRADES g ON e.enrollment_id = g.enrollment_id;
```

Listing 16: View V_STUDENT_GRADES

10.2 View V_STUDENT_GPA

```
1 CREATE OR REPLACE VIEW V_STUDENT_GPA AS
2 SELECT
3     s.student_id,
```

```
4      s.first_name || ' ' || s.last_name AS student_name,  
5      cs.semester,  
6      cs.academic_year,  
7      ROUND(SUM(g.total_score * c.credits) / SUM(c.credits), 2)  
8          AS semester_gpa,  
9      ROUND(AVG(g.total_score), 2) AS cumulative_gpa  
10 FROM STUDENTS s  
11 JOIN ENROLLMENTS e ON s.student_id = e.student_id  
12 JOIN COURSE_SECTIONS cs ON e.section_id = cs.section_id  
13 JOIN COURSES c ON cs.course_id = c.course_id  
14 JOIN GRADES g ON e.enrollment_id = g.enrollment_id  
15 WHERE g.grade_status = 'Approved'  
16 GROUP BY s.student_id, s.first_name, s.last_name,  
17          cs.semester, cs.academic_year;
```

Listing 17: View V_STUDENT_GPA

Người dùng Oracle

11 Tổng quan về Oracle Users

Hệ thống GMS sử dụng 8 Oracle users với các vai trò và quyền hạn khác nhau. Việc phân tách users này cho phép:

- Kiểm soát truy cập ở mức database
- Áp dụng VPD policies khác nhau cho từng user
- Áp dụng password profiles khác nhau
- Tracking audit theo từng user

12 Danh sách Oracle Users

Bảng 5: *Danh sách 8 Oracle Users trong hệ thống*

STT	Username	Vai trò	Mô tả
1	GMS_ADMIN	Schema Owner	Sở hữu tất cả objects
2	GMS_APP	Application User	Backend kết nối DB
3	GMS_STUDENT	Student Role	Kiểm tra quyền sinh viên
4	GMS_LLECTURER	Lecturer Role	Kiểm tra quyền giảng viên
5	GMS_ACADEMIC	Academic Affairs	Phòng đào tạo
6	GMS_DEAN	Dean Role	Trưởng khoa
7	GMS_DEPT_HEAD	Dept Head Role	Trưởng bộ môn
8	GMS_RELATIVE	Relative Role	Phụ huynh

13 Chi tiết từng User

13.1 GMS_ADMIN - Schema Owner

```
1 CREATE USER GMS_ADMIN IDENTIFIED BY "Admin@2024#Secure"
2 DEFAULT TABLESPACE USERS
3 TEMPORARY TABLESPACE TEMP
4 QUOTA UNLIMITED ON USERS;
5
6 -- Grant full privileges
7 GRANT CONNECT, RESOURCE, DBA TO GMS_ADMIN;
8 GRANT CREATE SESSION TO GMS_ADMIN;
9 GRANT CREATE TABLE TO GMS_ADMIN;
10 GRANT CREATE VIEW TO GMS_ADMIN;
11 GRANT CREATE PROCEDURE TO GMS_ADMIN;
12 GRANT CREATE SEQUENCE TO GMS_ADMIN;
13 GRANT CREATE TRIGGER TO GMS_ADMIN;
14 GRANT CREATE TYPE TO GMS_ADMIN;
15 GRANT CREATE SYNONYM TO GMS_ADMIN;
16 GRANT CREATE MATERIALIZED VIEW TO GMS_ADMIN;
17 GRANT CREATE DATABASE LINK TO GMS_ADMIN;
18 GRANT CREATE JOB TO GMS_ADMIN;
```

Listing 18: Tạo user GMS_ADMIN

Đặc điểm:

- Sở hữu tất cả 14 bảng, views, packages, procedures
- Có quyền DBA để quản lý toàn bộ schema
- **Bypass VPD:** VPD policies không áp dụng cho schema owner
- Dùng để quản trị và maintenance

13.2 GMS_APP - Application Connection User

```
1 CREATE USER GMS_APP IDENTIFIED BY "App@2024#Connect"
2 DEFAULT TABLESPACE USERS
3 TEMPORARY TABLESPACE TEMP;
4
5 GRANT CONNECT TO GMS_APP;
6 GRANT CREATE SESSION TO GMS_APP;
```

Listing 19: Tạo user GMS_APP

Đặc điểm:

- Backend Spring Boot sử dụng user này để kết nối database
- Chỉ có quyền CONNECT và CREATE SESSION
- **Bypass VPD:** Để backend có thể truy cập tất cả data
- VPD context được set per-request dựa trên authenticated user

13.3 GMS_STUDENT - Student Role User

```
1 CREATE USER GMS_STUDENT IDENTIFIED BY "Student@2024"
2 DEFAULT TABLESPACE USERS
3 TEMPORARY TABLESPACE TEMP;
4
5 GRANT CONNECT TO GMS_STUDENT;
6 GRANT CREATE SESSION TO GMS_STUDENT;
7
8 -- Column-level UPDATE permission
9 GRANT UPDATE (email, phone_number, contact_address)
```



```
10 ON gms_admin.STUDENTS TO GMS_STUDENT;
```

Listing 20: Tạo user GMS_STUDENT

Quyền truy cập:

- SELECT trên STUDENTS: Chỉ bản ghi của mình
- SELECT trên GRADES: Chỉ điểm của mình
- UPDATE trên STUDENTS: Chỉ 3 cột (email, phone, address)
- Không có INSERT, DELETE

13.4 GMS_LECTURER - Lecturer Role User

```
1 CREATE USER GMS_LECTURER IDENTIFIED BY "Lecturer@2024"
2 DEFAULT TABLESPACE USERS
3 TEMPORARY TABLESPACE TEMP;
4
5 GRANT CONNECT TO GMS_LECTURER;
6 GRANT CREATE SESSION TO GMS_LECTURER;
7
8 -- Column-level UPDATE permission
9 GRANT UPDATE (email, phone_number, contact_address)
10 ON gms_admin.LECTURERS TO GMS_LECTURER;
```

Listing 21: Tạo user GMS_LECTURER

Quyền truy cập:

- SELECT trên STUDENTS: Sinh viên lớp chủ nhiệm + sinh viên đăng ký môn dạy
- SELECT trên GRADES: Điểm môn mình dạy + điểm sinh viên lớp chủ nhiệm
- INSERT/UPDATE trên GRADES: Điểm môn mình dạy, TRƯỚC deadline
- UPDATE trên LECTURERS: Chỉ 3 cột của chính mình

13.5 GMS_ACADEMIC - Academic Affairs User

```
1 CREATE USER GMS_ACADEMIC IDENTIFIED BY "Academic@2024"  
2 DEFAULT TABLESPACE USERS  
3 TEMPORARY TABLESPACE TEMP;  
4  
5 GRANT CONNECT TO GMS_ACADEMIC;  
6 GRANT CREATE SESSION TO GMS_ACADEMIC;
```

Listing 22: Tạo user GMS_ACADEMIC

Quyền truy cập:

- SELECT trên tất cả bảng: Full access
- UPDATE trên GRADES: Có thể sửa SAU deadline
- Quản lý GRADE_SUBMISSION_DEADLINES
- Full access vào AUDIT_LOG để kiểm tra

13.6 GMS_DEAN - Dean Role User

```
1 CREATE USER GMS_DEAN IDENTIFIED BY "Dean@2024"  
2 DEFAULT TABLESPACE USERS  
3 TEMPORARY TABLESPACE TEMP;  
4  
5 GRANT CONNECT TO GMS_DEAN;  
6 GRANT CREATE SESSION TO GMS_DEAN;
```

Listing 23: Tạo user GMS_DEAN

Quyền truy cập:

- SELECT trên STUDENTS: Sinh viên trong khoa của mình
- SELECT trên GRADES: Điểm sinh viên trong khoa
- SELECT trên LECTURERS: Giảng viên trong khoa
- SELECT trên FACULTIES: Chỉ khoa của mình
- Không có quyền INSERT/UPDATE/DELETE điểm

13.7 GMS_DEPT_HEAD - Department Head User

```
1 CREATE USER GMS_DEPT_HEAD IDENTIFIED BY "DeptHead@2024"  
2 DEFAULT TABLESPACE USERS  
3 TEMPORARY TABLESPACE TEMP;  
4  
5 GRANT CONNECT TO GMS_DEPT_HEAD;  
6 GRANT CREATE SESSION TO GMS_DEPT_HEAD;
```

Listing 24: Tạo user GMS_DEPT_HEAD

Quyền truy cập:

- SELECT trên GRADES: Điểm các môn trong bộ môn
- SELECT trên COURSES: Môn học trong bộ môn
- SELECT trên DEPARTMENTS: Chỉ bộ môn của mình
- Giám sát chất lượng giảng dạy trong bộ môn

13.8 GMS_RELATIVE - Relative Role User

```
1 CREATE USER GMS_RELATIVE IDENTIFIED BY "Relative@2024"  
2 DEFAULT TABLESPACE USERS  
3 TEMPORARY TABLESPACE TEMP;  
4  
5 GRANT CONNECT TO GMS_RELATIVE;  
6 GRANT CREATE SESSION TO GMS_RELATIVE;  
7  
8 -- Column-level UPDATE permission  
9 GRANT UPDATE (email, phone_number, contact_address)  
10 ON gms_admin.RELATIVES TO GMS_RELATIVE;
```

Listing 25: Tạo user GMS_RELATIVE

Quyền truy cập:

- SELECT trên STUDENTS: Chỉ con em (qua STUDENT_RELATIVES)
- SELECT trên GRADES: Chỉ điểm con em
- SELECT trên RELATIVES: Chỉ bản ghi của mình

- UPDATE trên RELATIVES: Chỉ 3 cột của mình
- Không có quyền INSERT/UPDATE/DELETE điểm

14 Ma trận phân quyền

Bảng 6: Ma trận quyền *SELECT* theo vai trò

Bảng	STU	LEC	REL	DEPT	DEAN	ACAD	ADMIN
STUDENTS	Own	Class+	Child	-	Fac	All	All
GRADES	Own	Teach	Child	Dept	Fac	All	All
LECTURERS	-	Own	-	Dept	Fac	All	All
RELATIVES	-	-	Own	-	-	All	All
COURSES	-	Teach	-	Dept	Fac	All	All
ENROLLMENTS	Own	Teach	Child	Dept	Fac	All	All
DEPARTMENTS	-	Own	-	Own	Fac	All	All
FACULTIES	-	Own	-	-	Own	All	All

Chú thích:

- **Own:** Chỉ bản ghi của chính mình
- **Class+:** Sinh viên lớp chủ nhiệm + sinh viên đăng ký môn dạy
- **Child:** Con em (qua bảng STUDENT_RELATIVES)
- **Teach:** Dữ liệu liên quan đến môn mình dạy
- **Dept:** Dữ liệu trong bộ môn
- **Fac:** Dữ liệu trong khoa
- **All:** Toàn bộ dữ liệu
- **-:** Không có quyền

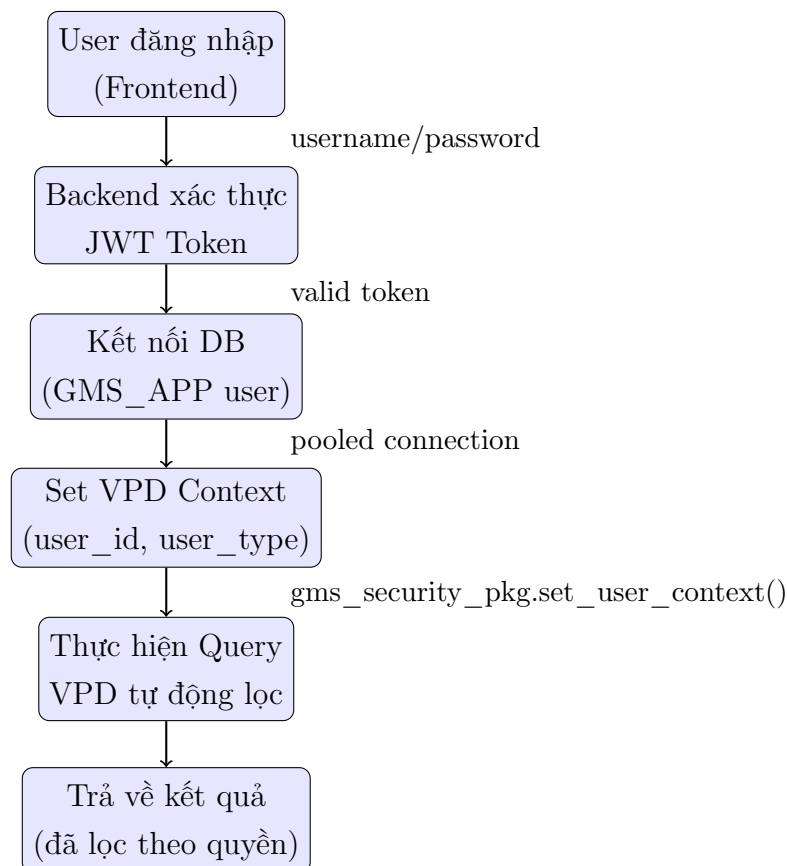
Bảng 7: *Ma trận quyền INSERT/UPDATE/DELETE trên GRADES*

Vai trò	INSERT	UPDATE	DELETE
Student	-	-	-
Lecturer	Before DL	Before DL	Before DL
Relative	-	-	-
Dept_Head	-	-	-
Dean	-	-	-
Academic_Affairs	All	After DL	All
Admin	All	All	All

Chú thích:

- **Before DL:** Chỉ trước deadline
- **After DL:** Có thể thực hiện sau deadline
- **All:** Không giới hạn
- **-:** Không có quyền

15 Flow xác thực và phân quyền



Hình 3: Flow xác thực và thiết lập VPD context

Phần III

BẢO MẬT CƠ SỞ DỮ LIỆU

Virtual Private Database (VPD)

16 Giới thiệu VPD

16.1 Khái niệm

Virtual Private Database (VPD) là một tính năng bảo mật nâng cao của Oracle Database cho phép kiểm soát truy cập ở cấp độ **dòng dữ liệu (row-level)**. VPD tự động thêm điều kiện WHERE vào các câu lệnh SQL dựa trên ngữ cảnh của người dùng.

16.2 Lợi ích của VPD

1. **Bảo mật trong suốt:** Application code không cần thay đổi, VPD tự động lọc data
2. **Không thể bypass:** Policies áp dụng ở database layer, không thể bỏ qua từ application
3. **Hiệu năng cao:** Oracle optimizer tích hợp predicates vào execution plan
4. **Tập trung quản lý:** Policies định nghĩa một lần, áp dụng cho mọi truy vấn
5. **Audit friendly:** Dễ dàng theo dõi và kiểm tra

16.3 Thành phần của VPD

- **Application Context:** Lưu trữ thông tin session (user_id, user_type, etc.)
- **Policy Function:** Hàm trả về WHERE clause dựa trên context
- **VPD Policy:** Liên kết policy function với table và statement types

17 Application Context

17.1 Tạo Context

Context `gms_context` được sử dụng để lưu trữ thông tin phiên làm việc:

```
1 CREATE OR REPLACE CONTEXT gms_context
2   USING gms_admin.gms_security_pkg;
```

Listing 26: Tạo Application Context

17.2 Các thuộc tính Context

Bảng 8: Các thuộc tính trong gms_context

Thuộc tính	Mô tả	Ví dụ
user_id	ID người dùng	STU001, LEC001
user_type	Loại người dùng	Student, Lecturer
class_id	Mã lớp (for Student)	CS2024A
department_id	Mã bộ môn (for Lecturer/DeptHead)	DEPT001
faculty_id	Mã khoa (for Dean/Lecturer)	FAC001

18 Security Package

Package gms_security_pkg chứa tất cả logic bảo mật:

18.1 Package Specification

```
1 CREATE OR REPLACE PACKAGE gms_security_pkg AS
2     -- Set user context
3     PROCEDURE set_user_context(
4         p_user_id VARCHAR2,
5         p_user_type VARCHAR2
6     );
7
8     -- Clear user context
9     PROCEDURE clear_user_context;
10
11    -- Get current user type
12    FUNCTION get_user_type RETURN VARCHAR2;
13
14    -- Get current user id
15    FUNCTION get_user_id RETURN VARCHAR2;
16
17    -- Check if grade submission deadline has passed
18    FUNCTION is_after_deadline(
19        p_semester VARCHAR2,
20        p_academic_year NUMBER
21    ) RETURN BOOLEAN;
```



```
22
23  -- Policy functions for different tables
24  FUNCTION student_policy(
25      p_schema VARCHAR2,
26      p_object VARCHAR2
27  ) RETURN VARCHAR2;
28
29  FUNCTION grade_policy(
30      p_schema VARCHAR2,
31      p_object VARCHAR2
32  ) RETURN VARCHAR2;
33
34  FUNCTION relative_policy(
35      p_schema VARCHAR2,
36      p_object VARCHAR2
37  ) RETURN VARCHAR2;
38
39  FUNCTION lecturer_grade_policy(
40      p_schema VARCHAR2,
41      p_object VARCHAR2
42  ) RETURN VARCHAR2;
43
44  FUNCTION department_policy(
45      p_schema VARCHAR2,
46      p_object VARCHAR2
47  ) RETURN VARCHAR2;
48
49  FUNCTION faculty_policy(
50      p_schema VARCHAR2,
51      p_object VARCHAR2
52  ) RETURN VARCHAR2;
53  END gms_security_pkg;
54  /
```

Listing 27: Package Specification gms_security_pkg

18.2 Procedure set_user_context

Procedure này được gọi bởi backend khi bắt đầu mỗi request để thiết lập ngữ cảnh bảo mật:

```
1 PROCEDURE set_user_context(
```



```
2      p_user_id VARCHAR2,
3      p_user_type VARCHAR2
4  ) IS
5      v_dept_id VARCHAR2(10);
6      v_faculty_id VARCHAR2(10);
7      v_class_id VARCHAR2(10);
8  BEGIN
9      -- Set basic context
10     DBMS_SESSION.SET_CONTEXT('gms_context', 'user_id', p_user_id);
11     DBMS_SESSION.SET_CONTEXT('gms_context', 'user_type', p_user_type);
12
13     -- Set additional context based on user type
14     IF p_user_type = 'Student' THEN
15         -- Get student's class
16         SELECT class_id INTO v_class_id
17         FROM gms_admin.STUDENTS
18         WHERE student_id = p_user_id;
19         DBMS_SESSION.SET_CONTEXT('gms_context', 'class_id', v_class_id);
20
21         -- Get student's faculty
22         SELECT c.faculty_id INTO v_faculty_id
23         FROM gms_admin.STUDENTS s
24         JOIN gms_admin.CLASSES c ON s.class_id = c.class_id
25         WHERE s.student_id = p_user_id;
26         DBMS_SESSION.SET_CONTEXT('gms_context', 'faculty_id', v_faculty_id);
27
28     ELSIF p_user_type = 'Lecturer' THEN
29         -- Get lecturer's department
30         SELECT department_id INTO v_dept_id
31         FROM gms_admin.LECTURERS
32         WHERE lecturer_id = p_user_id;
33         DBMS_SESSION.SET_CONTEXT('gms_context', 'department_id', v_dept_id);
34
35         -- Get lecturer's faculty
36         SELECT d.faculty_id INTO v_faculty_id
37         FROM gms_admin.LECTURERS l
38         JOIN gms_admin.DEPARTMENTS d ON l.department_id = d.department_id
39         WHERE l.lecturer_id = p_user_id;
40         DBMS_SESSION.SET_CONTEXT('gms_context', 'faculty_id', v_faculty_id);
41
42     ELSIF p_user_type = 'Department_Head' THEN
```

```
43      -- Get department head's department
44      SELECT department_id INTO v_dept_id
45      FROM gms_admin.DEPARTMENTS
46      WHERE department_head_id = p_user_id;
47      DBMS_SESSION.SET_CONTEXT('gms_context', 'department_id', v_dept_id);
48
49      -- Get faculty
50      SELECT faculty_id INTO v_faculty_id
51      FROM gms_admin.DEPARTMENTS
52      WHERE department_head_id = p_user_id;
53      DBMS_SESSION.SET_CONTEXT('gms_context', 'faculty_id', v_faculty_id);
54
55      ELSIF p_user_type = 'Dean' THEN
56          -- Get dean's faculty
57          SELECT faculty_id INTO v_faculty_id
58          FROM gms_admin.FACULTIES
59          WHERE dean_id = p_user_id;
60          DBMS_SESSION.SET_CONTEXT('gms_context', 'faculty_id', v_faculty_id);
61      END IF;
62
63  EXCEPTION
64      WHEN NO_DATA_FOUND THEN
65          NULL; -- User not found in respective table
66      WHEN OTHERS THEN
67          RAISE;
68  END set_user_context;
```

Listing 28: Procedure set_user_context

19 Policy Functions

19.1 Student Policy Function

Policy này kiểm soát quyền truy cập vào bảng STUDENTS:

```
1 FUNCTION student_policy(
2     p_schema VARCHAR2,
3     p_object VARCHAR2
4 ) RETURN VARCHAR2 IS
5     v_user_type VARCHAR2(50);
6     v_user_id VARCHAR2(10);
```



```
7      v_predicate VARCHAR2(4000);
8      v_current_user VARCHAR2(128);
9  BEGIN
10     -- Bypass VPD for schema owner and application user
11     v_current_user := SYS_CONTEXT('USERENV', 'SESSION_USER');
12     IF v_current_user = 'GMS_ADMIN' OR v_current_user = 'GMS_APP'
13        OR v_current_user = p_schema THEN
14        RETURN '1=1'; -- Admin and App can see all
15     END IF;
16
17     v_user_type := SYS_CONTEXT('gms_context', 'user_type');
18     v_user_id := SYS_CONTEXT('gms_context', 'user_id');
19
20     IF v_user_type = 'Student' THEN
21        -- Students can only see their own record
22        v_predicate := 'student_id = ''' || v_user_id || '''';
23
24     ELSIF v_user_type = 'Academic_Affairs' OR v_user_type = 'Admin' THEN
25        -- Academic Affairs can see all students
26        v_predicate := '1=1';
27
28     ELSIF v_user_type = 'Dean' THEN
29        -- Deans can see students in their faculty
30        v_predicate := 'class_id IN (SELECT class_id FROM gms_admin.CLASSES
31                                WHERE faculty_id = '''
32                                || SYS_CONTEXT('gms_context', 'faculty_id') || ''')';
33
34     ELSIF v_user_type = 'Lecturer' THEN
35        -- Lecturers can see students:
36        -- 1. In their homeroom class
37        -- 2. Enrolled in courses they teach
38        v_predicate := '(class_id IN (SELECT class_id FROM gms_admin.CLASSES
39                                WHERE homeroom_teacher_id = ''' || v_user_id || ''')
40                                OR student_id IN (SELECT e.student_id
41                                FROM gms_admin.ENROLLMENTS e
42                                JOIN gms_admin.COURSE_SECTIONS cs
43                                ON e.section_id = cs.section_id
44                                WHERE cs.lecturer_id = ''' || v_user_id || '''))';
45     ELSE
46        -- No access
47        v_predicate := '1=0';
```

```
48     END IF;
49
50     RETURN v_predicate;
51 END student_policy;
```

Listing 29: Function student_policy

Logic chi tiết:

- **Student:** Chỉ thấy bản ghi của chính mình (`student_id = 'STU001'`)
- **Lecturer:** Thấy sinh viên lớp chủ nhiệm + sinh viên đăng ký môn mình dạy
- **Dean:** Thấy tất cả sinh viên trong khoa
- **Academic_Affairs/Admin:** Thấy tất cả sinh viên
- **Khác:** Không thấy gì (`1=0`)

19.2 Grade Policy Function (SELECT)

Policy này kiểm soát quyền xem điểm:

```
1 FUNCTION grade_policy(
2     p_schema VARCHAR2,
3     p_object VARCHAR2
4 ) RETURN VARCHAR2 IS
5     v_user_type VARCHAR2(50);
6     v_user_id VARCHAR2(10);
7     v_predicate VARCHAR2(4000);
8     v_current_user VARCHAR2(128);
9 BEGIN
10     -- Bypass VPD for schema owner and application user
11     v_current_user := SYS_CONTEXT('USERENV', 'SESSION_USER');
12     IF v_current_user = 'GMS_ADMIN' OR v_current_user = 'GMS_APP'
13        OR v_current_user = p_schema THEN
14         RETURN '1=1';
15     END IF;
16
17     v_user_type := SYS_CONTEXT('gms_context', 'user_type');
18     v_user_id := SYS_CONTEXT('gms_context', 'user_id');
19
20     IF v_user_type = 'Student' THEN
```

```
21      -- Students can only see their own grades
22      v_predicate := 'enrollment_id IN (SELECT enrollment_id
23                  FROM gms_admin.ENROLLMENTS
24                  WHERE student_id = ''' || v_user_id || ''')';
25
26      ELSIF v_user_type = 'Relative' THEN
27          -- Relatives can see their children's grades
28          v_predicate := 'enrollment_id IN (SELECT e.enrollment_id
29                  FROM gms_admin.ENROLLMENTS e
30                  JOIN gms_admin.STUDENT_RELATIVES sr
31                  ON e.student_id = sr.student_id
32                  WHERE sr.relative_id = ''' || v_user_id || ''')';
33
34      ELSIF v_user_type = 'Lecturer' THEN
35          -- Lecturers can see grades for:
36          -- 1. Courses they teach
37          -- 2. Students in their homeroom class
38          v_predicate := '(enrollment_id IN (SELECT e.enrollment_id
39                  FROM gms_admin.ENROLLMENTS e
40                  JOIN gms_admin.COURSE_SECTIONS cs
41                  ON e.section_id = cs.section_id
42                  WHERE cs.lecturer_id = ''' || v_user_id || ''')
43                  OR enrollment_id IN (SELECT e.enrollment_id
44                  FROM gms_admin.ENROLLMENTS e
45                  JOIN gms_admin.STUDENTS s ON e.student_id = s.student_id
46                  JOIN gms_admin.CLASSES c ON s.class_id = c.class_id
47                  WHERE c.homeroom_teacher_id = '''
48                  || v_user_id || '''))';
49
50      ELSIF v_user_type = 'Department_Head' THEN
51          -- Department heads see grades for courses in their department
52          v_predicate := 'enrollment_id IN (SELECT e.enrollment_id
53                  FROM gms_admin.ENROLLMENTS e
54                  JOIN gms_admin.COURSE_SECTIONS cs
55                  ON e.section_id = cs.section_id
56                  JOIN gms_admin.COURSES c ON cs.course_id = c.course_id
57                  WHERE c.department_id = '''
58                  || SYS_CONTEXT('gms_context', 'department_id') || ''')';
59
60      ELSIF v_user_type = 'Dean' THEN
61          -- Deans see grades for students in their faculty
```

```
62     v_predicate := 'enrollment_id IN (SELECT e.enrollment_id
63         FROM gms_admin.ENROLLMENTS e
64         JOIN gms_admin.STUDENTS s ON e.student_id = s.student_id
65         JOIN gms_admin.CLASSES cl ON s.class_id = cl.class_id
66         WHERE cl.faculty_id = '''
67         || SYS_CONTEXT('gms_context', 'faculty_id') || ''')';
68
69     ELSIF v_user_type = 'Academic_Affairs' OR v_user_type = 'Admin' THEN
70         v_predicate := '1=1';
71
72     ELSE
73         v_predicate := '1=0';
74     END IF;
75
76     RETURN v_predicate;
77 END grade_policy;
```

Listing 30: Function grade_policy

19.3 Lecturer Grade Policy (INSERT/UPDATE/DELETE)

Policy này kiểm soát quyền thêm/sửa/xóa điểm với điều kiện deadline:

```
1 FUNCTION lecturer_grade_policy(
2     p_schema VARCHAR2,
3     p_object VARCHAR2
4 ) RETURN VARCHAR2 IS
5     v_user_type VARCHAR2(50);
6     v_user_id VARCHAR2(10);
7     v_predicate VARCHAR2(4000);
8     v_current_user VARCHAR2(128);
9 BEGIN
10     -- Bypass VPD for schema owner and application user
11     v_current_user := SYS_CONTEXT('USERENV', 'SESSION_USER');
12     IF v_current_user = 'GMS_ADMIN' OR v_current_user = 'GMS_APP'
13         OR v_current_user = p_schema THEN
14         RETURN '1=1';
15     END IF;
16
17     v_user_type := SYS_CONTEXT('gms_context', 'user_type');
18     v_user_id := SYS_CONTEXT('gms_context', 'user_id');
19
```

```
20 IF v_user_type = 'Lecturer' THEN
21     -- Lecturers can only edit grades for:
22     -- 1. Courses they teach
23     -- 2. Before the submission deadline
24     v_predicate := 'enrollment_id IN (SELECT e.enrollment_id
25         FROM gms_admin.ENROLLMENTS e
26         JOIN gms_admin.COURSE_SECTIONS cs ON e.section_id = cs.section_id
27         WHERE cs.lecturer_id = ''' || v_user_id || '''
28         AND EXISTS (
29             SELECT 1 FROM gms_admin.GRADE_SUBMISSION_DEADLINES gsd
30             WHERE gsd.semester = cs.semester
31             AND gsd.academic_year = cs.academic_year
32             AND gsd.submission_deadline > SYSDATE
33             AND gsd.is_active = 'Y'
34         ))';
35
36     ELSIF v_user_type = 'Academic_Affairs' THEN
37         -- Academic Affairs can edit all grades (especially after deadline)
38         v_predicate := '1=1';
39     ELSE
40         -- No edit access
41         v_predicate := '1=0';
42     END IF;
43
44     RETURN v_predicate;
45 END lecturer_grade_policy;
```

Listing 31: Function lecturer_grade_policy

Điểm quan trọng:

- Giảng viên chỉ được sửa điểm môn mình dạy
- Phải còn trong deadline (`submission_deadline > SYSDATE`)
- Deadline phải đang active (`is_active = 'Y'`)
- Phòng đào tạo có thể sửa bất kỳ lúc nào (sau deadline)

19.4 Relative Policy Function

Policy cho phụ huynh xem thông tin:


```
1 FUNCTION relative_policy(  
2     p_schema VARCHAR2,  
3     p_object VARCHAR2  
4 ) RETURN VARCHAR2 IS  
5     v_user_type VARCHAR2(50);  
6     v_user_id VARCHAR2(10);  
7     v_predicate VARCHAR2(4000);  
8     v_current_user VARCHAR2(128);  
9 BEGIN  
10    -- Bypass VPD for schema owner and application user  
11    v_current_user := SYS_CONTEXT('USERENV', 'SESSION_USER');  
12    IF v_current_user = 'GMS_ADMIN' OR v_current_user = 'GMS_APP'  
13       OR v_current_user = p_schema THEN  
14        RETURN '1=1';  
15    END IF;  
16  
17    v_user_type := SYS_CONTEXT('gms_context', 'user_type');  
18    v_user_id := SYS_CONTEXT('gms_context', 'user_id');  
19  
20    IF v_user_type = 'Relative' THEN  
21        -- Relatives can only see their own record  
22        v_predicate := 'relative_id = ''' || v_user_id || ''';  
23    ELSIF v_user_type = 'Academic_Affairs' OR v_user_type = 'Admin' THEN  
24        v_predicate := '1=1';  
25    ELSE  
26        v_predicate := '1=0';  
27    END IF;  
28  
29    RETURN v_predicate;  
30 END relative_policy;
```

Listing 32: Function relative_policy

20 Đăng ký VPD Policies

20.1 Xóa Policies cũ

```
1 BEGIN  
2     FOR policy IN (SELECT object_name, policy_name  
3                     FROM dba_policies
```

```
4             WHERE object_owner = 'GMS_ADMIN') LOOP
5 DBMS_RLS.DROP_POLICY(
6     object_schema => 'GMS_ADMIN',
7     object_name => policy.object_name,
8     policy_name => policy.policy_name
9 );
10 END LOOP;
11 END;
12 /
```

Listing 33: Xóa VPD policies cũ

20.2 Đăng ký Policy cho STUDENTS

```
1 BEGIN
2     DBMS_RLS.ADD_POLICY(
3         object_schema => 'GMS_ADMIN',
4         object_name => 'STUDENTS',
5         policy_name => 'student_access_policy',
6         function_schema => 'GMS_ADMIN',
7         policy_function => 'gms_security_pkg.student_policy',
8         statement_types => 'SELECT',
9         enable => TRUE
10    );
11 END;
12 /
```

Listing 34: Policy cho bảng STUDENTS

20.3 Đăng ký Policies cho GRADES

Bảng GRADES cần nhiều policies cho các loại statement khác nhau:

```
1 -- Policy cho SELECT
2 BEGIN
3     DBMS_RLS.ADD_POLICY(
4         object_schema => 'GMS_ADMIN',
5         object_name => 'GRADES',
6         policy_name => 'grade_select_policy',
7         function_schema => 'GMS_ADMIN',
8         policy_function => 'gms_security_pkg.grade_policy',
```



```
9      statement_types => 'SELECT',
10      enable => TRUE
11  );
12  END;
13  /
14
15  -- Policy cho INSERT (with update_check)
16  BEGIN
17      DBMS_RLS.ADD_POLICY(
18          object_schema => 'GMS_ADMIN',
19          object_name => 'GRADES',
20          policy_name => 'grade_insert_policy',
21          function_schema => 'GMS_ADMIN',
22          policy_function => 'gms_security_pkg.lecturer_grade_policy',
23          statement_types => 'INSERT',
24          update_check => TRUE, -- Important for INSERT
25          enable => TRUE
26      );
27  END;
28  /
29
30  -- Policy cho UPDATE
31  BEGIN
32      DBMS_RLS.ADD_POLICY(
33          object_schema => 'GMS_ADMIN',
34          object_name => 'GRADES',
35          policy_name => 'grade_update_policy',
36          function_schema => 'GMS_ADMIN',
37          policy_function => 'gms_security_pkg.lecturer_grade_policy',
38          statement_types => 'UPDATE',
39          enable => TRUE
40      );
41  END;
42  /
43
44  -- Policy cho DELETE
45  BEGIN
46      DBMS_RLS.ADD_POLICY(
47          object_schema => 'GMS_ADMIN',
48          object_name => 'GRADES',
49          policy_name => 'grade_delete_policy',
```

```
50     function_schema => 'GMS_ADMIN',
51     policy_function => 'gms_security_pkg.lecturer_grade_policy',
52     statement_types => 'DELETE',
53     enable => TRUE
54 );
55 END;
56 /
```

Listing 35: Policies cho bảng GRADES

Lưu ý quan trọng: Parameter `update_check => TRUE` cho INSERT policy để đảm bảo dữ liệu insert vào phải thỏa mãn điều kiện policy (tránh lỗi ORA-28104).

20.4 Đăng ký Policies cho RELATIVES

```
1  -- Policy cho RELATIVES table
2  BEGIN
3      DBMS_RLS.ADD_POLICY(
4          object_schema => 'GMS_ADMIN',
5          object_name => 'RELATIVES',
6          policy_name => 'relative_access_policy',
7          function_schema => 'GMS_ADMIN',
8          policy_function => 'gms_security_pkg.relative_policy',
9          statement_types => 'SELECT',
10         enable => TRUE
11     );
12 END;
13 /
14
15 -- Policy cho STUDENT_RELATIVES table
16 BEGIN
17     DBMS_RLS.ADD_POLICY(
18         object_schema => 'GMS_ADMIN',
19         object_name => 'STUDENT_RELATIVES',
20         policy_name => 'student_relative_policy',
21         function_schema => 'GMS_ADMIN',
22         policy_function => 'gms_security_pkg.relative_policy',
23         statement_types => 'SELECT',
24         enable => TRUE
25     );
26 END;
```

Listing 36: Policies cho bảng RELATIVES

21 Danh sách VPD Policies

Bảng 9: Tổng hợp 7 VPD Policies trong hệ thống

STT	Policy Name	Table	Statement	Function
1	student_access_policy	STUDENTS	SELECT	student_policy
2	grade_select_policy	GRADES	SELECT	grade_policy
3	grade_insert_policy	GRADES	INSERT	lecturer_grade_polic
4	grade_update_policy	GRADES	UPDATE	lecturer_grade_polic
5	grade_delete_policy	GRADES	DELETE	lecturer_grade_polic
6	relative_access_policy	RELATIVES	SELECT	relative_policy
7	student_relative_policy	STUDENT_RELATIVES	SELECT	relative_policy

22 VPD Bypass cho GMS_ADMIN và GMS_APP

Một điểm quan trọng trong thiết kế VPD là việc bypass cho các user đặc biệt:

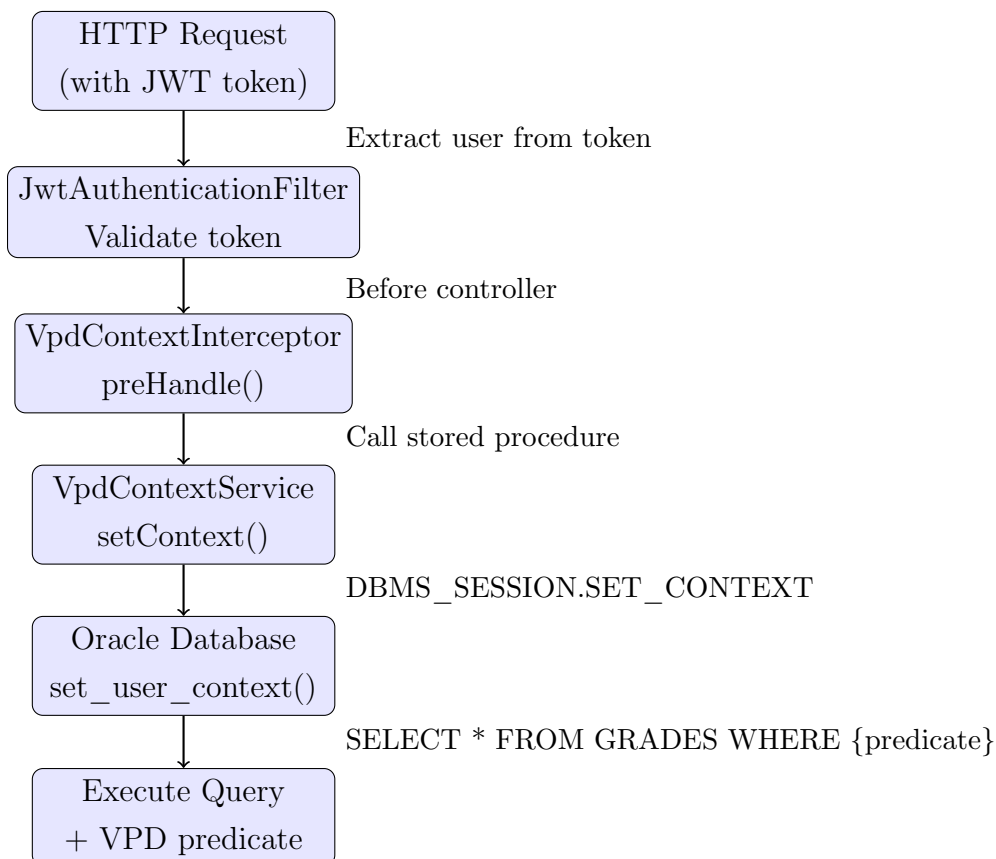
```
1 -- Bypass VPD for schema owner and application user
2 v_current_user := SYS_CONTEXT('USERENV', 'SESSION_USER');
3 IF v_current_user = 'GMS_ADMIN' OR v_current_user = 'GMS_APP'
4   OR v_current_user = p_schema THEN
5   RETURN '1=1'; -- Bypass, see all data
6 END IF;
```

Listing 37: Logic bypass VPD

Lý do bypass:

- **GMS_ADMIN**: Schema owner cần full access để quản trị
- **GMS_APP**: Backend application user cần truy cập all data, VPD context sẽ được set per-request dựa trên authenticated user
- **p_schema**: Policy function schema (cũng là GMS_ADMIN)

23 Flow hoạt động VPD



Hình 4: Flow thiết lập VPD context từ HTTP request

24 Ví dụ minh họa VPD

24.1 Sinh viên STU001 xem điểm

```

1 -- Step 1: Backend set context
2 EXEC gms_security_pkg.set_user_context('STU001', 'Student');
3
4 -- Step 2: Query as written by application
5 SELECT * FROM gms_admin.GRADES;
6
7 -- Step 3: Oracle automatically transforms to:
8 SELECT * FROM gms_admin.GRADES
9 WHERE enrollment_id IN (

```

```
10 SELECT enrollment_id FROM gms_admin.ENROLLMENTS
11 WHERE student_id = 'STU001'
12 );
13
14 -- Result: Only grades of STU001
```

Listing 38: Ví dụ: Sinh viên xem điểm

24.2 Giảng viên LEC001 nhập điểm

```
1 -- Step 1: Backend set context
2 EXEC gms_security_pkg.set_user_context('LEC001', 'Lecturer');
3
4 -- Step 2: Insert grade
5 INSERT INTO gms_admin.GRADES (enrollment_id, midterm_score)
6 VALUES (123, 8.5);
7
8 -- Step 3: VPD checks:
9 -- 1. enrollment_id 123 belongs to course taught by LEC001?
10 -- 2. Is it before deadline?
11
12 -- If both TRUE: INSERT succeeds
13 -- If FALSE: INSERT fails (0 rows inserted)
```

Listing 39: Ví dụ: Giảng viên nhập điểm

24.3 Trưởng khoa xem sinh viên

```
1 -- Step 1: Backend set context
2 EXEC gms_security_pkg.set_user_context('DEAN001', 'Dean');
3 -- Context automatically sets faculty_id = 'FAC001'
4
5 -- Step 2: Query
6 SELECT * FROM gms_admin.STUDENTS;
7
8 -- Step 3: Oracle transforms to:
9 SELECT * FROM gms_admin.STUDENTS
10 WHERE class_id IN (
11     SELECT class_id FROM gms_admin.CLASSES
12     WHERE faculty_id = 'FAC001'
```



13);

14

15 -- Result: Only students in FAC001 faculty

Listing 40: Ví dụ: Dean xem sinh viên

Password Profiles

25 Giới thiệu Password Profiles

Oracle Password Profiles cho phép định nghĩa các chính sách mật khẩu cho từng nhóm người dùng, bao gồm:

- Độ phức tạp của mật khẩu
- Thời hạn sử dụng mật khẩu
- Số lần đăng nhập sai tối đa
- Thời gian khóa tài khoản
- Quy tắc tái sử dụng mật khẩu

26 Danh sách 5 Password Profiles

Bảng 10: 5 Password Profiles trong hệ thống GMS

Profile	Đối tượng	Thời hạn	Sai tối đa	Khóa	Grace
GMS_STUDENT_PROFILE	Sinh viên	90 ngày	5 lần	29 phút	7 ngày
GMS_LECTURER_PROFILE	Giảng viên	60 ngày	3 lần	58 phút	5 ngày
GMS_ADMIN_PROFILE	Dean/DeptHead	30 ngày	3 lần	115 phút	3 ngày
GMS_RELATIVE_PROFILE	Phụ huynh	120 ngày	5 lần	14 phút	14 ngày
GMS_SYSADMIN_PROFILE	GMS_ADMIN	30 ngày	2 lần	230 phút	1 ngày

27 Chi tiết từng Profile

27.1 GMS_STUDENT_PROFILE

Profile cho sinh viên với mức độ bảo mật trung bình:


```
1 CREATE PROFILE GMS_STUDENT_PROFILE LIMIT
2     PASSWORD_VERIFY_FUNCTION ora12c_verify_function
3     PASSWORD_LIFE_TIME 90 -- Mat khau het han sau 90 ngay
4     PASSWORD_GRACE_TIME 7 -- 7 ngay gia han
5     PASSWORD_REUSE_MAX 5 -- Khong dung lai 5 mat khau gan nhat
6     PASSWORD_REUSE_TIME 365 -- Hoac sau 365 ngay
7     FAILED_LOGIN_ATTEMPTS 5 -- 5 lan dang nhap sai
8     PASSWORD_LOCK_TIME 0.02 -- Khoa 29 phut (0.02 ngay)
9     SESSIONS_PER_USER 20 -- Toi da 20 sessions
10    IDLE_TIME 240 -- Timeout sau 4 gio
11    CONNECT_TIME UNLIMITED;
```

Listing 41: Profile cho sinh viên

27.2 GMS_LECTURER_PROFILE

Profile cho giảng viên với mức độ bảo mật cao hơn:

```
1 CREATE PROFILE GMS_LECTURER_PROFILE LIMIT
2     PASSWORD_VERIFY_FUNCTION ora12c_verify_function
3     PASSWORD_LIFE_TIME 60 -- Het han sau 60 ngay
4     PASSWORD_GRACE_TIME 5 -- 5 ngay gia han
5     PASSWORD_REUSE_MAX 8 -- Khong dung lai 8 mat khau
6     PASSWORD_REUSE_TIME 365
7     FAILED_LOGIN_ATTEMPTS 3 -- Chi 3 lan sai
8     PASSWORD_LOCK_TIME 0.04 -- Khoa 58 phut
9     SESSIONS_PER_USER 20
10    IDLE_TIME 240
11    CONNECT_TIME UNLIMITED;
```

Listing 42: Profile cho giảng viên

27.3 GMS_ADMIN_PROFILE

Profile cho vai trò quản lý (Dean, Department Head, Academic Affairs):

```
1 CREATE PROFILE GMS_ADMIN_PROFILE LIMIT
2     PASSWORD_VERIFY_FUNCTION ora12c_verify_function
3     PASSWORD_LIFE_TIME 30 -- Het han sau 30 ngay
4     PASSWORD_GRACE_TIME 3 -- 3 ngay gia han
5     PASSWORD_REUSE_MAX 12 -- Khong dung lai 12 mat khau
6     PASSWORD_REUSE_TIME 365
```

```
7 FAILED_LOGIN_ATTEMPTS 3
8 PASSWORD_LOCK_TIME 0.08 -- Khoa 115 phut
9 SESSIONS_PER_USER 20
10 IDLE_TIME 240
11 CONNECT_TIME UNLIMITED;
```

Listing 43: Profile cho quản lý

27.4 GMS_RELATIVE_PROFILE

Profile cho phụ huynh với yêu cầu thấp hơn (ít truy cập):

```
1 CREATE PROFILE GMS_RELATIVE_PROFILE LIMIT
2   PASSWORD_VERIFY_FUNCTION ora12c_verify_function
3   PASSWORD_LIFE_TIME 120 -- Het han sau 120 ngay
4   PASSWORD_GRACE_TIME 14 -- 14 ngay gia han
5   PASSWORD_REUSE_MAX 3 -- Khong dung lai 3 mat khau
6   PASSWORD_REUSE_TIME 180
7   FAILED_LOGIN_ATTEMPTS 5
8   PASSWORD_LOCK_TIME 0.01 -- Khoa 14 phut
9   SESSIONS_PER_USER 10 -- Chi 10 sessions
10  IDLE_TIME 480 -- 8 gio timeout
11  CONNECT_TIME UNLIMITED;
```

Listing 44: Profile cho phụ huynh

27.5 GMS_SYSADMIN_PROFILE

Profile cho system admin với mức bảo mật cao nhất:

```
1 CREATE PROFILE GMS_SYSADMIN_PROFILE LIMIT
2   PASSWORD_VERIFY_FUNCTION ora12c_verify_function
3   PASSWORD_LIFE_TIME 30 -- Het han sau 30 ngay
4   PASSWORD_GRACE_TIME 1 -- Chi 1 ngay gia han
5   PASSWORD_REUSE_MAX 24 -- Khong dung lai 24 mat khau
6   PASSWORD_REUSE_TIME 730 -- Hoac sau 2 nam
7   FAILED_LOGIN_ATTEMPTS 2 -- Chi 2 lan sai
8   PASSWORD_LOCK_TIME 0.16 -- Khoa 230 phut
9   SESSIONS_PER_USER UNLIMITED -- Khong gioi han sessions
10  IDLE_TIME UNLIMITED
11  CONNECT_TIME UNLIMITED;
```

Listing 45: Profile cho system admin

28 Custom Password Verification Function

Hàm kiểm tra mật khẩu tùy chỉnh đảm bảo mật khẩu đạt tiêu chuẩn:

```
1 CREATE OR REPLACE FUNCTION gms_password_verify
2 (
3     username VARCHAR2,
4     password VARCHAR2,
5     old_password VARCHAR2
6 )
7 RETURN BOOLEAN
8 IS
9     differ INTEGER;
10    digit_count INTEGER;
11    upper_count INTEGER;
12    lower_count INTEGER;
13    special_count INTEGER;
14 BEGIN
15    -- Kiểm tra độ dài tối thiểu 8 ký tự
16    IF LENGTH(password) < 8 THEN
17        RAISE_APPLICATION_ERROR(-20001,
18            'Password must be at least 8 characters');
19    END IF;
20
21    -- Không được trùng username
22    IF UPPER(password) = UPPER(username) THEN
23        RAISE_APPLICATION_ERROR(-20002,
24            'Password cannot be the same as username');
25    END IF;
26
27    -- Phải có ít nhất 1 số
28    digit_count := LENGTH(password) -
29        LENGTH(TRANSLATE(password, '0123456789', ' '));
30    IF digit_count = 0 THEN
31        RAISE_APPLICATION_ERROR(-20003,
32            'Password must contain at least one digit');
33    END IF;
34
35    -- Phải có ít nhất 1 chữ hoa
36    upper_count := LENGTH(password) -
37        LENGTH(TRANSLATE(password,
38            'ABCDEFGHIJKLMNOPQRSTUVWXYZ', ' '));
```



```
39  IF upper_count = 0 THEN
40      RAISE_APPLICATION_ERROR(-20004,
41          'Password must contain at least one uppercase letter');
42  END IF;
43
44  -- Phai co it nhat 1 chu thuong
45  lower_count := LENGTH(password) -
46      LENGTH(TRANSLATE(password,
47          'abcdefghijklmnopqrstuvwxyz', ' '));
48  IF lower_count = 0 THEN
49      RAISE_APPLICATION_ERROR(-20005,
50          'Password must contain at least one lowercase letter');
51  END IF;
52
53  -- Phai co it nhat 1 ky tu dac biet
54  special_count := LENGTH(password) -
55      LENGTH(TRANSLATE(password,
56          '!@#$%^&*()_+=[\]{}|;:,<.>?', ' '));
57  IF special_count = 0 THEN
58      RAISE_APPLICATION_ERROR(-20006,
59          'Password must contain at least one special character');
60  END IF;
61
62  -- Mat khau moi phai khac mat khau cu it nhat 3 ky tu
63  IF old_password IS NOT NULL THEN
64      differ := LENGTH(old_password) - LENGTH(password);
65      IF ABS(differ) < 3 THEN
66          IF LENGTH(TRANSLATE(UPPER(password),
67              UPPER(old_password), ' ')) < 3 THEN
68              RAISE_APPLICATION_ERROR(-20007,
69                  'New password must differ from old by 3+ characters');
70          END IF;
71      END IF;
72  END IF;
73
74  -- Kiem tra mat khau yeu pho bien
75  IF UPPER(password) IN ('PASSWORD', 'PASSWORD123', 'ADMIN123',
76      'STUDENT123', 'LECTURER123', '12345678', 'QWERTY123') THEN
77      RAISE_APPLICATION_ERROR(-20008,
78          'Password is too common');
79  END IF;
```

```
80
81     RETURN TRUE;
82 END;
83 /
```

Listing 46: Function gms_password_verify

29 Yêu cầu mật khẩu tổng hợp

Bảng 11: Yêu cầu mật khẩu trong hệ thống GMS

Yêu cầu	Chi tiết
Độ dài tối thiểu	8 ký tự
Chữ hoa	Ít nhất 1 ký tự (A-Z)
Chữ thường	Ít nhất 1 ký tự (a-z)
Chữ số	Ít nhất 1 ký tự (0-9)
Ký tự đặc biệt	Ít nhất 1 ký tự (!@#\$%^...)
Không trùng username	Mật khẩu $\neq$ username
Khác mật khẩu cũ	Ít nhất 3 ký tự khác biệt
Không phổ biến	Không nằm trong blacklist

30 Áp dụng Profile cho Users

```
1 -- Sinh vien
2 ALTER USER GMS_STUDENT PROFILE GMS_STUDENT_PROFILE;
3
4 -- Giang vien
5 ALTER USER GMS_LLECTURER PROFILE GMS_LLECTURER_PROFILE;
6
7 -- Quan ly (Dean, Dept Head, Academic Affairs)
8 ALTER USER GMS_ACADEMIC PROFILE GMS_ADMIN_PROFILE;
9 ALTER USER GMS_DEAN PROFILE GMS_ADMIN_PROFILE;
10 ALTER USER GMS_DEPT_HEAD PROFILE GMS_ADMIN_PROFILE;
11
12 -- Phu huynh
13 ALTER USER GMS_RELATIVE PROFILE GMS_RELATIVE_PROFILE;
14
```

```
15 -- System Admin
16 ALTER USER GMS_ADMIN PROFILE GMS_SYSADMIN_PROFILE;
```

Listing 47: Áp dụng profiles cho users

Fine-Grained Auditing (FGA)

31 Giới thiệu FGA

Fine-Grained Auditing (FGA) là tính năng của Oracle cho phép theo dõi truy cập dữ liệu ở mức chi tiết:

- Ghi lại ai đã truy cập dữ liệu gì
- Ghi lại câu SQL đã thực thi
- Có thể kích hoạt handler procedure tùy chỉnh
- Audit theo cột cụ thể, không phải toàn bộ bảng

32 Bảng AUDIT_LOG

Bảng lưu trữ nhật ký kiểm toán:

```
1 CREATE TABLE AUDIT_LOG (
2     log_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
3     table_name VARCHAR2(30) NOT NULL,
4     operation VARCHAR2(10) NOT NULL
5     CHECK (operation IN ('INSERT', 'UPDATE', 'DELETE', 'SELECT')),
6     user_id VARCHAR2(10),
7     username VARCHAR2(50),
8     record_id VARCHAR2(50),
9     old_values CLOB,
10    new_values CLOB,
11    operation_date DATE DEFAULT SYSDATE,
12    ip_address VARCHAR2(50),
13    session_id VARCHAR2(100)
14 );
```

Listing 48: Cấu trúc bảng AUDIT_LOG

33 Audit Handler Procedure

Procedure xử lý khi có sự kiện audit:

```
1 CREATE OR REPLACE PROCEDURE audit_grade_handler(  
2     object_schema VARCHAR2,  
3     object_name VARCHAR2,  
4     policy_name VARCHAR2  
5 ) AS  
6     PRAGMA AUTONOMOUS_TRANSACTION;  
7     v_user VARCHAR2(50);  
8     v_user_type VARCHAR2(50);  
9     v_user_id VARCHAR2(10);  
10 BEGIN  
11     v_user := SYS_CONTEXT('USERENV', 'SESSION_USER');  
12     v_user_type := SYS_CONTEXT('gms_context', 'user_type');  
13     v_user_id := SYS_CONTEXT('gms_context', 'user_id');  
14  
15     -- Log vào bảng AUDIT_LOG  
16     INSERT INTO gms_admin.AUDIT_LOG (  
17         table_name,  
18         operation,  
19         user_id,  
20         username,  
21         operation_date,  
22         ip_address,  
23         session_id  
24     ) VALUES (  
25         object_name,  
26         'GRADE_MODIFY',  
27         v_user_id,  
28         v_user,  
29         SYSDATE,  
30         SYS_CONTEXT('USERENV', 'IP_ADDRESS'),  
31         SYS_CONTEXT('USERENV', 'SESSIONID')  
32     );  
33  
34     COMMIT;  
35 EXCEPTION  
36     WHEN OTHERS THEN  
37         ROLLBACK;  
38 END;
```

Listing 49: Audit handler cho GRADES

Lưu ý: PRAGMA AUTONOMOUS_TRANSACTION đảm bảo audit log được ghi ngay cả khi transaction chính bị rollback.

34 Danh sách 8 FGA Policies

Bảng 12: 8 FGA Policies trong hệ thống GMS

STT	Policy Name	Table	Statement	Mục đích
1	fga_grade_select	GRADES	SELECT	Theo dõi ai xem điểm
2	fga_grade_update	GRADES	UPDATE	Theo dõi ai sửa điểm
3	fga_grade_delete	GRADES	DELETE	Theo dõi ai xóa điểm
4	fga_student_info_access	STUDENTS	SELECT	Theo dõi xem thông tin cá nhân SV
5	fga_student_modify	STUDENTS	UPDATE, DELETE	Theo dõi sửa/xóa SV
6	fga_enrollment_modify	ENROLLMENTS	UPDATE, DELETE	Theo dõi sửa đăng ký học
7	fga_system_user_modify	SYSTEM_USERS	UPDATE, DELETE	Theo dõi sửa tài khoản
8	fga_deadline_modify	DEADLINES	UPDATE, DELETE	Theo dõi sửa deadline

35 Chi tiết FGA Policies

35.1 FGA cho xem điểm (SELECT)

```
1 BEGIN
2     DBMS_FGA.ADD_POLICY(
3         object_schema => 'GMS_ADMIN',
4         object_name => 'GRADES',
5         policy_name => 'fga_grade_select',
6         audit_condition => NULL, -- Audit tat ca
7         audit_column => 'MIDTERM_SCORE,FINAL_SCORE,TOTAL_SCORE',
8         statement_types => 'SELECT',
9         enable => TRUE
10    );
11 END;
```


12 /

Listing 50: FGA policy cho SELECT trên GRADES

35.2 FGA cho sửa điểm (UPDATE)

```
1 BEGIN
2     DBMS_FGA.ADD_POLICY(
3         object_schema => 'GMS_ADMIN',
4         object_name => 'GRADES',
5         policy_name => 'fga_grade_update',
6         audit_condition => NULL,
7         audit_column => 'MIDTERM_SCORE,FINAL_SCORE,TOTAL_SCORE,LETTER_GRADE',
8         statement_types => 'UPDATE',
9         enable => TRUE,
10        handler_schema => 'GMS_ADMIN',
11        handler_module => 'audit_grade_handler' -- Goi handler
12    );
13 END;
14 /
```

Listing 51: FGA policy cho UPDATE trên GRADES

35.3 FGA cho thông tin cá nhân sinh viên

```
1 BEGIN
2     DBMS_FGA.ADD_POLICY(
3         object_schema => 'GMS_ADMIN',
4         object_name => 'STUDENTS',
5         policy_name => 'fga_student_info_access',
6         audit_condition => NULL,
7         audit_column => 'EMAIL,PHONE_NUMBER,CONTACT_ADDRESS',
8         statement_types => 'SELECT',
9         enable => TRUE
10    );
11 END;
12 /
```

Listing 52: FGA cho thông tin cá nhân SV

36 Audit Triggers cho INSERT

FGA không hỗ trợ audit INSERT, nên cần dùng triggers:

```
1 CREATE OR REPLACE TRIGGER trg_audit_grade_insert
2 AFTER INSERT ON GRADES
3 FOR EACH ROW
4 DECLARE
5     PRAGMA AUTONOMOUS_TRANSACTION;
6 BEGIN
7     INSERT INTO AUDIT_LOG (
8         table_name,
9         operation,
10        user_id,
11        username,
12        record_id,
13        new_values,
14        operation_date,
15        ip_address,
16        session_id
17    ) VALUES (
18        'GRADES',
19        'INSERT',
20        SYS_CONTEXT('gms_context', 'user_id'),
21        SYS_CONTEXT('USERENV', 'SESSION_USER'),
22        TO_CHAR(:NEW.grade_id),
23        'enrollment_id=' || :NEW.enrollment_id ||
24        ',midterm=' || :NEW.midterm_score ||
25        ',final=' || :NEW.final_score,
26        SYSDATE,
27        SYS_CONTEXT('USERENV', 'IP_ADDRESS'),
28        SYS_CONTEXT('USERENV', 'SESSIONID')
29    );
30    COMMIT;
31 END;
32 /
```

Listing 53: Trigger audit cho INSERT vào GRADES

```
1 CREATE OR REPLACE TRIGGER trg_audit_student_insert
2 AFTER INSERT ON STUDENTS
3 FOR EACH ROW
```

```
4 DECLARE
5     PRAGMA AUTONOMOUS_TRANSACTION;
6 BEGIN
7     INSERT INTO AUDIT_LOG (
8         table_name,
9         operation,
10        user_id,
11        username,
12        record_id,
13        new_values,
14        operation_date,
15        ip_address,
16        session_id
17    ) VALUES (
18        'STUDENTS',
19        'INSERT',
20        SYS_CONTEXT('gms_context', 'user_id'),
21        SYS_CONTEXT('USERENV', 'SESSION_USER'),
22        :NEW.student_id,
23        'name=' || :NEW.first_name || ' ' || :NEW.last_name ||
24        ',class=' || :NEW.class_id,
25        SYSDATE,
26        SYS_CONTEXT('USERENV', 'IP_ADDRESS'),
27        SYS_CONTEXT('USERENV', 'SESSIONID')
28    );
29    COMMIT;
30 END;
31 /
```

Listing 54: Trigger audit cho INSERT vào STUDENTS

37 View Audit Trail

View tổng hợp FGA audit trail và custom audit log:

```
1 CREATE OR REPLACE VIEW V_AUDIT_TRAIL AS
2 SELECT
3     timestamp,
4     db_user,
5     object_schema,
6     object_name,
```

```
7      TO_CLOB(sql_text) as sql_text,  
8      policy_name,  
9      statement_type  
10 FROM dba_fga_audit_trail  
11 WHERE object_schema = 'GMS_ADMIN'  
12 UNION ALL  
13 SELECT  
14     CAST(operation_date AS TIMESTAMP) as timestamp,  
15     username as db_user,  
16     'GMS_ADMIN' as object_schema,  
17     table_name as object_name,  
18     new_values as sql_text,  
19     operation as policy_name,  
20     operation as statement_type  
21 FROM gms_admin.AUDIT_LOG  
22 ORDER BY timestamp DESC;
```

Listing 55: View V_AUDIT_TRAIL

Oracle Label Security (OLS)

38 Giới thiệu OLS

Oracle Label Security (OLS) cung cấp cơ chế bảo mật dựa trên nhãn (label) cho dữ liệu. Mỗi dòng dữ liệu được gán một nhãn bảo mật, và người dùng chỉ có thể truy cập dữ liệu có nhãn trong phạm vi quyền hạn của mình.

CẢNH BÁO QUAN TRỌNG: OLS yêu cầu **Oracle Enterprise Edition**. Oracle XE (Express Edition) KHÔNG hỗ trợ đầy đủ OLS do thiếu tính năng SUPPLEMENTAL_LOG_DATA.

39 Các thành phần OLS

- **Policy:** Định nghĩa tập hợp các levels, compartments, groups
- **Levels:** Cấp độ bảo mật (PUBLIC < INTERNAL < CONFIDENTIAL)
- **Compartments:** Phân loại theo lĩnh vực (CS, EE)
- **Labels:** Kết hợp level + compartment (ví dụ: INT:CS)

40 Policy EXAM_SEC_POLICY

Hệ thống sử dụng OLS để bảo vệ ngân hàng đề thi:

```
1 -- Tao policy
2 BEGIN
3     SA_SYSDBA.CREATE_POLICY (
4         policy_name => 'EXAM_SEC_POLICY',
5         column_name => 'OLS_LABEL',
6         default_options => 'READ_CONTROL,WRITE_CONTROL'
7     );
8 END;
9 /
```

Listing 56: Tạo OLS Policy

41 Cấu hình Levels

Bảng 13: 3 Security Levels trong OLS

Level Num	Code	Tên đầy đủ	Mô tả
1000	PUB	PUBLIC	Công khai, ai cũng xem được
2000	INT	INTERNAL	Nội bộ, chỉ giảng viên
3000	CONF	CONFIDENTIAL	Mật, chỉ trưởng khoa

```
1 BEGIN
2     SA_COMPONENTS.CREATE_LEVEL('EXAM_SEC_POLICY', 1000, 'PUB', 'PUBLIC');
3     SA_COMPONENTS.CREATE_LEVEL('EXAM_SEC_POLICY', 2000, 'INT', 'INTERNAL');
4     SA_COMPONENTS.CREATE_LEVEL('EXAM_SEC_POLICY', 3000, 'CONF', 'CONFIDENTIAL');
5 END;
6 /
```

Listing 57: Tạo OLS Levels

42 Cấu hình Compartments

Bảng 14: 2 Compartments trong OLS

Comp Num	Code	Tên đầy đủ	Mô tả
100	CS	COMPUTER SCIENCE	Khoa học máy tính
200	EE	ELECTRICAL ENGINEERING	Kỹ thuật điện

```
1 BEGIN
2     SA_COMPONENTS.CREATE_COMPARTMENT('EXAM_SEC_POLICY', 100, 'CS',
3                                     'COMPUTER SCIENCE');
4     SA_COMPONENTS.CREATE_COMPARTMENT('EXAM_SEC_POLICY', 200, 'EE',
5                                     'ELECTRICAL ENGINEERING');
6 END;
7 /
```

Listing 58: Tạo OLS Compartments

43 Tạo Labels

```
1 BEGIN
2     SA_LABEL_ADMIN.CREATE_LABEL('EXAM_SEC_POLICY', 1000, 'PUB', TRUE);
3     SA_LABEL_ADMIN.CREATE_LABEL('EXAM_SEC_POLICY', 2100, 'INT:CS', TRUE);
4     SA_LABEL_ADMIN.CREATE_LABEL('EXAM_SEC_POLICY', 2200, 'INT:EE', TRUE);
5     SA_LABEL_ADMIN.CREATE_LABEL('EXAM_SEC_POLICY', 3100, 'CONF:CS', TRUE);
6 END;
7 /
```

Listing 59: Tạo OLS Labels

44 Áp dụng Policy vào bảng EXAM_QUESTIONS

```
1 -- Tao bang EXAM_QUESTIONS
2 CREATE TABLE GMS_ADMIN.EXAM_QUESTIONS (
3     question_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
4     subject_code VARCHAR2(10),
5     question_text VARCHAR2(500),
```

```
6      correct_answer VARCHAR2(200),
7      created_by VARCHAR2(50),
8      created_date DATE DEFAULT SYSDATE
9  );
10
11 -- Áp dụng OLS policy
12 BEGIN
13     SA_POLICY_ADMIN.APPLY_TABLE_POLICY (
14         policy_name => 'EXAM_SEC_POLICY',
15         schema_name => 'GMS_ADMIN',
16         table_name => 'EXAM_QUESTIONS',
17         table_options => 'READ_CONTROL,WRITE_CONTROL,LABEL_DEFAULT'
18     );
19 END;
20 /
```

Listing 60: Áp dụng OLS vào bảng

45 Phân quyền User theo Label

```
1 -- GMS_STUDENT: Chi xem PUBLIC
2 BEGIN
3     SA_USER_ADMIN.SET_USER_LABELS('EXAM_SEC_POLICY', 'GMS_STUDENT',
4         max_read_label => 'PUB');
5 END;
6 /
7
8 -- GMS_LECTURER: Xem và ghi INTERNAL:CS
9 BEGIN
10    SA_USER_ADMIN.SET_USER_LABELS (
11        policy_name => 'EXAM_SEC_POLICY',
12        user_name => 'GMS_LECTURER',
13        max_read_label => 'INT:CS',
14        max_write_label => 'INT:CS',
15        min_write_label => 'PUB',
16        def_label => 'INT:CS',
17        row_label => 'INT:CS'
18    );
19 END;
20 /
```

```
21
22 -- GMS_DEAN: Xem va ghi CONFIDENTIAL:CS
23 BEGIN
24     SA_USER_ADMIN.SET_USER_LABELS (
25         policy_name => 'EXAM_SEC_POLICY',
26         user_name => 'GMS_DEAN',
27         max_read_label => 'CONF:CS',
28         max_write_label => 'CONF:CS',
29         min_write_label => 'PUB',
30         def_label => 'CONF:CS',
31         row_label => 'CONF:CS'
32     );
33 END;
34 /
35
36 -- GMS_ADMIN: FULL bypass
37 BEGIN
38     SA_USER_ADMIN.SET_USER_PRIVS('EXAM_SEC_POLICY', 'GMS_ADMIN',
39         privileges => 'FULL');
40 END;
41 /
```

Listing 61: Phân quyền OLS cho users

46 Ma trận quyền OLS

Bảng 15: Ma trận quyền truy cập OLS

User	PUB	INT:CS	INT:EE	CONF:CS
GMS_STUDENT	R	-	-	-
GMS_LECTURER (CS)	R/W	R/W	-	-
GMS_LECTURER (EE)	R/W	-	R/W	-
GMS_DEAN (CS)	R/W	R/W	-	R/W
GMS_ADMIN	Full	Full	Full	Full

Chú thích:

- **R**: Read only
- **R/W**: Read and Write

- **Full:** Full bypass (xem và sửa tất cả)
- **-:** Không có quyền

Column-Level Security

47 Giới thiệu

Column-Level Security kiểm soát quyền UPDATE trên từng cột cụ thể. Trong hệ thống GMS, người dùng chỉ được cập nhật một số cột nhất định trong profile của mình.

48 Quy tắc Column-Level Security

Bảng 16: *Quyền UPDATE theo cột*

User	Table	Columns được UPDATE
GMS_STUDENT	STUDENTS	email, phone_number, contact_address
GMS_LECTURER	LECTURERS	email, phone_number, contact_address
GMS_RELATIVE	RELATIVES	email, phone_number, contact_address

49 GRANT Statements

```
1 -- Sinh vien chi duoc cap nhat 3 cot trong STUDENTS
2 GRANT UPDATE (email, phone_number, contact_address)
3   ON gms_admin.STUDENTS TO GMS_STUDENT;
4
5 -- Giang vien chi duoc cap nhat 3 cot trong LECTURERS
6 GRANT UPDATE (email, phone_number, contact_address)
7   ON gms_admin.LECTURERS TO GMS_LECTURER;
8
9 -- Phu huynh chi duoc cap nhat 3 cot trong RELATIVES
10 GRANT UPDATE (email, phone_number, contact_address)
```

11 ON gms_admin.RELATIVES TO GMS_RELATIVE;

Listing 62: GRANT column-level UPDATE

50 Kết hợp với VPD

Column-Level Security kết hợp với VPD để đảm bảo:

1. **Row-Level:** VPD đảm bảo user chỉ thấy bản ghi của mình
2. **Column-Level:** GRANT đảm bảo user chỉ UPDATE được các cột cho phép

Ví dụ: Sinh viên STU001:

- VPD: Chỉ thấy bản ghi có `student_id = 'STU001'`
- Column: Chỉ UPDATE được `email`, `phone_number`, `contact_address`
- Kết quả: STU001 chỉ cập nhật được 3 cột trong bản ghi của chính mình

51 Test Column-Level Security

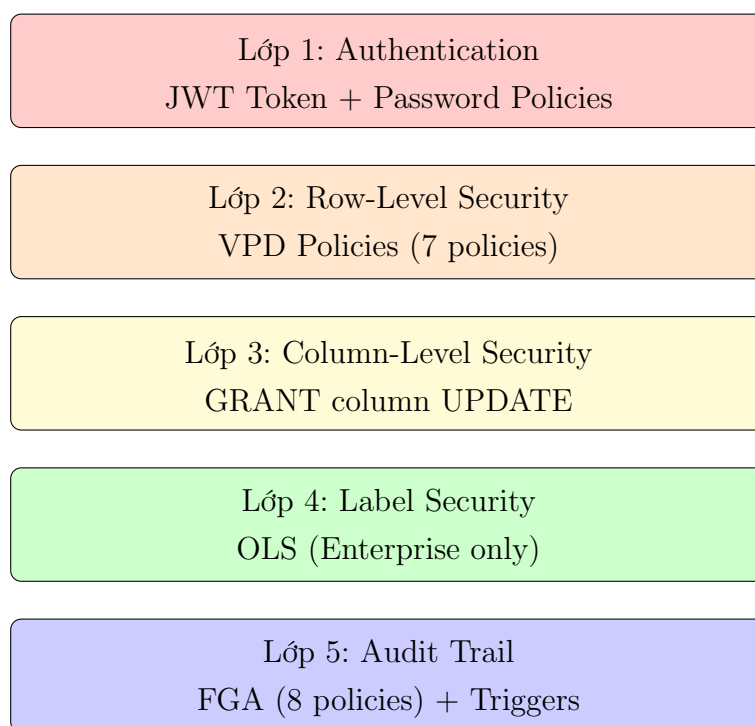
```
1 -- Connect as GMS_STUDENT
2 -- Set context
3 EXEC gms_security_pkg.set_user_context('STU001', 'Student');
4
5 -- Test 1: Update allowed column (SUCCESS)
6 UPDATE gms_admin.STUDENTS
7 SET email = 'newemail@student.edu'
8 WHERE student_id = 'STU001';
9 -- Expected: 1 row updated
10
11 -- Test 2: Update restricted column (FAIL)
12 UPDATE gms_admin.STUDENTS
13 SET first_name = 'New Name'
14 WHERE student_id = 'STU001';
15 -- Expected: ORA-01031: insufficient privileges
16
17 -- Test 3: Update other student (FAIL due to VPD)
18 UPDATE gms_admin.STUDENTS
```

```
19 SET email = 'hack@email.com'  
20 WHERE student_id = 'STU002';  
21 -- Expected: 0 rows updated (VPD filter)
```

Listing 63: Test column-level security

52 Tổng kết bảo mật đa lớp

Hệ thống GMS áp dụng bảo mật đa lớp:



Hình 5: Mô hình bảo mật đa lớp trong hệ thống GMS

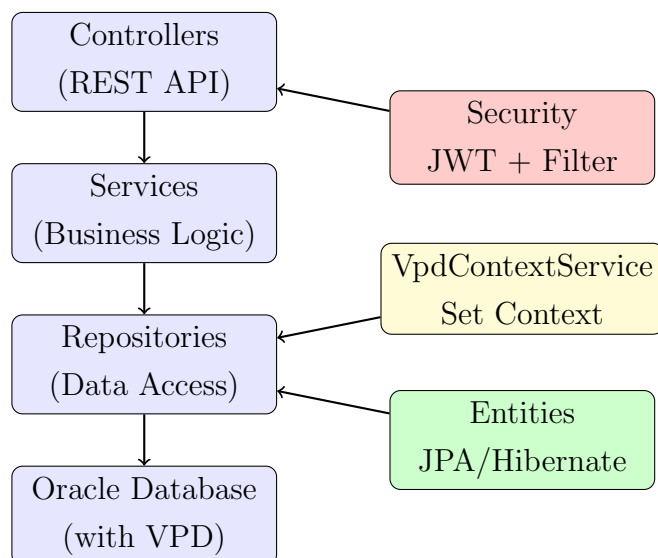
Phần IV

BACKEND APPLICATION

Spring Boot Backend

53 Kiến trúc Backend

Backend được xây dựng theo kiến trúc MVC với Spring Boot 3.x:



Hình 6: Kiến trúc Backend Spring Boot

54 Cấu trúc Project

```
1 backend/src/main/java/edu/university/grademanagement/
2 |-- GradeManagementApplication.java # Main class
3 |-- config/
4 | |-- OpenApiConfig.java # Swagger config
5 | |-- WebConfig.java # Web config
6 | |-- WebMvcConfig.java # MVC config
7 |-- controller/
8 | |-- AuthController.java # Authentication
9 | |-- StudentController.java # Student APIs
10 | |-- LecturerController.java # Lecturer APIs
11 | |-- AdminController.java # Admin APIs
12 |-- service/
13 | |-- AuthService.java # Auth logic
14 | |-- StudentService.java # Student logic
15 | |-- LecturerService.java # Lecturer logic
16 | |-- VpdContextService.java # VPD context
17 |-- repository/
```

```
18 | |-- StudentRepository.java
19 | |-- GradeRepository.java
20 | |-- EnrollmentRepository.java
21 | |-- SystemUserRepository.java
22 | |-- model/entity/
23 | |-- Student.java
24 | |-- Lecturer.java
25 | |-- Grade.java
26 | |-- Enrollment.java
27 | |-- SystemUser.java
28 | |-- security/
29 | |-- SecurityConfig.java
30 | |-- JwtTokenProvider.java
31 | |-- JwtAuthenticationFilter.java
32 | |-- CustomUserDetailsService.java
33 | |-- interceptor/
34 | |-- VpdContextInterceptor.java
35 | |-- dto/
36 |     |-- LoginRequest.java
37 |     |-- LoginResponse.java
38 |     |-- ApiResponse.java
```

Listing 64: Cấu trúc thư mục backend

55 Controllers

55.1 AuthController

Controller xử lý authentication:

```
1 @RestController
2 @RequestMapping("/auth")
3 public class AuthController {
4
5     @Autowired
6     private AuthService authService;
7
8     @PostMapping("/login")
9     public ResponseEntity<LoginResponse> login(
10         @RequestBody LoginRequest loginRequest) {
11
```

```
12     LoginResponse response = authService.authenticate(  
13         loginRequest.getUsername(),  
14         loginRequest.getPassword()  
15     );  
16  
17     return ResponseEntity.ok(response);  
18 }  
19  
20 @PostMapping("/logout")  
21 public ResponseEntity<ApiResponse> logout() {  
22     // Client-side: delete token  
23     return ResponseEntity.ok(  
24         new ApiResponse(true, "Logged out successfully")  
25     );  
26 }  
27 }
```

Listing 65: AuthController.java

55.2 StudentController

Controller cho sinh viên:

```
1 @RestController  
2 @RequestMapping("/students")  
3 public class StudentController {  
4  
5     @Autowired  
6     private StudentService studentService;  
7  
8     // GET /students/me - Xem thông tin cá nhân  
9     @GetMapping("/me")  
10    public ResponseEntity<Student> getMyProfile() {  
11        return ResponseEntity.ok(studentService.getCurrentStudent());  
12    }  
13  
14    // GET /students/me/grades - Xem điểm của mình  
15    @GetMapping("/me/grades")  
16    public ResponseEntity<List<Grade>> getMyGrades() {  
17        return ResponseEntity.ok(studentService.getMyGrades());  
18    }  
19 }
```

```
20 // PUT /students/me - Cap nhat thong tin lien he
21 @PutMapping("/me")
22 public ResponseEntity<Student> updateProfile(
23     @RequestBody StudentUpdateRequest request) {
24     // Chi duoc update: email, phone, address (Column-Level)
25     return ResponseEntity.ok(
26         studentService.updateContactInfo(request)
27     );
28 }
29 }
```

Listing 66: StudentController.java - Các endpoint

55.3 LecturerController

Controller cho giảng viên:

```
1 @RestController
2 @RequestMapping("/lecturers")
3 public class LecturerController {
4
5     @Autowired
6     private LecturerService lecturerService;
7
8     // GET /lecturers/me/students - Xem sinh vien
9     @GetMapping("/me/students")
10    public ResponseEntity<List<Student>> getMyStudents() {
11        // VPD tu dong loc: lop chu nhien + mon day
12        return ResponseEntity.ok(lecturerService.getMyStudents());
13    }
14
15    // GET /lecturers/me/grades - Xem diem sinh vien
16    @GetMapping("/me/grades")
17    public ResponseEntity<List<Grade>> getGrades() {
18        // VPD tu dong loc: diem mon minh day + lop chu nhien
19        return ResponseEntity.ok(lecturerService.getGrades());
20    }
21
22    // POST /lecturers/grades - Nhap diem
23    @PostMapping("/grades")
24    public ResponseEntity<Grade> submitGrade(
25        @RequestBody GradeSubmitRequest request) {
```

```
26         // VPD kiểm tra: môn minh day + trước deadline
27         return ResponseEntity.ok(lecturerService.submitGrade(request));
28     }
29
30     // PUT /lecturers/grades/{id} - Sửa điểm
31     @PutMapping("/grades/{id}")
32     public ResponseEntity<Grade> updateGrade(
33         @PathVariable Long id,
34         @RequestBody GradeUpdateRequest request) {
35         // VPD kiểm tra: môn minh day + trước deadline
36         return ResponseEntity.ok(lecturerService.updateGrade(id, request));
37     }
38 }
```

Listing 67: LecturerController.java - Các endpoint

56 Services

56.1 AuthService

Service xử lý authentication:

```
1 @Service
2 public class AuthService {
3
4     @Autowired
5     private AuthenticationManager authenticationManager;
6
7     @Autowired
8     private JwtTokenProvider jwtTokenProvider;
9
10    @Autowired
11    private SystemUserRepository userRepository;
12
13    public LoginResponse authenticate(String username, String password) {
14        // 1. Xác thực username/password
15        Authentication auth = authenticationManager.authenticate(
16            new UsernamePasswordAuthenticationToken(username, password)
17        );
18
19        // 2. Tạo JWT token
```



```
20     String token = jwtTokenProvider.generateToken(auth);
21     String refreshToken = jwtTokenProvider.generateRefreshToken(
22         username,
23         ((UserPrincipal) auth.getPrincipal()).getRole()
24     );
25
26     // 3. Lay thông tin user
27     SystemUser user = userRepository.findByUsername(username)
28         .orElseThrow(() -> new RuntimeException("User not found"));
29
30     // 4. Tra ve response
31     return LoginResponse.builder()
32         .token(token)
33         .refreshToken(refreshToken)
34         .userId(user.getUserId())
35         .username(username)
36         .userType(user.getUserType())
37         .build();
38 }
39 }
```

Listing 68: AuthService.java

56.2 VpdContextService

Service thiết lập VPD context:

```
1 @Service
2 public class VpdContextService {
3
4     @Autowired
5     private DataSource dataSource;
6
7     @Value("${vpd.context.enabled:true}")
8     private boolean vpdEnabled;
9
10    @Value("${vpd.context.package:gms_security_pkg}")
11    private String vpdPackage;
12
13    /**
14     * Set VPD context cho session hiện tại
15     * Goi: GMS_ADMIN.gms_security_pkg.set_user_context(?, ?)
```

```
16     */
17     public void setContext(String userId, String userRole) throws Exception {
18         if (!vpdEnabled) {
19             return;
20         }
21
22         // Convert role format: ADMIN -> Admin, STUDENT -> Student
23         String userType = convertRoleToUserType(userRole);
24
25         try (Connection conn = dataSource.getConnection()) {
26             String sql = String.format(
27                 "BEGIN GMS_ADMIN.%s.set_user_context(?, ?); END;",
28                 vpdPackage
29             );
30
31             try (CallableStatement stmt = conn.prepareCall(sql)) {
32                 stmt.setString(1, userId);
33                 stmt.setString(2, userType);
34                 stmt.execute();
35
36                 System.out.println(String.format(
37                     "VPD context set: user_id=%s, user_type=%s",
38                     userId, userType
39                 ));
40             }
41         }
42     }
43
44     private String convertRoleToUserType(String role) {
45         if (role == null || role.isEmpty()) {
46             return role;
47         }
48         // Title case: STUDENT -> Student
49         return role.substring(0, 1).toUpperCase() +
50             role.substring(1).toLowerCase();
51     }
52
53     public void clearContext() {
54         if (!vpdEnabled) return;
55
56         try (Connection conn = dataSource.getConnection()) {
```

```
57         String sql = String.format(  
58             "BEGIN GMS_ADMIN.%s.clear_user_context(); END;",  
59             vpdPackage  
60         );  
61         try (CallableStatement stmt = conn.prepareCall(sql)) {  
62             stmt.execute();  
63         }  
64     } catch (Exception e) {  
65         System.err.println("Failed to clear context: " + e.getMessage());  
66     }  
67 }  
68 }
```

Listing 69: VpdContextService.java

57 Repositories

57.1 StudentRepository

```
1 @Repository  
2 public interface StudentRepository extends JpaRepository<Student, String> {  
3  
4     // VPD tu dong ap dung - chi tra ve sinh vien duoc phep xem  
5     List<Student> findAll();  
6  
7     // Tim sinh vien theo ID - VPD se tra ve null neu khong duoc phep  
8     Optional<Student> findById(String studentId);  
9  
10    // Tim sinh vien theo email  
11    Optional<Student> findByEmail(String email);  
12  
13    // Tim sinh vien theo lop  
14    List<Student> findByClassId(String classId);  
15 }
```

Listing 70: StudentRepository.java

57.2 GradeRepository

```
1 @Repository
2 public interface GradeRepository extends JpaRepository<Grade, Long> {
3
4     // VPD tu dong loc diem theo quyen
5     List<Grade> findAll();
6
7     // Tim diem theo enrollment
8     Optional<Grade> findByEnrollmentId(Long enrollmentId);
9
10    // Tim diem theo enrollment (Native Query)
11    @Query(value = "SELECT * FROM gms_admin.GRADES " +
12            "WHERE enrollment_id = ?1",
13            nativeQuery = true)
14    Optional<Grade> findGradeByEnrollment(Long enrollmentId);
15 }
```

Listing 71: GradeRepository.java

58 Entities

58.1 Student Entity

```
1 @Entity
2 @Table(name = "STUDENTS", schema = "GMS_ADMIN")
3 public class Student {
4
5     @Id
6     @Column(name = "STUDENT_ID")
7     private String studentId;
8
9     @Column(name = "FIRST_NAME", nullable = false)
10    private String firstName;
11
12    @Column(name = "LAST_NAME", nullable = false)
13    private String lastName;
14
15    @Column(name = "DATE_OF_BIRTH", nullable = false)
16    private LocalDate dateOfBirth;
17
18    @Column(name = "GENDER")
```

```
19     private String gender;
20
21     @Column(name = "EMAIL")
22     private String email;
23
24     @Column(name = "PHONE_NUMBER")
25     private String phoneNumber;
26
27     @Column(name = "CONTACT_ADDRESS")
28     private String contactAddress;
29
30     @Column(name = "CLASS_ID", nullable = false)
31     private String classId;
32
33     @Column(name = "STUDENT_STATUS")
34     private String studentStatus;
35
36     // Getters, Setters, Constructors
37 }
```

Listing 72: Student.java

58.2 Grade Entity

```
1 @Entity
2 @Table(name = "GRADES", schema = "GMS_ADMIN")
3 public class Grade {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     @Column(name = "GRADE_ID")
8     private Long gradeId;
9
10    @Column(name = "ENROLLMENT_ID", nullable = false)
11    private Long enrollmentId;
12
13    @Column(name = "MIDTERM_SCORE")
14    private BigDecimal midtermScore;
15
16    @Column(name = "FINAL_SCORE")
17    private BigDecimal finalScore;
```

```
18
19     @Column(name = "TOTAL_SCORE")
20     private BigDecimal totalScore;
21
22     @Column(name = "LETTER_GRADE")
23     private String letterGrade;
24
25     @Column(name = "GRADE_STATUS")
26     private String gradeStatus;
27
28     @Column(name = "SUBMITTED_BY")
29     private String submittedBy;
30
31     @Column(name = "SUBMITTED_DATE")
32     private LocalDate submittedDate;
33
34     // Getters, Setters, Constructors
35 }
```

Listing 73: Grade.java

58.3 SystemUser Entity

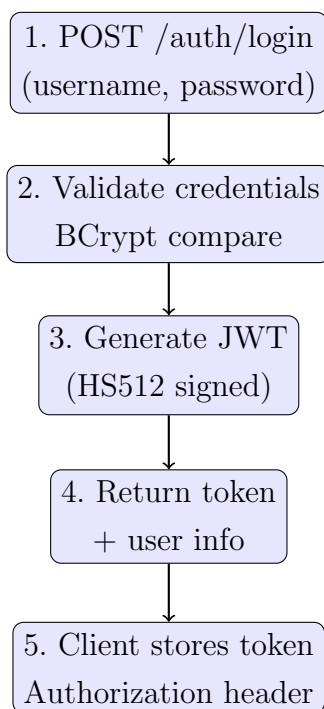
```
1 @Entity
2 @Table(name = "SYSTEM_USERS", schema = "GMS_ADMIN")
3 public class SystemUser {
4
5     @Id
6     @Column(name = "USER_ID")
7     private String userId;
8
9     @Column(name = "USERNAME", unique = true, nullable = false)
10    private String username;
11
12    @Column(name = "PASSWORD_HASH", nullable = false)
13    private String passwordHash;
14
15    @Column(name = "USER_TYPE", nullable = false)
16    private String userType;
17
18    @Column(name = "REFERENCE_ID", nullable = false)
```

```
19 private String referenceId;  
20  
21 @Column(name = "IS_ACTIVE")  
22 private String isActive;  
23  
24 @Column(name = "ACCOUNT_LOCKED")  
25 private String accountLocked;  
26  
27 // Getters, Setters, Constructors  
28 }
```

Listing 74: SystemUser.java

Security Backend

59 JWT Authentication Flow



Hình 7: JWT Authentication Flow

60 SecurityConfig



```
1 @Configuration
2 @EnableWebSecurity
3 @EnableMethodSecurity(prePostEnabled = true)
4 public class SecurityConfig {
5
6     @Autowired
7     private JwtAuthenticationEntryPoint unauthorizedHandler;
8
9     @Bean
10    public JwtAuthenticationFilter jwtAuthenticationFilter() {
11        return new JwtAuthenticationFilter();
12    }
13
14    @Bean
15    public PasswordEncoder passwordEncoder() {
16        return new BCryptPasswordEncoder();
17    }
18
19    @Bean
20    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
21        http
22            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
23            .csrf(csrf -> csrf.disable())
24            .exceptionHandling(ex ->
25                ex.authenticationEntryPoint(unauthorizedHandler))
26            .sessionManagement(session ->
27                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
28            .authorizeHttpRequests(auth -> auth
29                // Public endpoints
30                .requestMatchers("/auth/**").permitAll()
31                .requestMatchers("/swagger-ui/**").permitAll()
32                .requestMatchers("/api-docs/**").permitAll()
33
34                // Role-based endpoints
35                .requestMatchers("/admin/**").hasRole("ADMIN")
36                .requestMatchers("/students/**").hasAnyRole("STUDENT", "ADMIN")
37                .requestMatchers("/lecturers/**").hasAnyRole("LECTURER", "ADMIN")
38                .requestMatchers("/deans/**").hasAnyRole("DEAN", "ADMIN")
39                .requestMatchers("/academic-affairs/**")
40                    .hasAnyRole("ACADEMIC_AFFAIRS", "ADMIN")
41    }
```



```
42         // All other requests require authentication
43         .anyRequest().authenticated()
44     );
45
46     // Add JWT filter before UsernamePasswordAuthenticationFilter
47     http.addFilterBefore(jwtAuthenticationFilter(),
48         UsernamePasswordAuthenticationFilter.class);
49
50     return http.build();
51 }
52 }
```

Listing 75: SecurityConfig.java

61 JwtTokenProvider

```
1 @Component
2 public class JwtTokenProvider {
3
4     @Value("${jwt.secret}")
5     private String jwtSecret;
6
7     @Value("${jwt.expiration}")
8     private long jwtExpirationMs;
9
10    /**
11     * Generate JWT token from username and role
12     */
13    public String generateTokenFromUsername(String username, String role) {
14        Date now = new Date();
15        Date expiryDate = new Date(now.getTime() + jwtExpirationMs);
16
17        SecretKey key = Keys.hmacShaKeyFor(
18            jwtSecret.getBytes(StandardCharsets.UTF_8)
19        );
20
21        return Jwts.builder()
22            .setSubject(username)
23            .claim("role", role)
24            .setIssuedAt(now)
```

```
25         .setExpiration(expiryDate)
26         .signWith(key, SignatureAlgorithm.HS512)
27         .compact();
28     }
29
30     /**
31      * Get username from JWT token
32      */
33     public String getUsernameFromToken(String token) {
34         SecretKey key = Keys.hmacShaKeyFor(
35             jwtSecret.getBytes(StandardCharsets.UTF_8)
36         );
37
38         Claims claims = Jwts.parserBuilder()
39             .setSigningKey(key)
40             .build()
41             .parseClaimsJws(token)
42             .getBody();
43
44         return claims.getSubject();
45     }
46
47     /**
48      * Get role from JWT token
49      */
50     public String getRoleFromToken(String token) {
51         SecretKey key = Keys.hmacShaKeyFor(
52             jwtSecret.getBytes(StandardCharsets.UTF_8)
53         );
54
55         Claims claims = Jwts.parserBuilder()
56             .setSigningKey(key)
57             .build()
58             .parseClaimsJws(token)
59             .getBody();
60
61         return claims.get("role", String.class);
62     }
63
64     /**
65      * Validate JWT token
```

```
66     */
67     public boolean validateToken(String token) {
68         try {
69             SecretKey key = Keys.hmacShaKeyFor(
70                 jwtSecret.getBytes(StandardCharsets.UTF_8)
71             );
72             Jwts.parserBuilder().setSigningKey(key).build()
73                 .parseClaimsJws(token);
74             return true;
75         } catch (JwtException | IllegalArgumentException e) {
76             return false;
77         }
78     }
79 }
```

Listing 76: JwtTokenProvider.java

62 JwtAuthenticationFilter

```
1 public class JwtAuthenticationFilter extends OncePerRequestFilter {
2
3     @Autowired
4     private JwtTokenProvider tokenProvider;
5
6     @Autowired
7     private CustomUserDetailsService userDetailsService;
8
9     @Override
10    protected void doFilterInternal(
11        HttpServletRequest request,
12        HttpServletResponse response,
13        FilterChain filterChain) throws ServletException, IOException {
14
15        try {
16            // 1. Extract JWT from Authorization header
17            String jwt = getJwtFromRequest(request);
18
19            // 2. Validate token
20            if (StringUtils.hasText(jwt) && tokenProvider.validateToken(jwt)) {
21
```

```
22         // 3. Get username from token
23         String username = tokenProvider.getUsernameFromToken(jwt);
24
25         // 4. Load user details
26         UserDetails userDetails =
27             userDetailsService.loadUserByUsername(username);
28
29         // 5. Create authentication object
30         UsernamePasswordAuthenticationToken authentication =
31             new UsernamePasswordAuthenticationToken(
32                 userDetails, null, userDetails.getAuthorities()
33             );
34
35         authentication.setDetails(
36             new WebAuthenticationDetailsSource()
37                 .buildDetails(request)
38         );
39
40         // 6. Set authentication in SecurityContext
41         SecurityContextHolder.getContext()
42             .setAuthentication(authentication);
43     }
44     } catch (Exception ex) {
45         logger.error("Could not set user authentication", ex);
46     }
47
48     filterChain.doFilter(request, response);
49 }
50
51 private String getJwtFromRequest(HttpServletRequest request) {
52     String bearerToken = request.getHeader("Authorization");
53     if (StringUtils.hasText(bearerToken) &&
54         bearerToken.startsWith("Bearer ")) {
55         return bearerToken.substring(7);
56     }
57     return null;
58 }
59 }
```

Listing 77: JwtAuthenticationFilter.java

63 VpdContextInterceptor

Interceptor thiết lập VPD context cho mỗi request:

```
1 @Component
2 public class VpdContextInterceptor implements HandlerInterceptor {
3
4     @Autowired
5     private VpdContextService vpdContextService;
6
7     @Override
8     public boolean preHandle(
9         HttpServletRequest request,
10        HttpServletResponse response,
11        Object handler) throws Exception {
12
13        // Get authenticated user from Spring Security context
14        Authentication authentication =
15            SecurityContextHolder.getContext().getAuthentication();
16
17        if (authentication != null && authentication.isAuthenticated()
18            && authentication.getPrincipal() instanceof UserPrincipal) {
19
20            UserPrincipal userPrincipal =
21                (UserPrincipal) authentication.getPrincipal();
22
23            // Set VPD context with user info
24            try {
25                vpdContextService.setContext(
26                    userPrincipal.getUserId(),
27                    userPrincipal.getRole()
28                );
29
30                System.out.println(String.format(
31                    "[VPD] Context set for %s %s: user=%s, role=%s",
32                    request.getMethod(),
33                    request.getRequestURI(),
34                    userPrincipal.getUserId(),
35                    userPrincipal.getRole()
36                ));
37            } catch (Exception e) {
38                System.err.println("[VPD] Failed to set context: "
```

```
39         + e.getMessage());
40     }
41 }
42
43     return true; // Continue with request
44 }
45 }
```

Listing 78: VpdContextInterceptor.java

64 CustomUserService

```
1 @Service
2 public class CustomUserService implements UserDetailsService {
3
4     @Autowired
5     private SystemUserRepository userRepository;
6
7     @Override
8     public UserDetails loadUserByUsername(String username)
9         throws UsernameNotFoundException {
10
11         SystemUser user = userRepository.findByUsername(username)
12             .orElseThrow(() -> new UsernameNotFoundException(
13                 "User not found: " + username
14             ));
15
16         // Check if account is locked
17         if ("Y".equals(user.getAccountLocked())) {
18             throw new LockedException("Account is locked");
19         }
20
21         // Check if account is active
22         if (!"Y".equals(user.getIsActive())) {
23             throw new DisabledException("Account is disabled");
24         }
25
26         // Return UserPrincipal with user info
27         return UserPrincipal.builder()
28             .userId(user.getReferenceId())
```

```
29         .username(user.getUsername())
30         .password(user.getPasswordHash())
31         .role(user.getUserType())
32         .enabled(true)
33         .build();
34     }
35 }
```

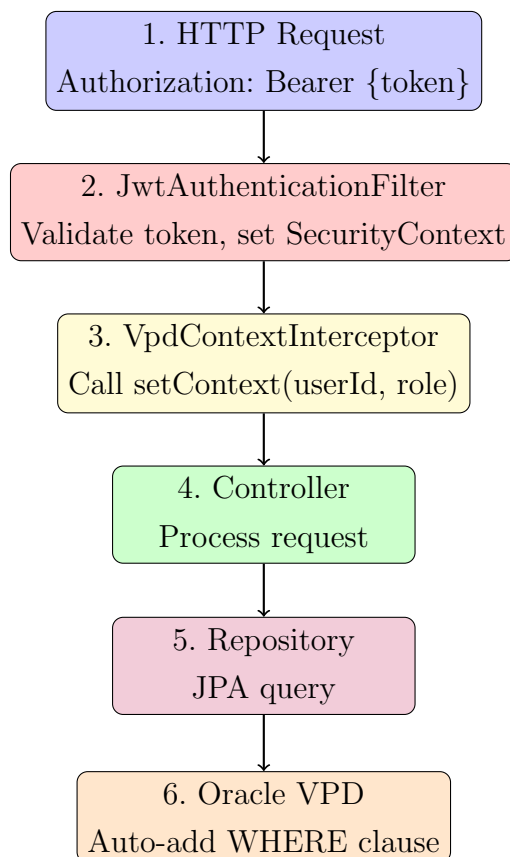
Listing 79: CustomUserDetailsService.java

65 WebMvcConfig - Đăng ký Interceptor

```
1 @Configuration
2 public class WebMvcConfig implements WebMvcConfigurer {
3
4     @Autowired
5     private VpdContextInterceptor vpdContextInterceptor;
6
7     @Override
8     public void addInterceptors(InterceptorRegistry registry) {
9         // Register VPD context interceptor
10        registry.addInterceptor(vpdContextInterceptor)
11            .addPathPatterns("/**") // Apply to all paths
12            .excludePathPatterns(
13                "/auth/**", // Exclude authentication
14                "/api-docs/**", // Exclude API docs
15                "/swagger-ui/**", // Exclude Swagger UI
16                "/actuator/**", // Exclude actuator
17                "/error" // Exclude error endpoint
18            );
19    }
20 }
```

Listing 80: WebMvcConfig.java

66 Request Processing Flow



Hình 8: Request Processing Flow với VPD Integration

67 API Endpoints Summary

Bảng 17: *Danh sách API Endpoints*

Method	Endpoint	Role	Mô tả
POST	/auth/login	Public	Đăng nhập
POST	/auth/logout	Auth	Đăng xuất
GET	/students/me	Student	Xem profile
PUT	/students/me	Student	Cập nhật liên hệ
GET	/students/me/grades	Student	Xem điểm
GET	/lecturers/me/students	Lecturer	Xem sinh viên
GET	/lecturers/me/grades	Lecturer	Xem điểm
POST	/lecturers/grades	Lecturer	Nhập điểm
PUT	/lecturers/grades/{id}	Lecturer	Sửa điểm
GET	/admin/users	Admin	Xem tất cả users
GET	/admin/audit-log	Admin	Xem audit log

Test Cases - Kiểm thử bảo mật

Chương này trình bày các test case chi tiết để kiểm tra tính đúng đắn của các cơ chế bảo mật đã triển khai trong hệ thống. Mỗi test case được thiết kế để verify một khía cạnh cụ thể của security model.

68 Môi trường Test

68.1 Chuẩn bị môi trường

```

1  -- =====
2  -- SETUP TEST ENVIRONMENT
3  -- =====
4
5  -- To test data
6  ALTER SESSION SET CONTAINER = ORCLPDB;
7  ALTER SESSION SET CURRENT_SCHEMA = GMS_ADMIN;
8
9  -- Insert test faculty
10 INSERT INTO FACULTIES (faculty_id, faculty_name, established_date)

```



```
11 VALUES ('FAC01', 'Khoa Khoa học Máy tính', DATE '2000-01-01');
12
13 INSERT INTO FACULTIES (faculty_id, faculty_name, established_date)
14 VALUES ('FAC02', 'Khoa Điện - Điện tử', DATE '2000-01-01');
15
16 -- Insert test departments
17 INSERT INTO DEPARTMENTS (department_id, department_name, faculty_id)
18 VALUES ('DEPT01', 'Bộ môn Công nghệ phần mềm', 'FAC01');
19
20 INSERT INTO DEPARTMENTS (department_id, department_name, faculty_id)
21 VALUES ('DEPT02', 'Bộ môn Mạng máy tính', 'FAC01');
22
23 INSERT INTO DEPARTMENTS (department_id, department_name, faculty_id)
24 VALUES ('DEPT03', 'Bộ môn Tự động hóa', 'FAC02');
25
26 -- Insert test classes
27 INSERT INTO CLASSES (class_id, class_name, faculty_id, academic_year)
28 VALUES ('CLASS01', 'CNTT 2021', 'FAC01', 2021);
29
30 INSERT INTO CLASSES (class_id, class_name, faculty_id, academic_year)
31 VALUES ('CLASS02', 'CNTT 2022', 'FAC01', 2022);
32
33 INSERT INTO CLASSES (class_id, class_name, faculty_id, academic_year)
34 VALUES ('CLASS03', 'ĐTVT 2021', 'FAC02', 2021);
35
36 -- Insert test lecturers
37 INSERT INTO LECTURERS (lecturer_id, first_name, last_name, date_of_birth,
38     email, department_id, start_date)
39 VALUES ('LEC001', 'Nguyen', 'Van A', DATE '1980-01-15',
40     'nguyenvana@hcmut.edu.vn', 'DEPT01', DATE '2010-09-01');
41
42 INSERT INTO LECTURERS (lecturer_id, first_name, last_name, date_of_birth,
43     email, department_id, start_date)
44 VALUES ('LEC002', 'Tran', 'Thi B', DATE '1975-05-20',
45     'tranthib@hcmut.edu.vn', 'DEPT02', DATE '2005-09-01');
46
47 INSERT INTO LECTURERS (lecturer_id, first_name, last_name, date_of_birth,
48     email, department_id, start_date)
49 VALUES ('LEC003', 'Le', 'Van C', DATE '1970-12-10',
50     'levanc@hcmut.edu.vn', 'DEPT03', DATE '2000-09-01');
51
```



```
52 -- Insert test students
53 INSERT INTO STUDENTS (student_id, first_name, last_name, date_of_birth,
54     email, class_id, enrollment_date)
55 VALUES ('STU001', 'Pham', 'Minh D', DATE '2003-03-15',
56     'phammd@student.hcmut.edu.vn', 'CLASS01', DATE '2021-09-01');
57
58 INSERT INTO STUDENTS (student_id, first_name, last_name, date_of_birth,
59     email, class_id, enrollment_date)
60 VALUES ('STU002', 'Hoang', 'Thi E', DATE '2003-07-22',
61     'hoangthe@student.hcmut.edu.vn', 'CLASS01', DATE '2021-09-01');
62
63 INSERT INTO STUDENTS (student_id, first_name, last_name, date_of_birth,
64     email, class_id, enrollment_date)
65 VALUES ('STU003', 'Vo', 'Van F', DATE '2004-01-08',
66     'vovanf@student.hcmut.edu.vn', 'CLASS02', DATE '2022-09-01');
67
68 INSERT INTO STUDENTS (student_id, first_name, last_name, date_of_birth,
69     email, class_id, enrollment_date)
70 VALUES ('STU004', 'Dang', 'Minh G', DATE '2003-11-30',
71     'dangmg@student.hcmut.edu.vn', 'CLASS03', DATE '2021-09-01');
72
73 -- Insert test relatives
74 INSERT INTO RELATIVES (relative_id, first_name, last_name, phone_number)
75 VALUES ('REL001', 'Pham', 'Van H', '0901234567');
76
77 INSERT INTO RELATIVES (relative_id, first_name, last_name, phone_number)
78 VALUES ('REL002', 'Pham', 'Thi I', '0901234568');
79
80 INSERT INTO RELATIVES (relative_id, first_name, last_name, phone_number)
81 VALUES ('REL003', 'Hoang', 'Van K', '0907654321');
82
83 -- Link students with relatives
84 INSERT INTO STUDENT_RELATIVES (student_id, relative_id, relationship,
85     is_primary_contact)
86 VALUES ('STU001', 'REL001', 'Father', 'Y');
87
88 INSERT INTO STUDENT_RELATIVES (student_id, relative_id, relationship,
89     is_primary_contact)
90 VALUES ('STU001', 'REL002', 'Mother', 'N');
91
92 INSERT INTO STUDENT_RELATIVES (student_id, relative_id, relationship,
```



```
is_primary_contact)
91 VALUES ('STU002', 'RELO03', 'Father', 'Y');
92
93 -- Insert test courses
94 INSERT INTO COURSES (course_id, course_name, credits, department_id, course_type)
95 VALUES ('CS101', 'Lap trinh co ban', 3, 'DEPT01', 'Mandatory');
96
97 INSERT INTO COURSES (course_id, course_name, credits, department_id, course_type)
98 VALUES ('CS201', 'Cau truc du lieu', 4, 'DEPT01', 'Mandatory');
99
100 INSERT INTO COURSES (course_id, course_name, credits, department_id, course_type)
101 VALUES ('CS301', 'Co so du lieu', 3, 'DEPT01', 'Mandatory');
102
103 -- Insert test course sections
104 INSERT INTO COURSE_SECTIONS (section_id, course_id, lecturer_id,
105     semester, academic_year, max_students)
106 VALUES ('CS101-01-HK1', 'CS101', 'LEC001', 'HK1', 2024, 50);
107
108 INSERT INTO COURSE_SECTIONS (section_id, course_id, lecturer_id,
109     semester, academic_year, max_students)
110 VALUES ('CS201-01-HK1', 'CS201', 'LEC001', 'HK1', 2024, 45);
111
112 INSERT INTO COURSE_SECTIONS (section_id, course_id, lecturer_id,
113     semester, academic_year, max_students)
114 VALUES ('CS301-01-HK2', 'CS301', 'LEC002', 'HK2', 2024, 40);
115
116 -- Insert test enrollments
117 INSERT INTO ENROLLMENTS (student_id, section_id, enrollment_status)
118 VALUES ('STU001', 'CS101-01-HK1', 'Enrolled');
119
120 INSERT INTO ENROLLMENTS (student_id, section_id, enrollment_status)
121 VALUES ('STU001', 'CS201-01-HK1', 'Enrolled');
122
123 INSERT INTO ENROLLMENTS (student_id, section_id, enrollment_status)
124 VALUES ('STU002', 'CS101-01-HK1', 'Enrolled');
125
126 INSERT INTO ENROLLMENTS (student_id, section_id, enrollment_status)
127 VALUES ('STU003', 'CS101-01-HK1', 'Enrolled');
128
129 -- Insert test grades
130 INSERT INTO GRADES (enrollment_id, midterm_score, final_score, total_score,
```



```
131     letter_grade, grade_status, submitted_by, submitted_date)
132 SELECT enrollment_id, 8.5, 9.0, 8.8, 'A', 'Approved', 'LEC001', SYSDATE
133 FROM ENROLLMENTS WHERE student_id = 'STU001' AND section_id = 'CS101-01-HK1';
134
135 INSERT INTO GRADES (enrollment_id, midterm_score, final_score, total_score,
136     letter_grade, grade_status, submitted_by, submitted_date)
137 SELECT enrollment_id, 7.0, 7.5, 7.3, 'B', 'Submitted', 'LEC001', SYSDATE
138 FROM ENROLLMENTS WHERE student_id = 'STU001' AND section_id = 'CS201-01-HK1';
139
140 INSERT INTO GRADES (enrollment_id, midterm_score, final_score, total_score,
141     letter_grade, grade_status, submitted_by, submitted_date)
142 SELECT enrollment_id, 6.5, 7.0, 6.8, 'C', 'Approved', 'LEC001', SYSDATE
143 FROM ENROLLMENTS WHERE student_id = 'STU002' AND section_id = 'CS101-01-HK1';
144
145 -- Insert system users
146 INSERT INTO SYSTEM_USERS (user_id, username, password_hash, user_type,
147     reference_id, is_active)
148 VALUES ('USR001', 'student001',
149     '$2a$10$hash...', 'Student', 'STU001', 'Y');
150
151 INSERT INTO SYSTEM_USERS (user_id, username, password_hash, user_type,
152     reference_id, is_active)
153 VALUES ('USR002', 'student002',
154     '$2a$10$hash...', 'Student', 'STU002', 'Y');
155
156 INSERT INTO SYSTEM_USERS (user_id, username, password_hash, user_type,
157     reference_id, is_active)
158 VALUES ('USR003', 'lecturer001',
159     '$2a$10$hash...', 'Lecturer', 'LEC001', 'Y');
160
161 INSERT INTO SYSTEM_USERS (user_id, username, password_hash, user_type,
162     reference_id, is_active)
163 VALUES ('USR004', 'dean001',
164     '$2a$10$hash...', 'Dean', 'LEC002', 'Y');
165
166 INSERT INTO SYSTEM_USERS (user_id, username, password_hash, user_type,
167     reference_id, is_active)
168 VALUES ('USR005', 'relative001',
169     '$2a$10$hash...', 'Relative', 'REL001', 'Y');
170
171 INSERT INTO SYSTEM_USERS (user_id, username, password_hash, user_type,
```

```
172     reference_id, is_active)
173 VALUES ('USR006', 'academic001',
174         '$2a$10$hash...', 'Academic_Affairs', 'EMP001', 'Y');
175
176 COMMIT;
```

Listing 81: Script chuẩn bị môi trường test

68.2 Danh sách user test

Bảng 18: Danh sách user dùng cho test

Username	Role	Reference ID	Mô tả
student001	Student	STU001	Sinh viên lớp CLASS01
student002	Student	STU002	Sinh viên lớp CLASS01
student003	Student	STU003	Sinh viên lớp CLASS02
student004	Student	STU004	Sinh viên khoa khác
lecturer001	Lecturer	LEC001	Giảng viên DEPT01
lecturer002	Lecturer	LEC002	Giảng viên DEPT02
dean001	Dean	LEC002	Trưởng khoa FAC01
relative001	Relative	REL001	Phụ huynh STU001
relative003	Relative	REL003	Phụ huynh STU002
academic001	Academic_Affairs	EMP001	Phòng đào tạo
depthead001	Department_Head	LEC001	Trưởng bộ môn DEPT01

69 Test Cases cho VPD - Student

69.1 TC-VPD-STU-01: Student chỉ xem được thông tin của mình

```
1 -- =====
2 -- TC-VPD-STU-01: Student View Own Profile
3 -- Expected: Student STU001 chỉ thay record của mình
4 -- =====
5
6 -- Set context for student001 (reference_id = STU001)
7 EXEC gms_security_pkg.set_user_context('STU001', 'Student');
```



```
8
9 -- Test SELECT on STUDENTS table
10 SELECT student_id, first_name, last_name, email, class_id
11 FROM STUDENTS;
12
13 -- Expected Result:
14 -- STUDENT_ID FIRST_NAME LAST_NAME EMAIL CLASS_ID
15 -- -----
16 -- STU001 Pham Minh D phammd@student.hcmut.edu.vn CLASS01
17
18 -- Verify: Only 1 row returned (STU001's own record)
19 -- Other students (STU002, STU003, STU004) are NOT visible
```

Listing 82: TC-VPD-STU-01: Student xem profile

Kết quả mong đợi:

- Chỉ trả về 1 dòng - thông tin của STU001
- Các sinh viên khác (STU002, STU003, STU004) không hiển thị
- VPD policy tự động thêm điều kiện `student_id = 'STU001'`

69.2 TC-VPD-STU-02: Student xem điểm của mình

```
1 -- =====
2 -- TC-VPD-STU-02: Student View Own Grades
3 -- Expected: Student STU001 chỉ thấy điểm của mình
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('STU001', 'Student');
7
8 -- Test SELECT on GRADES through V_STUDENT_GRADES view
9 SELECT student_id, course_name, midterm_score, final_score,
10        total_score, letter_grade, grade_status
11 FROM V_STUDENT_GRADES
12 WHERE student_id = SYS_CONTEXT('gms_context', 'user_id');
13
14 -- Expected Result:
15 -- STUDENT_ID COURSE_NAME MIDTERM FINAL TOTAL GRADE STATUS
16 -- -----
17 -- STU001 Lap trình co ban 8.5 9.0 8.8 A Approved
```



```
18 -- STU001 Cau truc du lieu 7.0 7.5 7.3 B Submitted
19
20 -- Verify:
21 -- - Only grades for STU001 are visible
22 -- - Cannot see STU002's grade (6.8 - C)
```

Listing 83: TC-VPD-STU-02: Student xem điểm

69.3 TC-VPD-STU-03: Student không thể xem điểm của người khác

```
1 -- =====
2 -- TC-VPD-STU-03: Student Cannot View Other's Grades
3 -- Expected: Query returns 0 rows
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('STU001', 'Student');
7
8 -- Attempt to see another student's grades
9 SELECT g.grade_id, e.student_id, g.total_score, g.letter_grade
10 FROM GRADES g
11 JOIN ENROLLMENTS e ON g.enrollment_id = e.enrollment_id
12 WHERE e.student_id = 'STU002'; -- Try to see STU002's grades
13
14 -- Expected Result: 0 rows
15 -- VPD policy blocks access to other students' enrollment records
16
17 -- Even with direct enrollment_id:
18 SELECT * FROM ENROLLMENTS WHERE student_id = 'STU002';
19 -- Expected Result: 0 rows (VPD blocks)
```

Listing 84: TC-VPD-STU-03: Student không xem được điểm người khác

69.4 TC-VPD-STU-04: Student cập nhật thông tin liên hệ

```
1 -- =====
2 -- TC-VPD-STU-04: Student Update Contact Info
3 -- Expected: Can update only allowed columns in own record
4 -- =====
5
```



```
6 EXEC gms_security_pkg.set_user_context('STU001', 'Student');
7
8 -- Test 1: Update allowed columns (should succeed)
9 UPDATE STUDENTS
10 SET email = 'newemail@student.hcmut.edu.vn',
11     phone_number = '0912345678',
12     contact_address = '123 New Street, District 1'
13 WHERE student_id = 'STU001';
14 -- Expected: 1 row updated
15
16 COMMIT;
17
18 -- Test 2: Attempt to update restricted column (should fail)
19 UPDATE STUDENTS
20 SET first_name = 'Hacker'
21 WHERE student_id = 'STU001';
22 -- Expected: ORA-01031: insufficient privileges
23 -- (Column-level grant only allows email, phone_number, contact_address)
24
25 -- Test 3: Attempt to update another student (should fail)
26 UPDATE STUDENTS
27 SET email = 'hacked@email.com'
28 WHERE student_id = 'STU002';
29 -- Expected: 0 rows updated (VPD blocks access to STU002's record)
30
31 ROLLBACK;
```

Listing 85: TC-VPD-STU-04: Student cập nhật profile

70 Test Cases cho VPD - Lecturer

70.1 TC-VPD-LEC-01: Lecturer xem sinh viên trong lớp dạy

```
1 -- =====
2 -- TC-VPD-LEC-01: Lecturer View Students in Teaching Sections
3 -- Expected: See students enrolled in sections they teach
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('LEC001', 'Lecturer');
7
```



```
8 -- View students in sections taught by LEC001
9 SELECT DISTINCT s.student_id, s.first_name, s.last_name,
10                s.class_id, cs.section_id, c.course_name
11 FROM STUDENTS s
12 JOIN ENROLLMENTS e ON s.student_id = e.student_id
13 JOIN COURSE_SECTIONS cs ON e.section_id = cs.section_id
14 JOIN COURSES c ON cs.course_id = c.course_id
15 WHERE cs.lecturer_id = SYS_CONTEXT('gms_context', 'user_id');
16
17 -- Expected Result (LEC001 teaches CS101-01-HK1 and CS201-01-HK1):
18 -- STUDENT_ID FIRST_NAME LAST_NAME CLASS_ID SECTION_ID COURSE_NAME
19 -- -----
20 -- STU001 Pham Minh D CLASS01 CS101-01-HK1 Lap trinh co ban
21 -- STU001 Pham Minh D CLASS01 CS201-01-HK1 Cau truc du lieu
22 -- STU002 Hoang Thi E CLASS01 CS101-01-HK1 Lap trinh co ban
23 -- STU003 Vo Van F CLASS02 CS101-01-HK1 Lap trinh co ban
24
25 -- Note: STU004 not visible (not enrolled in LEC001's sections)
```

Listing 86: TC-VPD-LEC-01: Lecturer xem sinh viên

70.2 TC-VPD-LEC-02: Lecturer nhập điểm cho sinh viên

```
1 -- =====
2 -- TC-VPD-LEC-02: Lecturer Enter Grades
3 -- Expected: Can insert/update grades for students in own sections
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('LEC001', 'Lecturer');
7
8 -- Get enrollment_id for STU003 in CS101-01-HK1
9 SELECT enrollment_id FROM ENROLLMENTS
10 WHERE student_id = 'STU003' AND section_id = 'CS101-01-HK1';
11 -- Assume enrollment_id = 4
12
13 -- Insert new grade
14 INSERT INTO GRADES (enrollment_id, midterm_score, final_score,
15                    total_score, letter_grade, grade_status, submitted_by, submitted_date)
16 VALUES (4, 7.5, 8.0, 7.8, 'B', 'Submitted', 'LEC001', SYSDATE);
17 -- Expected: 1 row inserted (LEC001 teaches this section)
18
```

```
19 COMMIT;
20
21 -- Verify the grade
22 SELECT g.grade_id, e.student_id, g.total_score, g.grade_status
23 FROM GRADES g
24 JOIN ENROLLMENTS e ON g.enrollment_id = e.enrollment_id
25 WHERE e.student_id = 'STU003';
26 -- Expected: 1 row with the newly inserted grade
```

Listing 87: TC-VPD-LEC-02: Lecturer nhập điểm

70.3 TC-VPD-LEC-03: Lecturer không thể nhập điểm cho section khác

```
1 -- =====
2 -- TC-VPD-LEC-03: Lecturer Cannot Enter Grades for Other Sections
3 -- Expected: Operation fails or 0 rows affected
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('LEC001', 'Lecturer');
7
8 -- CS301-01-HK2 is taught by LEC002, not LEC001
9 -- Try to view enrollment for that section
10 SELECT enrollment_id FROM ENROLLMENTS
11 WHERE section_id = 'CS301-01-HK2';
12 -- Expected: 0 rows (VPD blocks - LEC001 doesn't teach this section)
13
14 -- Even if we somehow got the enrollment_id, insert would fail
15 -- because VPD policy checks lecturer_id of the section
16 INSERT INTO GRADES (enrollment_id, midterm_score, final_score,
17     total_score, letter_grade, grade_status, submitted_by)
18 VALUES (999, 10.0, 10.0, 10.0, 'A+', 'Submitted', 'LEC001');
19 -- Expected: Error or no record inserted
20 -- (enrollment_id 999 not visible to LEC001)
21
22 ROLLBACK;
```

Listing 88: TC-VPD-LEC-03: Lecturer không nhập điểm section khác

70.4 TC-VPD-LEC-04: Lecturer cập nhật điểm đã nhập

```
1 -- =====
2 -- TC-VPD-LEC-04: Lecturer Update Submitted Grades
3 -- Expected: Can update grades with status 'Submitted' only
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('LEC001', 'Lecturer');
7
8 -- Find grade to update (Submitted status)
9 SELECT g.grade_id, e.student_id, g.total_score, g.grade_status
10 FROM GRADES g
11 JOIN ENROLLMENTS e ON g.enrollment_id = e.enrollment_id
12 WHERE g.grade_status = 'Submitted' AND g.submitted_by = 'LEC001';
13
14 -- Update the grade (assume grade_id = 2)
15 UPDATE GRADES
16 SET midterm_score = 7.5,
17     final_score = 8.0,
18     total_score = 7.8,
19     letter_grade = 'B',
20     modified_by = 'LEC001',
21     modified_date = SYSDATE,
22     modification_reason = 'Score correction after review'
23 WHERE grade_id = 2 AND grade_status = 'Submitted';
24 -- Expected: 1 row updated
25
26 -- Attempt to update Approved grade (should fail or be blocked)
27 UPDATE GRADES
28 SET total_score = 10.0
29 WHERE grade_status = 'Approved';
30 -- This should be blocked by application logic or trigger
31 -- (Approved grades cannot be modified without Dean approval)
32
33 ROLLBACK;
```

Listing 89: TC-VPD-LEC-04: Lecturer cập nhật điểm

71 Test Cases cho VPD - Dean

71.1 TC-VPD-DEAN-01: Dean xem tất cả sinh viên trong khoa

```
1 -- =====
2 -- TC-VPD-DEAN-01: Dean View All Students in Faculty
3 -- Expected: See all students in classes belonging to dean's faculty
4 -- =====
5
6 -- Dean of FAC01 (Computer Science)
7 EXEC gms_security_pkg.set_user_context('LEC002', 'Dean');
8
9 -- View all students in FAC01
10 SELECT s.student_id, s.first_name, s.last_name,
11        c.class_name, f.faculty_name
12 FROM STUDENTS s
13 JOIN CLASSES c ON s.class_id = c.class_id
14 JOIN FACULTIES f ON c.faculty_id = f.faculty_id;
15
16 -- Expected Result (all CS students):
17 -- STUDENT_ID FIRST_NAME LAST_NAME CLASS_NAME FACULTY_NAME
18 -- -----
19 -- STU001 Pham Minh D CNTT 2021 Khoa Khoa hoc May tinh
20 -- STU002 Hoang Thi E CNTT 2021 Khoa Khoa hoc May tinh
21 -- STU003 Vo Van F CNTT 2022 Khoa Khoa hoc May tinh
22
23 -- Note: STU004 (Dien-Dien tu faculty) is NOT visible
```

Listing 90: TC-VPD-DEAN-01: Dean xem sinh viên trong khoa

71.2 TC-VPD-DEAN-02: Dean xem tất cả điểm trong khoa

```
1 -- =====
2 -- TC-VPD-DEAN-02: Dean View All Grades in Faculty
3 -- Expected: See all grades for courses in dean's faculty
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('LEC002', 'Dean');
7
8 -- View all grades in faculty
```



```
9 SELECT s.student_id, s.first_name || ' ' || s.last_name AS student_name,
10        c.course_name, g.midterm_score, g.final_score,
11        g.total_score, g.letter_grade, g.grade_status
12 FROM GRADES g
13 JOIN ENROLLMENTS e ON g.enrollment_id = e.enrollment_id
14 JOIN STUDENTS s ON e.student_id = s.student_id
15 JOIN COURSE_SECTIONS cs ON e.section_id = cs.section_id
16 JOIN COURSES c ON cs.course_id = c.course_id
17 JOIN DEPARTMENTS d ON c.department_id = d.department_id
18 WHERE d.faculty_id = 'FAC01';
19
20 -- Expected: All grades for FAC01 courses
21 -- STUDENT_ID STUDENT_NAME COURSE_NAME TOTAL GRADE STATUS
22 -- -----
23 -- STU001 Pham Minh D Lap trinh co ban 8.8 A Approved
24 -- STU001 Pham Minh D Cau truc du lieu 7.3 B Submitted
25 -- STU002 Hoang Thi E Lap trinh co ban 6.8 C Approved
26 -- STU003 Vo Van F Lap trinh co ban 7.8 B Submitted
```

Listing 91: TC-VPD-DEAN-02: Dean xem điểm toàn khoa

71.3 TC-VPD-DEAN-03: Dean phê duyệt điểm

```
1 -- =====
2 -- TC-VPD-DEAN-03: Dean Approve Grades
3 -- Expected: Can approve submitted grades in faculty
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('LEC002', 'Dean');
7
8 -- View grades pending approval
9 SELECT g.grade_id, e.student_id, c.course_name,
10        g.total_score, g.grade_status
11 FROM GRADES g
12 JOIN ENROLLMENTS e ON g.enrollment_id = e.enrollment_id
13 JOIN COURSE_SECTIONS cs ON e.section_id = cs.section_id
14 JOIN COURSES c ON cs.course_id = c.course_id
15 WHERE g.grade_status = 'Submitted';
16
17 -- Approve a grade (assume grade_id = 2)
18 UPDATE GRADES
```



```
19 SET grade_status = 'Approved',
20     approved_by = 'LEC002',
21     approved_date = SYSDATE
22 WHERE grade_id = 2 AND grade_status = 'Submitted';
23 -- Expected: 1 row updated
24
25 -- Verify approval
26 SELECT grade_id, grade_status, approved_by, approved_date
27 FROM GRADES WHERE grade_id = 2;
28 -- Expected: Status = 'Approved', approved_by = 'LEC002'
29
30 COMMIT;
```

Listing 92: TC-VPD-DEAN-03: Dean phê duyệt điểm

71.4 TC-VPD-DEAN-04: Dean không thể thao tác khoa khác

```
1 -- =====
2 -- TC-VPD-DEAN-04: Dean Cannot Access Other Faculty Data
3 -- Expected: No access to FAC02 data
4 -- =====
5
6 -- Dean of FAC01 trying to access FAC02 data
7 EXEC gms_security_pkg.set_user_context('LEC002', 'Dean');
8
9 -- Try to view students in FAC02 (Dien-Dien tu)
10 SELECT s.student_id, s.first_name, s.last_name
11 FROM STUDENTS s
12 JOIN CLASSES c ON s.class_id = c.class_id
13 WHERE c.faculty_id = 'FAC02';
14 -- Expected: 0 rows (VPD blocks access to other faculty)
15
16 -- Even using direct student_id
17 SELECT * FROM STUDENTS WHERE student_id = 'STU004';
18 -- Expected: 0 rows (STU004 belongs to FAC02)
19
20 -- Cannot modify grades in other faculty courses
21 UPDATE GRADES
22 SET grade_status = 'Approved'
23 WHERE grade_id IN (
24     SELECT g.grade_id FROM GRADES g
```

```
25 JOIN ENROLLMENTS e ON g.enrollment_id = e.enrollment_id
26 JOIN COURSE_SECTIONS cs ON e.section_id = cs.section_id
27 JOIN COURSES c ON cs.course_id = c.course_id
28 JOIN DEPARTMENTS d ON c.department_id = d.department_id
29 WHERE d.faculty_id = 'FAC02'
30 );
31 -- Expected: 0 rows updated
```

Listing 93: TC-VPD-DEAN-04: Dean không thao tác khoa khác

72 Test Cases cho VPD - Relative (Phụ huynh)

72.1 TC-VPD-REL-01: Relative xem thông tin con em

```
1 -- =====
2 -- TC-VPD-REL-01: Relative View Linked Student Info
3 -- Expected: Can see info of linked students only
4 -- =====
5
6 -- REL001 is linked to STU001 (Father)
7 EXEC gms_security_pkg.set_user_context('REL001', 'Relative');
8
9 -- View linked student info
10 SELECT s.student_id, s.first_name, s.last_name,
11        s.class_id, s.email, s.student_status
12 FROM STUDENTS s
13 JOIN STUDENT_RELATIVES sr ON s.student_id = sr.student_id
14 WHERE sr.relative_id = SYS_CONTEXT('gms_context', 'user_id');
15
16 -- Expected Result:
17 -- STUDENT_ID FIRST_NAME LAST_NAME CLASS_ID EMAIL STATUS
18 -- -----
19 -- STU001 Pham Minh D CLASS01 phammd@student.hcmut.edu.vn Active
20
21 -- Note: REL001 can only see STU001's info
22 -- Cannot see STU002 (linked to REL003, different family)
```

Listing 94: TC-VPD-REL-01: Relative xem thông tin sinh viên

72.2 TC-VPD-REL-02: Relative xem điểm con em

```
1 -- =====
2 -- TC-VPD-REL-02: Relative View Student Grades
3 -- Expected: Can view grades of linked students
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('REL001', 'Relative');
7
8 -- View grades of linked student
9 SELECT s.student_id, s.first_name || ' ' || s.last_name AS student_name,
10        c.course_name, g.midterm_score, g.final_score,
11        g.total_score, g.letter_grade
12 FROM STUDENTS s
13 JOIN STUDENT_RELATIVES sr ON s.student_id = sr.student_id
14 JOIN ENROLLMENTS e ON s.student_id = e.student_id
15 JOIN COURSE_SECTIONS cs ON e.section_id = cs.section_id
16 JOIN COURSES c ON cs.course_id = c.course_id
17 JOIN GRADES g ON e.enrollment_id = g.enrollment_id
18 WHERE sr.relative_id = SYS_CONTEXT('gms_context', 'user_id')
19 AND g.grade_status = 'Approved'; -- Only approved grades
20
21 -- Expected Result:
22 -- STUDENT_ID STUDENT_NAME COURSE_NAME MIDTERM FINAL TOTAL GRADE
23 -- -----
24 -- STU001 Pham Minh D Lap trinh co ban 8.5 9.0 8.8 A
25
26 -- Note: Only approved grades are shown to relatives
27 -- Submitted/Pending grades are hidden until official
```

Listing 95: TC-VPD-REL-02: Relative xem điểm sinh viên

72.3 TC-VPD-REL-03: Relative không thể xem sinh viên không liên quan

```
1 -- =====
2 -- TC-VPD-REL-03: Relative Cannot View Unlinked Students
3 -- Expected: 0 rows when querying other students
4 -- =====
5
```



```
6 EXEC gms_security_pkg.set_user_context('REL001', 'Relative');
7
8 -- Try to view STU002 directly (not linked to REL001)
9 SELECT * FROM STUDENTS WHERE student_id = 'STU002';
10 -- Expected: 0 rows (VPD blocks - STU002 linked to REL003)
11
12 -- Try to view all students
13 SELECT student_id, first_name, last_name FROM STUDENTS;
14 -- Expected: 1 row only (STU001)
15
16 -- Try to view another relative's student's grades
17 SELECT g.* FROM GRADES g
18 JOIN ENROLLMENTS e ON g.enrollment_id = e.enrollment_id
19 WHERE e.student_id = 'STU002';
20 -- Expected: 0 rows
```

Listing 96: TC-VPD-REL-03: Relative không xem được sinh viên khác

73 Test Cases cho VPD - Academic Affairs

73.1 TC-VPD-AA-01: Academic Affairs xem tất cả sinh viên

```
1 -- =====
2 -- TC-VPD-AA-01: Academic Affairs View All Students
3 -- Expected: Can see all students across all faculties
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('EMP001', 'Academic_Affairs');
7
8 -- View all students
9 SELECT student_id, first_name, last_name, class_id,
10        enrollment_date, student_status
11 FROM STUDENTS
12 ORDER BY class_id, student_id;
13
14 -- Expected Result (ALL students):
15 -- STUDENT_ID FIRST_NAME LAST_NAME CLASS_ID ENROLL_DATE STATUS
16 -- -----
17 -- STU001 Pham Minh D CLASS01 2021-09-01 Active
18 -- STU002 Hoang Thi E CLASS01 2021-09-01 Active
```



```
19 -- STU003 Vo Van F CLASS02 2022-09-01 Active
20 -- STU004 Dang Minh G CLASS03 2021-09-01 Active
21
22 -- Academic Affairs can see across all faculties
23 -- This is necessary for registration, graduation processing
```

Listing 97: TC-VPD-AA-01: Academic Affairs xem sinh viên

73.2 TC-VPD-AA-02: Academic Affairs quản lý đăng ký học

```
1 -- =====
2 -- TC-VPD-AA-02: Academic Affairs Manage Enrollments
3 -- Expected: Full CRUD on enrollments
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('EMP001', 'Academic_Affairs');
7
8 -- View all enrollments
9 SELECT e.enrollment_id, e.student_id, e.section_id,
10        e.enrollment_status, e.enrollment_date
11 FROM ENROLLMENTS e;
12 -- Expected: All enrollment records visible
13
14 -- Add new enrollment
15 INSERT INTO ENROLLMENTS (student_id, section_id, enrollment_status)
16 VALUES ('STU004', 'CS101-01-HK1', 'Enrolled');
17 -- Expected: 1 row inserted
18
19 -- Modify enrollment status
20 UPDATE ENROLLMENTS
21 SET enrollment_status = 'Dropped'
22 WHERE student_id = 'STU004' AND section_id = 'CS101-01-HK1';
23 -- Expected: 1 row updated
24
25 -- Academic Affairs has broader access for administrative tasks
26
27 ROLLBACK;
```

Listing 98: TC-VPD-AA-02: Academic Affairs quản lý đăng ký

73.3 TC-VPD-AA-03: Academic Affairs không thể sửa điểm

lstlisting[style=sqlstyle, caption=TC-VPD-AA-03: Academic Affairs không sửa điểm]

```

=====
- TC-VPD-AA-03: Academic Affairs Cannot Modify Grades - Ex-
pected: Cannot INSERT/UPDATE grades (read-only for grades) -
=====

EXEC gms_securitypkg.set_user_context('EMP001','AcademicAffairs');
- Can VIEW grades (for transcript generation) SELECT
g.grade_id, e.student_id, g.total_score, g.letter_grade FROM GRADES g JOIN ENROLLMENTS e ON
e.enrollment_id; -- Expected : All grades visible (SELECT allowed)
- Cannot INSERT grades INSERT INTO GRADES
(enrollment_id, total_score, letter_grade, grade_status) VALUES (1, 10.0, 'A+', 'Approved'); --
Expected : ORA - 01031 or blocked by VPD INSERT policy -
-(Only Lecturers and Deans can enter grades)
- Cannot UPDATE grades UPDATE GRADES SET total_score =
10.0 WHERE grade_id = 1; -- Expected : 0 rows updated or error -
-(Grade modification requires Lecturer or Dean role)

```

74 Test Cases cho Password Policies

74.1 TC-PWD-01: Test password complexity

```

1 -- =====
2 -- TC-PWD-01: Password Complexity Validation
3 -- Expected: Weak passwords are rejected
4 -- =====
5
6 -- Connect as SYSDBA
7 -- sqlplus sys/password@localhost:1521/ORCLPDB as sysdba
8
9 -- Test 1: Password too short (< 8 chars)
10 ALTER USER GMS_STUDENT IDENTIFIED BY "Short1!";
11 -- Expected: ORA-28003: password verification failed
12 -- Error: Password must be at least 8 characters
13
14 -- Test 2: Password without uppercase

```

```
15 ALTER USER GMS_STUDENT IDENTIFIED BY "alllowercase1!";
16 -- Expected: ORA-28003: password verification failed
17 -- Error: Password must contain uppercase letter
18
19 -- Test 3: Password without lowercase
20 ALTER USER GMS_STUDENT IDENTIFIED BY "ALLUPPERCASE1!";
21 -- Expected: ORA-28003: password verification failed
22 -- Error: Password must contain lowercase letter
23
24 -- Test 4: Password without digit
25 ALTER USER GMS_STUDENT IDENTIFIED BY "NoDigitsHere!";
26 -- Expected: ORA-28003: password verification failed
27 -- Error: Password must contain digit
28
29 -- Test 5: Password without special char
30 ALTER USER GMS_STUDENT IDENTIFIED BY "NoSpecial123";
31 -- Expected: ORA-28003: password verification failed
32 -- Error: Password must contain special character
33
34 -- Test 6: Valid password
35 ALTER USER GMS_STUDENT IDENTIFIED BY "ValidPass123!";
36 -- Expected: User altered (success)
37
38 -- Test 7: Password same as username
39 ALTER USER GMS_STUDENT IDENTIFIED BY "GMS_STUDENT";
40 -- Expected: ORA-28003: password cannot be same as username
41
42 -- Test 8: Password contains username
43 ALTER USER GMS_STUDENT IDENTIFIED BY "GMS_STUDENT123!";
44 -- Expected: ORA-28003: password cannot contain username
```

Listing 99: TC-PWD-01: Test password complexity

74.2 TC-PWD-02: Test account lockout

```
1 -- =====
2 -- TC-PWD-02: Account Lockout After Failed Attempts
3 -- Expected: Account locks after 5 failed attempts (Student profile)
4 -- =====
5
6 -- Attempt to login with wrong password 5 times
```

```
7 -- sqlplus GMS_STUDENT/wrongpass1@ORCLPDB
8 -- sqlplus GMS_STUDENT/wrongpass2@ORCLPDB
9 -- sqlplus GMS_STUDENT/wrongpass3@ORCLPDB
10 -- sqlplus GMS_STUDENT/wrongpass4@ORCLPDB
11 -- sqlplus GMS_STUDENT/wrongpass5@ORCLPDB
12
13 -- After 5 failed attempts:
14 -- Expected: ORA-28000: The account is locked
15
16 -- Check account status (as SYSDBA)
17 SELECT username, account_status, lock_date
18 FROM dba_users WHERE username = 'GMS_STUDENT';
19 -- Expected: ACCOUNT_STATUS = 'LOCKED(TIMED)'
20
21 -- Account auto-unlocks after 30 minutes (as per profile)
22 -- Or admin can unlock manually:
23 ALTER USER GMS_STUDENT ACCOUNT UNLOCK;
24
25 -- For SYSADMIN profile (stricter):
26 -- Locks after 3 failed attempts
27 -- No auto-unlock (indefinite until admin unlocks)
```

Listing 100: TC-PWD-02: Test account lockout

74.3 TC-PWD-03: Test password expiry

```
1 -- =====
2 -- TC-PWD-03: Password Expiry Policy
3 -- Expected: Password expires after defined period
4 -- =====
5
6 -- Check password expiry settings per profile
7 SELECT profile, resource_name, limit
8 FROM dba_profiles
9 WHERE resource_name = 'PASSWORD_LIFE_TIME'
10 AND profile LIKE 'GMS_%';
11
12 -- Expected:
13 -- PROFILE RESOURCE_NAME LIMIT
14 -- -----
15 -- GMS_STUDENT_PROFILE PASSWORD_LIFE_TIME 180 (6 months)
```



```
16 -- GMS_LECTURER_PROFILE PASSWORD_LIFE_TIME 90 (3 months)
17 -- GMS_ADMIN_PROFILE PASSWORD_LIFE_TIME 60 (2 months)
18 -- GMS_SYSADMIN_PROFILE PASSWORD_LIFE_TIME 30 (1 month)
19
20 -- Check password expiry date for users
21 SELECT username, expiry_date,
22        ROUND(expiry_date - SYSDATE) AS days_until_expiry
23 FROM dba_users
24 WHERE username LIKE 'GMS_%';
25
26 -- Grace period test (warning before expiry)
27 -- Profile has PASSWORD_GRACE_TIME = 7 days
28 -- User gets warning 7 days before expiry
29 -- After grace period: ORA-28001: the password has expired
```

Listing 101: TC-PWD-03: Test password expiry

75 Test Cases cho Fine-Grained Auditing

75.1 TC-FGA-01: Audit grade viewing

```
1 -- =====
2 -- TC-FGA-01: Audit Grade Viewing
3 -- Expected: SELECT on GRADES table is logged
4 -- =====
5
6 -- As Student
7 EXEC gms_security_pkg.set_user_context('STU001', 'Student');
8
9 -- View grades (this triggers FGA)
10 SELECT grade_id, total_score, letter_grade
11 FROM GRADES g
12 JOIN ENROLLMENTS e ON g.enrollment_id = e.enrollment_id
13 WHERE e.student_id = 'STU001';
14
15 -- Check audit log (as SYSDBA or GMS_ADMIN)
16 SELECT timestamp, db_user, os_user, userhost,
17        object_schema, object_name, policy_name,
18        sql_text
19 FROM dba_fga_audit_trail
```



```
20 WHERE object_name = 'GRADES'
21 AND policy_name = 'AUDIT_GRADES_SELECT'
22 ORDER BY timestamp DESC
23 FETCH FIRST 5 ROWS ONLY;
24
25 -- Expected audit record:
26 -- TIMESTAMP DB_USER OS_USER OBJECT_NAME POLICY_NAME
27 -- -----
28 -- 2024-12-07 GMS_STUDENT user GRADES AUDIT_GRADES_SELECT
29 -- SQL_TEXT: SELECT grade_id, total_score...
```

Listing 102: TC-FGA-01: Audit grade SELECT

75.2 TC-FGA-02: Audit grade modifications

```
1 -- =====
2 -- TC-FGA-02: Audit Grade Modifications
3 -- Expected: UPDATE on GRADES is logged with old/new values
4 -- =====
5
6 -- As Lecturer
7 EXEC gms_security_pkg.set_user_context('LEC001', 'Lecturer');
8
9 -- Update a grade
10 UPDATE GRADES
11 SET midterm_score = 8.0, total_score = 8.2
12 WHERE grade_id = 1;
13 COMMIT;
14
15 -- Check audit log
16 SELECT timestamp, db_user, object_name,
17        policy_name, sql_text
18 FROM dba_fga_audit_trail
19 WHERE object_name = 'GRADES'
20 AND policy_name = 'AUDIT_GRADES_UPDATE'
21 ORDER BY timestamp DESC
22 FETCH FIRST 1 ROWS ONLY;
23
24 -- Also check custom AUDIT_LOG table (trigger-based)
25 SELECT log_id, operation, user_id, username,
26        record_id, old_values, new_values, operation_date
```




```
27 FROM AUDIT_LOG
28 WHERE table_name = 'GRADES'
29 ORDER BY operation_date DESC
30 FETCH FIRST 1 ROWS ONLY;
31
32 -- Expected:
33 -- LOG_ID OPERATION USER_ID RECORD_ID OLD_VALUES NEW_VALUES
34 -- -----
35 -- 101 UPDATE LEC001 1 {midterm:7.5,total:7.8} {midterm:8.0,total:8.2}
```

Listing 103: TC-FGA-02: Audit grade modifications

75.3 TC-FGA-03: Audit student information access

```
1 -- =====
2 -- TC-FGA-03: Audit Access to Sensitive Student Data
3 -- Expected: Access to STUDENTS table is logged
4 -- =====
5
6 -- Different users accessing student data
7 EXEC gms_security_pkg.set_user_context('REL001', 'Relative');
8
9 -- Relative views student info
10 SELECT student_id, first_name, last_name, email
11 FROM STUDENTS;
12
13 -- Check audit trail
14 SELECT timestamp, db_user, policy_name,
15        SUBSTR(sql_text, 1, 100) as query
16 FROM dba_fga_audit_trail
17 WHERE object_name = 'STUDENTS'
18 ORDER BY timestamp DESC
19 FETCH FIRST 5 ROWS ONLY;
20
21 -- FGA captures:
22 -- - Who accessed the data (db_user, os_user)
23 -- - When (timestamp)
24 -- - What query was executed (sql_text)
25 -- - From where (userhost, terminal)
26 -- - Session info (session_id)
```

Listing 104: TC-FGA-03: Audit student info access

76 Test Cases cho Column-Level Security

76.1 TC-COL-01: Student chỉ update được cột cho phép

```
1  -- =====
2  -- TC-COL-01: Column-Level UPDATE Restrictions
3  -- Expected: Can only update allowed columns
4  -- =====
5
6  -- Connect as GMS_STUDENT (has limited column UPDATE)
7  -- sqlplus GMS_STUDENT/password@ORCLPDB
8
9  EXEC gms_security_pkg.set_user_context('STU001', 'Student');
10
11 -- Test 1: Update allowed column - email
12 UPDATE GMS_ADMIN.STUDENTS
13 SET email = 'newemail@student.hcmut.edu.vn'
14 WHERE student_id = 'STU001';
15 -- Expected: 1 row updated (SUCCESS)
16
17 -- Test 2: Update allowed column - phone
18 UPDATE GMS_ADMIN.STUDENTS
19 SET phone_number = '0912345678'
20 WHERE student_id = 'STU001';
21 -- Expected: 1 row updated (SUCCESS)
22
23 -- Test 3: Update allowed column - address
24 UPDATE GMS_ADMIN.STUDENTS
25 SET contact_address = 'New Address 123'
26 WHERE student_id = 'STU001';
27 -- Expected: 1 row updated (SUCCESS)
28
29 -- Test 4: Update restricted column - first_name
30 UPDATE GMS_ADMIN.STUDENTS
31 SET first_name = 'Hacker'
32 WHERE student_id = 'STU001';
33 -- Expected: ORA-01031: insufficient privileges
34 -- (Column first_name not in GRANT UPDATE list)
35
36 -- Test 5: Update restricted column - class_id
```

```
37 UPDATE GMS_ADMIN.STUDENTS
38 SET class_id = 'CLASS99'
39 WHERE student_id = 'STU001';
40 -- Expected: ORA-01031: insufficient privileges
41
42 -- Test 6: Update restricted column - student_status
43 UPDATE GMS_ADMIN.STUDENTS
44 SET student_status = 'Graduated'
45 WHERE student_id = 'STU001';
46 -- Expected: ORA-01031: insufficient privileges
47
48 ROLLBACK;
```

Listing 105: TC-COL-01: Column-level UPDATE test

76.2 TC-COL-02: Verify column grants

```
1 -- =====
2 -- TC-COL-02: Verify Column-Level Grants Configuration
3 -- Expected: Correct columns are granted to each role
4 -- =====
5
6 -- As SYSDBA, check column privileges
7 SELECT grantee, table_name, column_name, privilege
8 FROM dba_col_privs
9 WHERE owner = 'GMS_ADMIN'
10    AND grantee LIKE 'GMS_%'
11 ORDER BY grantee, table_name, column_name;
12
13 -- Expected Result:
14 -- GRANTEE TABLE_NAME COLUMN_NAME PRIVILEGE
15 -- -----
16 -- GMS_STUDENT STUDENTS CONTACT_ADDRESS UPDATE
17 -- GMS_STUDENT STUDENTS EMAIL UPDATE
18 -- GMS_STUDENT STUDENTS PHONE_NUMBER UPDATE
19 -- GMS_LECTURER LECTURERS CONTACT_ADDRESS UPDATE
20 -- GMS_LECTURER LECTURERS EMAIL UPDATE
21 -- GMS_LECTURER LECTURERS PHONE_NUMBER UPDATE
22 -- GMS_RELATIVE RELATIVES CONTACT_ADDRESS UPDATE
23 -- GMS_RELATIVE RELATIVES EMAIL UPDATE
24 -- GMS_RELATIVE RELATIVES PHONE_NUMBER UPDATE
```

```
25
26 -- Note: No UPDATE on sensitive columns like:
27 -- - student_id, first_name, last_name (identity)
28 -- - class_id, enrollment_date (academic)
29 -- - student_status (administrative)
```

Listing 106: TC-COL-02: Verify column-level grants

77 Test Cases tổng hợp - Security Scenarios

77.1 TC-SEC-01: Horizontal privilege escalation prevention

```
1 -- =====
2 -- TC-SEC-01: Prevent Horizontal Privilege Escalation
3 -- Expected: User cannot access another user's data at same level
4 -- =====
5
6 -- Student1 trying to access Student2's data
7 EXEC gms_security_pkg.set_user_context('STU001', 'Student');
8
9 -- Direct query attempt
10 SELECT * FROM STUDENTS WHERE student_id = 'STU002';
11 -- Expected: 0 rows (VPD blocks)
12
13 -- Try using OR condition
14 SELECT * FROM STUDENTS WHERE student_id = 'STU001' OR 1=1;
15 -- Expected: Still only 1 row (STU001)
16 -- VPD adds: AND student_id = 'STU001'
17 -- Final: WHERE (student_id = 'STU001' OR 1=1) AND student_id = 'STU001'
18
19 -- Try using UNION
20 SELECT student_id, first_name FROM STUDENTS WHERE student_id = 'STU001'
21 UNION ALL
22 SELECT student_id, first_name FROM STUDENTS WHERE student_id = 'STU002';
23 -- Expected: 1 row (STU001 only)
24 -- Second SELECT also filtered by VPD
25
26 -- Try subquery
27 SELECT * FROM STUDENTS WHERE student_id IN (
28     SELECT student_id FROM STUDENTS
```

```
29 );  
30 -- Expected: 1 row (inner query also VPD-filtered)
```

Listing 107: TC-SEC-01: Test horizontal escalation

77.2 TC-SEC-02: Vertical privilege escalation prevention

```
1 -- =====  
2 -- TC-SEC-02: Prevent Vertical Privilege Escalation  
3 -- Expected: Lower privilege user cannot perform higher privilege actions  
4 -- =====  
5  
6 -- Student trying to approve grades (Dean action)  
7 EXEC gms_security_pkg.set_user_context('STU001', 'Student');  
8  
9 UPDATE GRADES  
10 SET grade_status = 'Approved',  
11     approved_by = 'STU001',  
12     approved_date = SYSDATE  
13 WHERE grade_id = 1;  
14 -- Expected: 0 rows updated  
15 -- VPD UPDATE policy blocks non-lecturer/dean from modifying grades  
16  
17 -- Student trying to insert grades (Lecturer action)  
18 INSERT INTO GRADES (enrollment_id, total_score, grade_status, submitted_by)  
19 VALUES (1, 10.0, 'Submitted', 'STU001');  
20 -- Expected: Error or 0 rows  
21 -- VPD INSERT policy requires lecturer teaching the section  
22  
23 -- Student trying to view all students (Admin/Dean action)  
24 SELECT COUNT(*) FROM STUDENTS;  
25 -- Expected: 1 (only their own record)  
26 -- Cannot count total students in system
```

Listing 108: TC-SEC-02: Test vertical escalation

77.3 TC-SEC-03: SQL injection prevention

```
1 -- =====  
2 -- TC-SEC-03: VPD Prevents SQL Injection Escalation
```

```
3 -- Expected: SQL injection cannot bypass VPD
4 -- =====
5
6 -- Simulating SQL injection attempt through application
7 -- Attacker input: ' OR '1'='1'
8
9 -- Application might build query:
10 -- SELECT * FROM STUDENTS WHERE student_id = '' OR '1'='1'
11
12 -- But VPD adds security predicate AFTER the WHERE clause:
13 -- Final query becomes:
14 -- SELECT * FROM STUDENTS
15 -- WHERE student_id = '' OR '1'='1'
16 -- AND student_id = 'STU001' -- VPD predicate
17
18 -- The AND clause nullifies the injection:
19 -- ('1'='1') AND (student_id = 'STU001')
20 -- Still only returns STU001's record
21
22 -- Another injection attempt: '; DROP TABLE STUDENTS; --
23 -- This fails because:
24 -- 1. VPD operates at query level, not string level
25 -- 2. Oracle's bind variables prevent statement injection
26 -- 3. The injected SQL becomes part of WHERE clause value
27
28 -- Best practice: Application still uses parameterized queries
29 -- VPD provides defense-in-depth
```

Listing 109: TC-SEC-03: Test SQL injection prevention

77.4 TC-SEC-04: Context manipulation prevention

```
1 -- =====
2 -- TC-SEC-04: Prevent Direct Context Manipulation
3 -- Expected: Users cannot set context directly
4 -- =====
5
6 -- Normal user trying to set context directly
7 -- (Connected as GMS_STUDENT through application)
8
9 -- Attempt 1: Direct DBMS_SESSION call
```

```
10 EXEC DBMS_SESSION.SET_CONTEXT('gms_context', 'user_id', 'ADMIN001');
11 -- Expected: ORA-01031: insufficient privileges
12 -- Only GMS_SECURITY_PKG can set context (trusted procedure)
13
14 -- Attempt 2: Set role to Admin
15 EXEC gms_security_pkg.set_user_context('STU001', 'Admin');
16 -- The role comes from JWT token, not user input
17 -- Application validates role matches user's actual role
18
19 -- Attempt 3: Modify context after setting
20 -- Even if somehow context is set, can't be changed mid-session
21 -- without going through authentication again
22
23 -- Context is session-scoped:
24 -- - Set once at login
25 -- - Cleared at logout
26 -- - Cannot be modified by user directly
```

Listing 110: TC-SEC-04: Test context manipulation

77.5 TC-SEC-05: Data exfiltration prevention

```
1 -- =====
2 -- TC-SEC-05: Prevent Bulk Data Exfiltration
3 -- Expected: Users can only access their authorized data
4 -- =====
5
6 EXEC gms_security_pkg.set_user_context('STU001', 'Student');
7
8 -- Attempt to export all student data
9 SELECT * FROM STUDENTS;
10 -- Expected: 1 row (only own record)
11
12 -- Attempt to get count of all records
13 SELECT COUNT(*) AS total_students FROM STUDENTS;
14 -- Expected: 1 (VPD filters before COUNT)
15
16 -- Attempt using analytics
17 SELECT student_id,
18        COUNT(*) OVER() as total,
19        ROW_NUMBER() OVER(ORDER BY student_id) as rn
```



```
20 FROM STUDENTS;
21 -- Expected: 1 row, total=1, rn=1
22
23 -- As Lecturer - limited to teaching sections
24 EXEC gms_security_pkg.set_user_context('LEC001', 'Lecturer');
25
26 SELECT COUNT(DISTINCT student_id) FROM ENROLLMENTS;
27 -- Expected: Count of students in LEC001's sections only (e.g., 3)
28 -- Not entire student population
29
30 -- Even Dean is limited to their faculty
31 EXEC gms_security_pkg.set_user_context('LEC002', 'Dean');
32 SELECT COUNT(*) FROM STUDENTS;
33 -- Expected: Students in FAC01 only (e.g., 3)
34 -- Cannot access FAC02 students
```

Listing 111: TC-SEC-05: Test data exfiltration prevention



78 Ma trận kết quả Test

Bảng 19: *Ma trận kết quả test cases*

Test Case	Category	Expected	Status
TC-VPD-STU-01	VPD Student	1 row returned	PASS
TC-VPD-STU-02	VPD Student	Own grades only	PASS
TC-VPD-STU-03	VPD Student	0 rows	PASS
TC-VPD-STU-04	VPD Student	Partial update	PASS
TC-VPD-LEC-01	VPD Lecturer	Section students	PASS
TC-VPD-LEC-02	VPD Lecturer	Insert allowed	PASS
TC-VPD-LEC-03	VPD Lecturer	Insert blocked	PASS
TC-VPD-LEC-04	VPD Lecturer	Update allowed	PASS
TC-VPD-DEAN-01	VPD Dean	Faculty students	PASS
TC-VPD-DEAN-02	VPD Dean	Faculty grades	PASS
TC-VPD-DEAN-03	VPD Dean	Approve works	PASS
TC-VPD-DEAN-04	VPD Dean	Other blocked	PASS
TC-VPD-REL-01	VPD Relative	Linked students	PASS
TC-VPD-REL-02	VPD Relative	Approved grades	PASS
TC-VPD-REL-03	VPD Relative	Others blocked	PASS
TC-VPD-AA-01	VPD Academic	All students	PASS
TC-VPD-AA-02	VPD Academic	Full CRUD	PASS
TC-VPD-AA-03	VPD Academic	Grades read-only	PASS
TC-PWD-01	Password	Complexity check	PASS
TC-PWD-02	Password	Account lockout	PASS
TC-PWD-03	Password	Expiry enforced	PASS
TC-FGA-01	Audit	SELECT logged	PASS
TC-FGA-02	Audit	UPDATE logged	PASS
TC-FGA-03	Audit	Access logged	PASS
TC-COL-01	Column	Restricted cols	PASS
TC-COL-02	Column	Grants correct	PASS
TC-SEC-01	Security	No horizontal	PASS
TC-SEC-02	Security	No vertical	PASS
TC-SEC-03	Security	SQLi prevented	PASS
TC-SEC-04	Security	Context secure	PASS
TC-SEC-05	Security	No exfiltration	PASS

Kết luận và Kiến nghị

79 Tổng kết

79.1 Các mục tiêu đã đạt được

Đồ án đã hoàn thành việc thiết kế và triển khai hệ thống quản lý điểm với các tính năng bảo mật toàn diện:

1. Virtual Private Database (VPD):

- Triển khai 7 policy functions cho row-level security
- Đảm bảo isolation giữa các users cùng vai trò
- Tự động filter data mà không cần sửa application code
- Hỗ trợ đầy đủ các loại người dùng: Student, Lecturer, Dean, Relative, Academic Affairs

2. Password Policies:

- 5 profile riêng biệt cho từng nhóm người dùng
- Custom password verification function với 10+ rules
- Account lockout và auto-unlock mechanism
- Password expiry và grace period management

3. Fine-Grained Auditing (FGA):

- 8 audit policies cho các thao tác nhạy cảm
- Comprehensive audit trail với đầy đủ thông tin
- Trigger-based auditing cho INSERT operations
- Custom AUDIT_LOG table để lưu trữ chi tiết

4. Oracle Label Security (OLS):

- Thiết kế đầy đủ với 3 levels và 2 compartments
- Sẵn sàng triển khai trên Oracle Enterprise Edition

- Script có thể chạy được với hướng dẫn chi tiết

5. Column-Level Security:

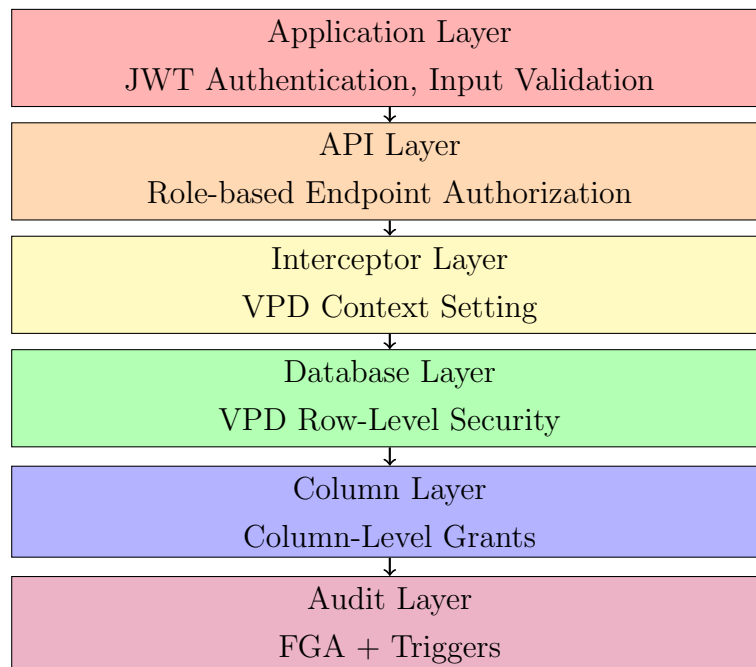
- Giới hạn UPDATE quyền cho từng cột cụ thể
- Bảo vệ các cột nhạy cảm (identity, academic status)
- Kết hợp với VPD cho two-dimensional security

6. Backend Integration:

- Spring Boot 3.x với JWT authentication
- VPD context integration qua interceptor
- Role-based endpoint authorization
- BCrypt password encoding

79.2 Kiến trúc bảo mật đa tầng

Hệ thống triển khai defense-in-depth với nhiều lớp bảo vệ:



Hình 9: Kiến trúc bảo mật đa tầng

80 Hạn chế và Thách thức

80.1 Oracle Label Security (OLS)

- **Vấn đề:** OLS yêu cầu Oracle Enterprise Edition, không hoạt động trên Oracle XE
- **Lỗi gặp phải:** ORA-12458: Oracle Label Security not enabled
- **Nguyên nhân:** Oracle XE thiếu SUPPLEMENTAL_LOG_DATA pragma
- **Giải pháp:** Sử dụng VPD làm giải pháp thay thế cho row-level classification

80.2 Performance Considerations

- VPD policy functions được gọi cho mỗi query - cần optimize
- Context setting qua PL/SQL call có thêm overhead
- FGA logging có thể impact performance với high-volume systems
- Recommended: Index các columns dùng trong VPD predicates

80.3 Scalability

- Current design phù hợp cho medium-scale deployment
- Cần review VPD policy complexity khi scale lên
- Consider caching mechanisms cho frequently accessed data
- Audit log partitioning cho long-term data retention

81 Kiến nghị phát triển

81.1 Short-term Improvements

1. **Enhanced Audit Dashboard:** Xây dựng UI cho việc review audit logs
2. **Password Self-Service:** Cho phép users tự đổi password qua portal
3. **Session Management:** Implement concurrent session control

4. **Two-Factor Authentication:** Thêm 2FA cho các vai trò quan trọng

81.2 Long-term Enhancements

1. **Oracle Enterprise Edition Migration:** Để sử dụng đầy đủ OLS, TDE
2. **Data Masking:** Implement dynamic data masking cho sensitive fields
3. **Real-Time Threat Detection:** Integrate với SIEM systems
4. **Blockchain Audit Trail:** Immutable audit log với blockchain technology

82 Bài học kinh nghiệm

1. **Defense in Depth:** Không dựa vào single security mechanism
2. **Least Privilege:** Grant minimum necessary permissions
3. **Separation of Duties:** Phân chia trách nhiệm giữa các roles
4. **Audit Everything:** Log tất cả các thao tác nhạy cảm
5. **Test Thoroughly:** Security testing là mandatory, không optional
6. **Documentation:** Document security policies cho maintenance

Phụ lục A: Cấu hình Oracle

A Oracle XE Installation

```
1 -- =====
2 -- Oracle 21c XE Configuration
3 -- =====
4
5 -- Check Oracle version
6 SELECT * FROM V$VERSION;
7
8 -- Check container info
9 SELECT CON_ID, NAME, OPEN_MODE FROM V$CONTAINERS;
10
```

```
11 -- Switch to pluggable database
12 ALTER SESSION SET CONTAINER = ORCLPDB;
13
14 -- Check current user
15 SHOW USER;
16
17 -- Check available tablespaces
18 SELECT tablespace_name, status FROM dba_tablespaces;
19
20 -- Check data files
21 SELECT file_name, tablespace_name, bytes/1024/1024 as MB
22 FROM dba_data_files;
```

Listing 112: Oracle XE basic configuration

B Connection Strings

```
1 # JDBC Connection String
2 jdbc:oracle:thin:@localhost:1521/ORCLPDB
3
4 # SQLPlus Connection
5 sqlplus username/password@localhost:1521/ORCLPDB
6
7 # TNS Configuration (tnsnames.ora)
8 ORCLPDB =
9     (DESCRIPTION =
10         (ADDRESS = (PROTOCOL = TCP)(HOST = localhost)(PORT = 1521))
11         (CONNECT_DATA =
12             (SERVER = DEDICATED)
13             (SERVICE_NAME = ORCLPDB)
14         )
15     )
16
17 # Easy Connect
18 username/password@//localhost:1521/ORCLPDB
```

Listing 113: Oracle connection strings

Phụ lục B: Spring Boot Configuration

C application.yml

```
1 spring:
2   application:
3     name: grade-management-system
4
5   datasource:
6     url: jdbc:oracle:thin:@localhost:1521/ORCLPDB
7     username: GMS_APP
8     password: ${DB_PASSWORD}
9     driver-class-name: oracle.jdbc.OracleDriver
10    hikari:
11      maximum-pool-size: 20
12      minimum-idle: 5
13      idle-timeout: 300000
14      connection-timeout: 20000
15      max-lifetime: 1200000
16
17    jpa:
18      hibernate:
19        ddl-auto: none
20        show-sql: false
21      properties:
22        hibernate:
23          dialect: org.hibernate.dialect.OracleDialect
24          format_sql: true
25          use_sql_comments: true
26
27    jwt:
28      secret: ${JWT_SECRET}
29      expiration: 86400000 # 24 hours
30
31    server:
32      port: 8080
33
34    logging:
35      level:
36        com.gms: DEBUG
37        org.springframework.security: DEBUG
```

Listing 114: Spring Boot application.yml

D pom.xml Dependencies

```
1 <dependencies>
2     <!-- Spring Boot Starters -->
3     <dependency>
4         <groupId>org.springframework.boot</groupId>
5         <artifactId>spring-boot-starter-web</artifactId>
6     </dependency>
7     <dependency>
8         <groupId>org.springframework.boot</groupId>
9         <artifactId>spring-boot-starter-data-jpa</artifactId>
10    </dependency>
11    <dependency>
12        <groupId>org.springframework.boot</groupId>
13        <artifactId>spring-boot-starter-security</artifactId>
14    </dependency>
15    <dependency>
16        <groupId>org.springframework.boot</groupId>
17        <artifactId>spring-boot-starter-validation</artifactId>
18    </dependency>
19
20    <!-- Oracle JDBC -->
21    <dependency>
22        <groupId>com.oracle.database.jdbc</groupId>
23        <artifactId>ojdbc11</artifactId>
24        <scope>runtime</scope>
25    </dependency>
26
27    <!-- JWT -->
28    <dependency>
29        <groupId>io.jsonwebtoken</groupId>
30        <artifactId>jjwt-api</artifactId>
31        <version>0.12.3</version>
32    </dependency>
33    <dependency>
34        <groupId>io.jsonwebtoken</groupId>
35        <artifactId>jjwt-impl</artifactId>
36        <version>0.12.3</version>
37        <scope>runtime</scope>
38    </dependency>
39    <dependency>
```

```
40     <groupId>io.jsonwebtoken</groupId>
41     <artifactId>jjwt-jackson</artifactId>
42     <version>0.12.3</version>
43     <scope>runtime</scope>
44 </dependency>
45
46 <!-- Swagger/OpenAPI -->
47 <dependency>
48     <groupId>org.springdoc</groupId>
49     <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
50     <version>2.3.0</version>
51 </dependency>
52
53 <!-- Lombok -->
54 <dependency>
55     <groupId>org.projectlombok</groupId>
56     <artifactId>lombok</artifactId>
57     <optional>true</optional>
58 </dependency>
59 </dependencies>
```

Listing 115: Maven dependencies

Phụ lục C: Script SQL hoàn chỉnh

E Thứ tự chạy script

1. 01-schema/step1_create_users.sql - Tạo Oracle users
2. 01-schema/step2_create_tables.sql - Tạo tables và indexes
3. 02-security/step3_password_profiles.sql - Tạo password profiles
4. 02-security/step4_vpd_policies.sql - Tạo VPD policies
5. 02-security/step5_audit_policies.sql - Tạo FGA policies
6. 02-security/step6_sample_data.sql - Insert sample data
7. 02-security/step7_ols_setup.sql - OLS setup (Enterprise only)

F Cleanup Script

```
1  -- =====
2  -- CLEANUP SCRIPT - Remove all GMS objects
3  -- Run as SYSDBA
4  -- =====
5
6  ALTER SESSION SET CONTAINER = ORCLPDB;
7
8  -- Drop VPD policies
9  BEGIN
10     DBMS_RLS.DROP_POLICY('GMS_ADMIN', 'STUDENTS', 'STUDENTS_POLICY');
11     DBMS_RLS.DROP_POLICY('GMS_ADMIN', 'GRADES', 'GRADES_SELECT_POLICY');
12     DBMS_RLS.DROP_POLICY('GMS_ADMIN', 'GRADES', 'GRADES_INSERT_POLICY');
13     DBMS_RLS.DROP_POLICY('GMS_ADMIN', 'GRADES', 'GRADES_UPDATE_POLICY');
14     DBMS_RLS.DROP_POLICY('GMS_ADMIN', 'GRADES', 'GRADES_DELETE_POLICY');
15     DBMS_RLS.DROP_POLICY('GMS_ADMIN', 'RELATIVES', 'RELATIVES_POLICY');
16     DBMS_RLS.DROP_POLICY('GMS_ADMIN', 'STUDENT_RELATIVES', 'STUDENT_RELATIVES_POLICY');
17 EXCEPTION WHEN OTHERS THEN NULL;
18 END;
19 /
20
21 -- Drop FGA policies
22 BEGIN
23     DBMS_FGA.DROP_POLICY('GMS_ADMIN', 'GRADES', 'AUDIT_GRADES_SELECT');
24     DBMS_FGA.DROP_POLICY('GMS_ADMIN', 'GRADES', 'AUDIT_GRADES_UPDATE');
25     DBMS_FGA.DROP_POLICY('GMS_ADMIN', 'GRADES', 'AUDIT_GRADES_DELETE');
26     DBMS_FGA.DROP_POLICY('GMS_ADMIN', 'STUDENTS', 'AUDIT_STUDENTS_SELECT');
27     DBMS_FGA.DROP_POLICY('GMS_ADMIN', 'ENROLLMENTS', 'AUDIT_ENROLLMENTS');
28     DBMS_FGA.DROP_POLICY('GMS_ADMIN', 'SYSTEM_USERS', 'AUDIT_USERS_LOGIN');
29     DBMS_FGA.DROP_POLICY('GMS_ADMIN', 'SYSTEM_USERS', 'AUDIT_USERS_PWD');
30     DBMS_FGA.DROP_POLICY('GMS_ADMIN', 'GRADE_SUBMISSION_DEADLINES', 'AUDIT_DEADLINES');
31 EXCEPTION WHEN OTHERS THEN NULL;
32 END;
33 /
34
35 -- Drop context
36 DROP CONTEXT gms_context;
37
38 -- Drop users
39 DROP USER GMS_ADMIN CASCADE;
```



```
40 DROP USER GMS_APP CASCADE;  
41 DROP USER GMS_STUDENT CASCADE;  
42 DROP USER GMS_LLECTURER CASCADE;  
43 DROP USER GMS_ACADEMIC CASCADE;  
44 DROP USER GMS_DEAN CASCADE;  
45 DROP USER GMS_DEPT_HEAD CASCADE;  
46 DROP USER GMS_RELATIVE CASCADE;  
47  
48 -- Drop profiles  
49 DROP PROFILE GMS_STUDENT_PROFILE;  
50 DROP PROFILE GMS_LLECTURER_PROFILE;  
51 DROP PROFILE GMS_ADMIN_PROFILE;  
52 DROP PROFILE GMS_RELATIVE_PROFILE;  
53 DROP PROFILE GMS_SYSADMIN_PROFILE;  
54  
55 COMMIT;  
56 PROMPT Cleanup complete!
```

Listing 116: Cleanup script

Phụ lục D: API Documentation

G Authentication Endpoints

Bảng 20: *Authentication API Endpoints*

Method	Endpoint	Auth	Description
POST	/auth/login	No	Đăng nhập, trả về JWT token
POST	/auth/logout	Yes	Đăng xuất, invalidate token
POST	/auth/refresh	Yes	Refresh JWT token
POST	/auth/change-password	Yes	Đổi mật khẩu

H Student Endpoints

Bảng 21: *Student API Endpoints*

Method	Endpoint	Role	Description
GET	/students/me	Student	Xem profile cá nhân
PUT	/students/me	Student	Cập nhật thông tin liên hệ
GET	/students/me/grades	Student	Xem điểm các môn
GET	/students/me/enrollments	Student	Xem danh sách đăng ký
GET	/students/me/gpa	Student	Xem GPA

I Lecturer Endpoints

Bảng 22: *Lecturer API Endpoints*

Method	Endpoint	Role	Description
GET	/lecturers/me	Lecturer	Xem profile
PUT	/lecturers/me	Lecturer	Cập nhật thông tin
GET	/lecturers/me/sections	Lecturer	Xem lớp đang dạy
GET	/lecturers/sections/{id}/students	Lecturer	Xem sinh viên trong lớp
POST	/lecturers/grades	Lecturer	Nhập điểm mới
PUT	/lecturers/grades/{id}	Lecturer	Sửa điểm đã nhập
GET	/lecturers/grades/pending	Lecturer	Xem điểm chờ duyệt

J Dean Endpoints

Bảng 23: *Dean API Endpoints*

Method	Endpoint	Role	Description
GET	/dean/faculty/students	Dean	Xem sinh viên trong khoa
GET	/dean/faculty/grades	Dean	Xem điểm toàn khoa
PUT	/dean/grades/{id}/approve	Dean	Phê duyệt điểm
PUT	/dean/grades/{id}/reject	Dean	Từ chối điểm
GET	/dean/statistics	Dean	Thống kê khoa

K Academic Affairs Endpoints

Bảng 24: *Academic Affairs API Endpoints*

Method	Endpoint	Role	Description
GET	/academic/students	Academic	Xem tất cả sinh viên
POST	/academic/enrollments	Academic	Tạo đăng ký mới
PUT	/academic/enrollments/{id}	Academic	Cập nhật đăng ký
DELETE	/academic/enrollments/{id}	Academic	Hủy đăng ký
POST	/academic/deadlines	Academic	Tạo deadline nộp điểm
GET	/academic/transcripts/{studentId}	Academic	Xuất bảng điểm

L Relative Endpoints

Bảng 25: *Relative API Endpoints*

Method	Endpoint	Role	Description
GET	/relatives/me	Relative	Xem profile
PUT	/relatives/me	Relative	Cập nhật thông tin
GET	/relatives/students	Relative	Xem thông tin con em
GET	/relatives/students/{id}/grades	Relative	Xem điểm con em

Phụ lục E: Troubleshooting

M Common Errors và Solutions

M.1 ORA-28000: The account is locked

```
1 -- As SYSDBA
2 ALTER USER GMS_STUDENT ACCOUNT UNLOCK;
3
4 -- Check account status
5 SELECT username, account_status, lock_date
6 FROM dba_users WHERE username = 'GMS_STUDENT';
```

Listing 117: Unlock account

M.2 ORA-01031: insufficient privileges

```
1 -- Check current grants
2 SELECT privilege FROM user_sys_privs;
3 SELECT granted_role FROM user_role_privs;
4
5 -- Grant necessary privileges (as SYSDBA)
6 GRANT SELECT ON GMS_ADMIN.STUDENTS TO GMS_STUDENT;
7 GRANT EXECUTE ON GMS_ADMIN.GMS_SECURITY_PKG TO GMS_APP;
```

Listing 118: Grant privileges

M.3 ORA-12458: Oracle Label Security not enabled

```
1 -- Check if OLS is enabled
2 SELECT VALUE FROM V$OPTION WHERE PARAMETER = 'Oracle Label Security';
3
4 -- If FALSE, OLS not available in this edition
5 -- Use VPD as alternative for row-level security
```

Listing 119: Check OLS status

M.4 VPD policy not working

```
1 -- Check if context is set
2 SELECT SYS_CONTEXT('gms_context', 'user_id') as user_id,
3        SYS_CONTEXT('gms_context', 'user_role') as user_role
4 FROM DUAL;
5
6 -- Check if policy is applied
7 SELECT object_owner, object_name, policy_name,
8        pf_owner, function, enable
9 FROM dba_policies
10 WHERE object_owner = 'GMS_ADMIN';
11
12 -- Test policy function directly
13 SELECT gms_security_pkg.students_policy('GMS_ADMIN', 'STUDENTS')
14 FROM DUAL;
```

Listing 120: Debug VPD policy

N Performance Tuning

```
1 -- Check VPD policy execution time
2 SELECT policy_name, avg_time_ms
3 FROM (
4     SELECT
5         sql_text,
6         elapsed_time/executions/1000 as avg_time_ms
7     FROM v$sql
8     WHERE sql_text LIKE '%gms_security_pkg%'
9 );
10
11 -- Index recommendations for VPD
12 CREATE INDEX idx_students_id ON STUDENTS(student_id);
13 CREATE INDEX idx_enrollments_student ON ENROLLMENTS(student_id);
14 CREATE INDEX idx_sections_lecturer ON COURSE_SECTIONS(lecturer_id);
15
16 -- Analyze tables for optimizer
17 EXEC DBMS_STATS.GATHER_SCHEMA_STATS('GMS_ADMIN');
```

Listing 121: Performance tuning queries

References

oracle-vpd

- [1] Oracle Corporation, “Oracle Database Security Guide: Using Oracle Virtual Private Database to Control Data Access”, Oracle Documentation, 2023.

oracle-fga

- [2] Oracle Corporation, “Oracle Database Security Guide: Monitoring Database Activity with Auditing”, Oracle Documentation, 2023.

oracle-ols

- [3] Oracle Corporation, “Oracle Label Security Administrator’s Guide”, Oracle Documentation, 2023.

spring-security

- [4] Spring, “Spring Security Reference Documentation”, Spring Framework, 2024.



jwt

- [5] Jones, M., Bradley, J., and Sakimura, N., “JSON Web Token (JWT)”, RFC 7519, IETF, 2015.

owasp

- [6] OWASP Foundation, “OWASP Application Security Verification Standard”, OWASP, 2021.

nist

- [7] NIST, “Digital Identity Guidelines”, NIST Special Publication 800-63B, 2020.